TITLE PAGE

TO BE FORMATTED PER UNIVERSITY TEMPLATE

Final Project Report

HealthWise AI: AI-Powered Hospital Management and Patient Health Report Analysis and History Tracking System

[Student name(s) and enrolment number(s): to be filled in by the author]

[Programme and branch: the repository README names a B.Tech Data Science Engineering project; to be confirmed]

[Guide name and designation: to be filled in by the author]

[Institution and university name: to be filled in by the author]

[Month and year of submission: to be filled in by the author]

CERTIFICATE

TO BE FORMATTED PER UNIVERSITY TEMPLATE

[Certificate text, signatures of guide, head of department and examiners: to be provided by the university template]

DECLARATION

TO BE FORMATTED PER UNIVERSITY TEMPLATE

[Declaration text and signature of the student(s): to be provided by the university template]

ACKNOWLEDGEMENT

TO BE FORMATTED PER UNIVERSITY TEMPLATE

[Acknowledgement text: to be written by the author]

TABLE OF CONTENTS

TO BE FORMATTED PER UNIVERSITY TEMPLATE

1. INTRODUCTION 1

1.1 Overview of the Healthcare and Digital-Records Domain 1

1.2 Reasons for Choosing the Project 1

1.3 Core Motivation 2

1.4 High-Level Objectives 2

1.5 Organisation of the Report 3

2. PRESENT PRACTICES AND TECHNOLOGY 4

2.1 Existing System and Analysis 4

2.1.1 Patient records 4

2.1.2 Reports 4

2.1.3 Prescriptions and pharmacy billing 4

2.1.4 Bottlenecks identified 5

2.2 Problem Statement 5

2.3 Proposed System 6

2.3.1 Unique value 6

3. INFORMATION GATHERING 8

3.1 Techniques Used 8

3.1.1 Literature and standards review 8

3.1.2 Analysis of existing hospital software and report formats 8

3.1.3 Sample reports and document formats 9

3.1.4 Iterative review with the author 9

3.2 Information Not Gathered 10

3.3 Resulting Requirement Sources 10

4. PROJECT SCOPE 11

4.1 In-Scope Features 11

4.2 Out-of-Scope Items 12

5. REQUIREMENT ANALYSIS AND FEASIBILITY STUDY 13

5.1 Requirement Analysis 13

5.1.1 Functional requirements 13

5.1.2 Non-functional requirements 16

5.2 Feasibility Study 17

5.2.1 Technical feasibility 17

5.2.2 Economic feasibility 17

5.2.3 Operational feasibility 18

6. DESIGN AND PLANNING 19

6.1 System Design and Analysis 19
6.1.1 Architecture 19
6.1.2 Data flow diagrams 20
6.1.3 Use-case diagrams 22
6.1.4 Class diagram 27
6.1.5 Entity-relationship diagram 28
6.1.6 Sequence diagrams 29
6.2 Effort Estimation 33
6.2.1 Timeline and phases from the git history 33
6.2.2 Size and COCOMO estimate 35
6.2.3 Interpretation and points to confirm 36
7. IMPLEMENTATION AND CODING 37
7.1 Implementation 37
7.1.1 Backend modules 37
7.1.2 Frontend modules 38
7.1.3 Environment setup and technology stack 38
7.2 Coding 40
7.2.1 Report extraction and abnormal-value flagging 40
7.2.2 Role-based access checks 42
7.2.3 Password and validation rules 43
7.2.4 Prescription countersign and dispensing 44
7.2.5 Assistant retrieval and the 'never invent values' guardrails 45
7.2.6 Bed and stock transaction handling 46
8. TESTING 48
8.1 Testing Strategy 48
8.2 Test Environment and Method 48
8.3 Results 49
8.4 Test Cases 50
8.5 Bust and Chaos Findings and Their Fixes 54
8.6 Observations and Limits of the Testing 55
9. RISK ANALYSIS 57
10. LIMITATION AND FUTURE SCOPE 59
10.1 Limitations 59
10.1.1 Clinical and legal status 59
10.1.2 Technical limits 59
10.1.3 Testing and process limits 60
10.2 Future Scope 60
11. USER MANUAL 62
11.1 Installing and Running the Software 62

11.1.1 Requirements 62
11.1.2 Starting the backend 62
11.1.3 Starting the frontend 62
11.1.4 Environment variables 63
11.1.5 Seed data and demonstration accounts 63
11.2 Logging In 64
11.3 Patient 65
11.3.1 Dashboard 65
11.3.2 My Reports and report detail 65
11.3.3 My Medicines 66
11.3.4 Ask HealthWise 67
11.3.5 My Timeline 68
11.4 Doctor 68
11.4.1 Dashboard 68
11.4.2 Reviewing a report 69
11.4.3 Patient page and prescriptions 70
11.5 Reception 71
11.6 Pharmacy 73
11.7 Laboratory 75
11.8 Administrator 75
12. BIBLIOGRAPHY 78

LIST OF FIGURES

TO BE FORMATTED PER UNIVERSITY TEMPLATE

Figure 6.1: High-Level System Architecture 20
Figure 6.2: Data Flow Diagram, Level 0 (Context Diagram) 21
Figure 6.3: Data Flow Diagram, Level 1 22
Figure 6.4: Use-Case Diagram for the Patient 23
Figure 6.5: Use-Case Diagram for the Doctor 24
Figure 6.6: Use-Case Diagram for Reception (Employee) 24
Figure 6.7: Use-Case Diagram for Pharmacy Staff 25
Figure 6.8: Use-Case Diagram for Laboratory and Radiology Staff 26
Figure 6.9: Use-Case Diagram for the Administrator 26
Figure 6.10: Class Diagram of the Main Entities 27
Figure 6.11: Entity-Relationship Diagram (Selected Tables) 29
Figure 6.12: Sequence Diagram: Login and Token Use 30
Figure 6.13: Sequence Diagram: Report Upload, Analysis and Review 31
Figure 6.14: Sequence Diagram: Prescription to Pharmacy 32
Figure 6.15: Sequence Diagram: Patient Assistant Question 33
Figure 6.16: Gantt Chart of the Development Phases from the Git History 34
Figure 7.1: Report Processing Pipeline 40
Figure 11.1: Login page 64
Figure 11.2: Patient dashboard 65
Figure 11.3: Patient report list 66
Figure 11.4: A reviewed report as the patient sees it 66
Figure 11.5: Patient prescriptions with dispensing status 67
Figure 11.6: Patient assistant answering from a reviewed report 67
Figure 11.7: Patient timeline 68
Figure 11.8: Doctor dashboard 69
Figure 11.9: Report review page for the doctor 69
Figure 11.10: Doctor's patient page 70
Figure 11.11: Prescriptions written by the doctor 70
Figure 11.12: Reception dashboard 71
Figure 11.13: Patient registration form 71
Figure 11.14: Report upload page 72
Figure 11.15: OPD desk 72
Figure 11.16: Pharmacy dashboard 73
Figure 11.17: Pharmacy prescription list 73

Figure 11.18: Dispensing a prescription 74
Figure 11.19: Medicine stock 74
Figure 11.20: Laboratory work list (empty state) 75
Figure 11.21: Administrator dashboard 76
Figure 11.22: People list 76
Figure 11.23: Activity (audit) log 77
Figure 11.24: Hospital set-up 77

LIST OF TABLES

TO BE FORMATTED PER UNIVERSITY TEMPLATE

Table 1.1: Project objectives and the components that address them 3
Table 2.1: Bottlenecks in present practice 5
Table 2.2: Problem gaps and the features that address them 7
Table 3.1: Sample documents used for information gathering and testing 9
Table 3.2: Sources of requirement groups 10
Table 4.1: Modules in scope 11
Table 4.2: Out-of-scope items 12
Table 5.1: Functional requirements 13
Table 5.2: Non-functional requirements 16
Table 5.3: Indicative cost items 18
Table 6.1: Development phases and commits 34
Table 6.2: Basic COCOMO results for 32.76 KLOC 36
Table 7.1: Backend modules by function 37
Table 7.2: Frontend modules 38
Table 7.3: Technology stack and versions 39
Table 8.1: Testing levels 48
Table 8.2: Automated test results of 4 November 2026 49
Table 8.3: Backend tests by area 50
Table 8.4: Summary test cases 50
Table 8.5: Main findings of the bust report and their status 55
Table 9.1: Risk register 57
Table 10.1: Future scope 61
Table 11.1: Main environment variables 63
Table 11.2: Demonstration accounts (password `Demo@12345`) 63

CHAPTER 1

1. INTRODUCTION

1.1 Overview of the Healthcare and Digital-Records Domain

A hospital produces information at every step of a patient's care: registration details, appointments, examination notes, laboratory and imaging reports, prescriptions, dispensed medicines, bills and the final discharge summary. Each item is created by a different person, in a different department, and for a different purpose. The value of this information depends on how quickly and how reliably it can be found again, and on whether the right person, and only the right person, can see it. Electronic health records address this problem by keeping the information of one patient in one structured store, so that a clinician can read a coherent history instead of assembling it from separate files [1].

Digital records have received sustained policy attention. The World Health Organization has described digital health as a means of strengthening health systems and has published a global strategy for the period 2020 to 2025 [2]. In India, the Ayushman Bharat Digital Mission promotes interoperable health records and health identifiers [3], and the Digital Personal Data Protection Act, 2023 sets the legal duties of organisations that process personal data [4]. At the technical level, standards such as HL7 FHIR describe how clinical data can be exchanged between systems [5]. Together these developments raise the expected quality of any hospital information system: it must be structured, interoperable, auditable and careful with personal data.

A large share of the clinical evidence that reaches a doctor is still a document, usually a laboratory report issued as a PDF or a printed sheet. Such a report holds numbers, units and printed reference ranges, but not in a form that software can compare, trend or explain. Patients, in turn, receive the same document and are expected to interpret values such as haemoglobin, HbA1c or serum creatinine without help, often in a language that is not their first. These two facts, unstructured report documents and an uneven understanding of their content, define the domain problem that this project addresses.

1.2 Reasons for Choosing the Project

HealthWise AI was chosen because it combines four areas that a final-year data science engineering project can address in one product: reading documents with text extraction and optical character recognition, applying deterministic rules to extracted

data, using language models in a controlled and auditable way, and building a multi-role web application with a real security model. The repository README identifies the work as a B.Tech Data Science Engineering final-year project; the programme name should be confirmed against the university template.

The project is also suited to honest engineering. A health-related system cannot be judged only by whether it runs; it must be judged by whether it refuses to guess. This requirement shaped the design from the first commit: values are read from the actual file, ranges come only from the report itself, an artificial intelligence component may reword an explanation but never decide a status, and every clinical output passes through a doctor before the patient sees it.

1.3 Core Motivation

The motivation follows from the gaps observed in the way reports and records are handled in a typical outpatient and inpatient workflow, which Chapter 2 analyses in detail. In summary:

- A laboratory report is a static document; a doctor has to read each value manually and compare it with earlier reports from memory or from a folder of files.
- A patient cannot reliably tell whether a value needs attention, and a report in one language is of limited use to a family that reads another.
- Reports, prescriptions, dispensing and bills are recorded separately, so no single view of a patient exists.
- Access to patient data is frequently decided by physical custody of a file rather than by an explicit rule, and actions are rarely recorded.

The project therefore aims to show that a small, carefully constrained piece of software can reduce these gaps without presenting itself as a diagnostic tool.

1.4 High-Level Objectives

The objectives were set at the start of the project and extended as the system grew from a report-analysis tool into a hospital management system. Table 1.1 lists them with the part of the implementation that addresses each one.

Table 1.1: Project objectives and the components that address them

No.	Objective	Where it is realised
O1	Extract real values, units and printed reference ranges from uploaded reports (text PDF, scanned PDF, image, text).	Report pipeline: `pdf_service`, `ocr_service`, `report_parser`
O2	Classify each value using only the range printed in the report, with a deterministic rule engine.	`analysis_engine.classify_status`
O3	Keep the doctor as the final authority: nothing clinical reaches the patient before review.	Doctor review workflow, `serializers`, patient portal
O4	Enforce role-based access and patient-level privacy, and record important actions in an audit log.	`core/permissions.py`, `audit_service`
O5	Maintain a complete patient history with comparison, trends and a searchable timeline.	History, comparison and clinical-intelligence services
O6	Connect prescriptions to pharmacy dispensing and stock, with safe handling of partial dispensing and expiry.	`prescription_service`, `dispensing_service`, `stock_service`
O7	Offer a patient assistant in English, Hindi and Marathi that answers only from the patient's own reviewed records.	`patient_assistant`, `assistant_safety`, voice front end
O8	Extend the platform to hospital workflows: OPD, nursing, emergency, wards and beds, diagnostics, billing, blood bank and operation theatre.	HMS phases 1 to 10 (Chapter 7)
O9	Demonstrate quality by automated tests, a red-team exercise and documented limits.	Chapter 8 and `docs/BUST_REPORT.md`

1.5 Organisation of the Report

Chapter 2 reviews present practice and states the problem and the proposed system. Chapter 3 describes how information was gathered. Chapter 4 fixes the scope, and Chapter 5 sets out the requirements and the feasibility study. Chapter 6 presents the design and the effort estimate, Chapter 7 the implementation, and Chapter 8 the testing carried out and its results. Chapter 9 analyses risks, Chapter 10 states the limitations and future scope, Chapter 11 is the user manual, and Chapter 12 lists the references.

CHAPTER 2

2. PRESENT PRACTICES AND TECHNOLOGY

2.1 Existing System and Analysis

This section describes how records, reports, prescriptions and pharmacy billing are commonly handled in smaller hospitals and clinics where digital tools are limited. The description is a synthesis of the repository's own inspection documents (`docs/HMS_PHASE1_AUDIT.md`, `docs/PHARMACY_UPGRADE_PLAN.md`) and of the document formats used in the sample reports; it is not based on a field study of a named hospital, and no hospital is named. Where practice is described, it is described as typical rather than as measured.

2.1.1 Patient records

In a paper-based or file-based setting, a patient has one or more physical or scanned files. The registration sheet, case history and visit notes are written by hand, and each visit adds a page. Searching for an earlier visit means turning pages; a second department keeps its own file. When software is used, it is frequently a registration or billing package that stores demographics and invoices but not clinical content. Two consequences follow. First, a duplicate registration of the same person is easy to create, and nothing prevents it. Second, there is no single chronological view that combines visits, reports, medicines and admissions.

2.1.2 Reports

Laboratory and imaging reports arrive as printed pages, PDF files or photographs of printed pages. A doctor reads the values, compares them with the printed range, and recalls or retrieves the previous report to judge a change. The report formats differ between laboratories: some print the range as 'a - b', others as '< x' or '> x', some print tiered interpretations such as 'Deficient < 20, Insufficient 20 to 30', some print separate ranges for men and women, and some print no range at all. A reader has to apply the right convention each time, and an automatic reader has to do the same without error. Section 7.2.1 and Chapter 8 show how this variety was handled and tested.

2.1.3 Prescriptions and pharmacy billing

A prescription is usually a handwritten sheet. The patient carries it to a pharmacy, where the pharmacist interprets the handwriting, picks stock and prepares a bill. The hospital has no direct knowledge of what was actually dispensed, whether a

prescription was dispensed twice, or whether the stock used was past its expiry date. In pharmacy software that does exist, stock is often a single quantity per medicine, changed directly, so that a mistake cannot be traced to a batch, a user or a time. The pharmacy inspection recorded in `docs/PHARMACY_UPGRADE_PLAN.md` found exactly this situation in an earlier version of the project: quantities were changed in several places without a ledger.

2.1.4 Bottlenecks identified

Table 2.1 summarises the bottlenecks that follow from the practices above.

Table 2.1: Bottlenecks in present practice

Area	Bottleneck	Effect on the patient or staff
Report language	Reports are issued in English with technical terms; no explanation or translation is attached.	Patients and families depend on the doctor or on informal help to understand a result.
Scattered records	Reports, notes, prescriptions and bills are held in different files or systems.	No single history; repeated tests and repeated questions.
Manual comparison	Earlier values are found by hand and compared by eye.	Trends are missed or noticed late; time is spent on retrieval rather than on judgement.
Unstructured prescriptions	Handwritten or free-text prescriptions.	Dispensing errors, no link between prescribed and dispensed quantity.
Stock control	Quantities changed without a ledger; expiry checked by eye.	Expired or recalled stock can be sold; shortages are found late.
Access control	Access is decided by who holds the file.	No rule limits staff to the data they need; no record of who looked at what.
Concurrency	Two staff acting on the same record at the same moment.	Double bookings, double payments and double dispensing.

2.2 Problem Statement

The specific gaps that the project addresses are the following.

- **No reliable machine reading of reports.** Values, units and printed ranges are not extracted in a form that supports comparison, and automatic readers risk misreading unusual layouts, for example a ratio line mistaken for the parameter itself or a tiered range mistaken for a normal range.
- **No explanation in the patient's language.** A patient cannot ask a question about a report or a prescription and receive an answer in English, Hindi or Marathi that is grounded in the patient's own records.
- **No single, role-controlled history.** Clinical, pharmacy and billing information is not combined in one record, and access is not tied to a clear role and patient relationship.

- **No safeguard on automated output.** Where artificial intelligence is applied to health documents, there is a risk that it invents values, changes a status or addresses the patient directly with a diagnosis. A design is needed that keeps the doctor in control.
- **Weak accountability.** Actions on records, stock and money are not recorded in a way that can be reviewed, and concurrent actions are not protected.

2.3 Proposed System

HealthWise AI is a web-based hospital management and report-analysis system. It was first built around one workflow, in which a reception employee uploads a report, the system reads and analyses it, a doctor reviews it, and the patient and the pharmacy then see what they are allowed to see. It was later extended with the clinical and administrative modules of a hospital. Only features that exist in the repository are described here; the exact scope is given in Chapter 4.

2.3.1 Unique value

- **Explained reports.** The system extracts values, units and printed ranges from PDFs, images (through optical character recognition when an OCR engine is installed) and text, assigns a status (within range, above, below or unknown) by deterministic rules, and produces plain-language statements for the doctor and general tips for the patient.
- **Doctor-reviewed AI.** The rules decide; an optional language model may only reword text, behind a safety filter. A patient sees analysis only after a doctor has reviewed the report, and doctor-only suggestions and red flags are never shown to the patient.
- **A patient assistant in three languages.** The assistant, available as text and as browser-based voice, answers questions from the signed-in patient's reviewed records in English, Hindi and Marathi, refuses medicine changes and diagnosis, and gives fixed emergency guidance first.
- **Role-based modules.** Patient, doctor (intern to consultant grades), reception, pharmacy, laboratory, radiology, nursing, billing, store, operation theatre, blood bank and administrator roles each see only their own screens and endpoints.
- **One history.** Appointments, reports, analysis, reviews, notes, prescriptions, dispensing, admissions and hospital-recorded results feed one searchable timeline.

- **Accountability.** An audit log records important actions; row locking protects money, stock, blood units and bookings from concurrent use; stock changes leave a ledger entry.

Table 2.2 relates the gaps of Section 2.2 to the corresponding features.

Table 2.2: Problem gaps and the features that address them

Gap	Corresponding feature in HealthWise AI
Machine reading of reports	Text, scan and image pipeline with a parser tested on six print layouts and on the real sample reports.
Explanation in the patient's language	Patient assistant with English, Hindi and Marathi wording; plain-language report statements.
Single, role-controlled history	Timeline and clinical summary; role and patient-relationship checks on every clinical endpoint.
Safeguard on automated output	Rule-based status, safety filter, doctor review before patient release, number-preservation check on AI wording.
Accountability	Audit log, stock ledger, row locks, strict database constraints for beds and admissions.

CHAPTER 3

3. INFORMATION GATHERING

This chapter states how the requirements were collected. It records only what the repository and the git history show. Where a normal information-gathering activity such as a survey or an interview is not documented, this is stated and the place for the author's own details is marked.

3.1 Techniques Used

3.1.1 Literature and standards review

The domain concepts and constraints were taken from public documentation and standards: the digital-health policy of the World Health Organization [2], the FHIR data model [5], the Digital Personal Data Protection Act, 2023 [4], the Web Content Accessibility Guidelines [6] and the OWASP Top 10 and Application Security Verification Standard [7, 8]. For the technical design, the documentation of FastAPI [9], React [10], Pydantic [11] and SQLAlchemy [12] was used, and retrieval-augmented generation [13] and the Tesseract engine [14] were studied as background for the assistant and the OCR path. The reasoning for the effort model is based on Boehm [15] and on standard software engineering texts [16, 17].

3.1.2 Analysis of existing hospital software and report formats

The existing state of the project itself was analysed twice in a structured way and the results were written down. First, `docs/PHARMACY_UPGRADE_PLAN.md` records an inspection of the pharmacy module with a baseline test run (125 tests passing at that time), a list of the rules already enforced and a gap table against a target pharmacy ERP. Second, `docs/HMS_PHASE1_AUDIT.md` records a read-only audit of the repository against a hospital management brief, module by module, with a status of working, partial or missing for each area (administration, registration, OPD, nursing, emergency, inpatient care, wards and beds, laboratory, radiology, operation theatre, pharmacy, blood bank, billing, insurance, procurement, discharge). These two documents served as the requirement baseline for the extension work of the final week of the project. They analyse the project's own software rather than a third-party product, and the report does not claim a comparison with named commercial hospital software.

3.1.3 Sample reports and document formats

The folder `sample_reports/` holds the synthetic documents used throughout development and testing. They were written for the project and contain no real patient data. Table 3.1 lists them.

Table 3.1: Sample documents used for information gathering and testing

File	Type	Purpose
Report_A_July_Rahul.pdf	Laboratory report (text PDF)	Earlier report of the demonstration patient; basis of comparison and trend.
Report_B_October_Rahul.pdf	Laboratory report (text PDF)	Later report; values differ so that changes between reports can be shown.
Report_A_scan.png	Image of a report	Tests the OCR path (requires Tesseract).
Chest_Xray_Rahul.pdf	Imaging report	Tests free-text statements, negation and urgency wording.
ECG_Rahul.txt	ECG report (text)	Tests another report type.
Discharge_Summary_Rahul.pdf	Discharge summary	Tests narrative documents.
Prescription_Rahul.txt	Prescription text	Tests prescription documents.
Report_C_other_patient.txt	Report of a different patient	Tests the wrong-patient warning.
Not_a_health_report.txt	Non-medical text	Must be rejected and not saved.

Beyond these files, the parser was exercised on six print layouts (plain, with a high or low flag, colon-separated, bracketed, 'Ref:' prefixed and pipe-separated tables), generated for 24 laboratory tests at low, normal and high values, giving 402 test values (`docs/TESTING_AND_HARDENING.md`). The tricky layouts met in practice, such as tiered ranges, ratio lines and sex-specific ranges, were collected as regression cases.

3.1.4 Iterative review with the author

The git history shows that requirements were refined in short iterations. The commit messages record, in order, the first report workflow, a review of usability for non-technical users, a pharmacy and billing extension, a patient history and assistant, a red-team exercise and its fixes, a validation rule set, doctor grades and the hospital phases. The phase documents (`docs/HMS_PHASE2.md` onwards) record decisions taken during these iterations, for example that the UHID format is branch, year and a running number. The decisions were taken in discussion between the author and the development assistant, as described in Section 6.2.3.

3.2 Information Not Gathered

The repository contains no record of a questionnaire, an interview with a clinician, a pharmacist or a patient, or an observation visit to a hospital. The report therefore makes no claim about the views of such users, and no survey result is presented.

[TO BE COMPLETED BY THE AUTHOR: if a survey, interview or visit was carried out outside the repository, describe here the method, the number and type of participants, the date and the main findings, with evidence. If none was carried out, this section stands as it is and the limitation is repeated in Chapter 10.]

3.3 Resulting Requirement Sources

Table 3.2 summarises how each group of requirements in Chapter 5 was derived.

Table 3.2: Sources of requirement groups

Requirement group	Source
Report extraction and analysis	Sample reports, layout test sets, bust-report findings T-02 to T-06.
Roles and permissions	Phase 1 audit, permission matrix in `docs/ARCHITECTURE.md`, endpoint matrix.
Pharmacy and stock	Pharmacy inspection and upgrade plan.
Clinical workflows (OPD, nursing, emergency, IPD)	Phase 1 audit gap table, phase documents 2 to 7.
Security and privacy	OWASP references, the bust report, `docs/TESTING_AND_HARDENING.md`.
Assistant behaviour	`docs/PATIENT_ASSISTANT.md`, golden question sets in three languages.
Usability	Iterative review of screens; accessibility checks with axe-core; mobile layout checks.

CHAPTER 4

4. PROJECT SCOPE

The scope below was checked against the code, the endpoint list and the integration status screen of the application (`GET /api/integrations/status`), so that nothing listed as in scope is merely planned and nothing listed as out of scope is in fact built.

4.1 In-Scope Features

Table 4.1 lists the modules that exist in the repository, with the roles that use them. The modules were built in the order shown in the git history (Section 6.2).

Table 4.1: Modules in scope

Module	Main features	Main roles
Authentication and accounts	Password login, mobile-code login, first-administrator set-up, temporary password change, lockout, profile and photo.	All
Report analysis	Upload of PDF, image, text; text extraction, OCR path, parsing, deterministic status, insights, doctor review, comparison and trends, printable report.	Reception, doctor, patient
Patient registry	Registration, UHID, duplicate check, consent, ABHA number storage, insurance record, merge of duplicates, patient photo.	Reception, administrator
Appointments and OPD	Booking with clash check, check-in and walk-in tokens, doctor queue, consultation with templates, signing and addenda.	Reception, doctor, patient
Doctor workspace	Patient summary, timeline, trends, notes, recommendations, prescriptions, past medical history, clinical assessment, referrals, countersign for junior doctors.	Doctor (four grades)
Nursing	Vitals with alert rules, medication administration record, intake and output, notes, handover.	Nurse
Emergency	Rapid registration including unidentified patients, triage categories, timeline, outcome and admission, patient emergency card.	Reception, doctor, nurse
Inpatient care	Wards and beds, admission, progress notes, medication orders, discharge summary and discharge.	Doctor, nurse, administrator
Laboratory and radiology	Test catalogue with hospital ranges, sample tracking, structured results, critical alerts, pathologist approval, radiology orders and reports, doctor-then-patient release.	Lab, radiology, doctor, patient
Pharmacy	Prescription search, full and partial dispensing, stock batches with expiry, barcode sales, GST bills, stock ledger, recall, write-off, returns.	Pharmacy
Store and procurement	Requisition, purchase order, goods receipt, stock count with second-person approval, ward medicine requests.	Store, pharmacy, nurse
Billing and claims	Invoices, charges, payments, advances, refunds with approval, packages, daily cash and revenue reports, insurance claims and pre-authorisation steps.	Billing, accountant, insurance desk

Module	Main features	Main roles
Operation theatre and blood bank	Theatre booking without overlap, pre-operative steps, notes and recovery; donors, screening, stock, cross-match, issue and transfusion.	OT staff, blood bank staff
Patient assistant	Chat and voice in English, Hindi and Marathi, answers from the patient's own reviewed records, safety rules.	Patient
Administration	Staff accounts, departments, hospital profile, branches, settings, price list, roles and permissions, break-glass review, audit log, integration status.	Administrator
Interoperability	E-mail outbox and reminders, FHIR R4-shaped read-only export marked unvalidated.	Administrator, doctor, patient

4.2 Out-of-Scope Items

Table 4.2 lists the items that are not built. Their state is taken from the integration status list in the code (`services/integration_service.py`), which labels each one so that the application itself does not claim more than it does.

Table 4.2: Out-of-scope items

Item	State in the application	Meaning for this project
Online payment gateway	NOT_BUILT	Payments are recorded by staff as cash, UPI, card or bank transfer; no money moves through the software.
DICOM and PACS	NOT_BUILT	Scan images are stored as JPG, PNG or PDF; no DICOM viewer or archive.
Real ABDM and ABHA exchange	NOT_BUILT	An ABHA number can be stored with consent; no gateway, consent manager or health-information exchange is connected.
Laboratory analyser link	NOT_BUILT	Results are entered by a technician and approved by a pathologist.
Insurance and TPA portal link	NOT_BUILT	Claims are processed step by step by staff inside the system.
SMS and WhatsApp delivery	ADAPTER_ONLY	Messages wait in an outbox; nothing is sent because no provider is connected. Mobile-code login shows the code on screen only in a demonstration mode.
FHIR export validation	BUILT_UNVERIFIED	A read-only bundle is produced but is not validated against a FHIR server or an ABDM profile.
Native store applications	Not built	The interface is a responsive web application; a Capacitor Android project is part of the repository (added on 4 November) and a debug build was made locally, but no store release was made (Chapter 10).
Diagnosis, disease prediction, medicine suggestion by the system	Excluded by design	The system is educational and supportive; clinical decisions remain with the doctor.
Telemedicine	Not built	No video consultation.
Clinical certification	Not done	The software is not a certified medical device and has had no clinical validation.

CHAPTER 5

5. REQUIREMENT ANALYSIS AND FEASIBILITY STUDY

5.1 Requirement Analysis

5.1.1 Functional requirements

The functional requirements are listed in Table 5.1, grouped by module and role. Each requirement has an identifier of the form FR-nn and names the API path prefix that implements it, so that the list can be traced to the endpoint list. The application exposes 385 operations in 38 router modules; the table groups them by purpose rather than listing each one. Requirements FR-01 to FR-37 of the earlier software requirements specification (`docs/SRS.md`) were merged into this list; the identifiers here are new.

Table 5.1: Functional requirements

ID	Role	Requirement	Endpoint or component
FR-01	All users	Log in with e-mail and password. A JWT is issued; wrong credentials give one generic message; 5 failures per e-mail and address in 15 minutes lock the login with a 429 answer.	POST /api/auth/login
FR-02	All users	Log in with a mobile number and a 6-digit code (valid 5 minutes, single use, stored hashed, 5 tries).	POST /api/auth/otp/request, /verify
FR-03	Administrator	Create the first administrator on an empty system only; registration is closed afterwards.	GET /api/auth/setup-status; POST /api/auth/register
FR-04	All users	A user with a temporary password may call only the account and change-password endpoints until the password is changed.	GET /api/auth/me; POST /api/auth/change-password
FR-05	All users	View and edit own profile and photo.	/api/profile
FR-06	Administrator	Create doctor, nurse, employee, pharmacy, laboratory and radiology staff accounts with a generated starting password.	POST /api/admin/{role}
FR-07	Administrator	Activate, deactivate, edit or delete a user, with a delete check; an administrator cannot deactivate self.	PUT, DELETE /api/admin/users/{id}
FR-08	Administrator	View the audit log with filters; no route can change or delete a log line.	GET /api/admin/audit-logs
FR-09	Administrator	View hospital statistics on a dashboard.	GET /api/admin/dashboard
FR-10	Administrator	Manage departments and linked specializations.	/api/catalog, /api/admin/departments
FR-11	Administrator	Maintain hospital profile, branches, typed settings and the service price list.	/api/hospital/*
FR-12	Administrator	Edit role permissions and assign extra roles to a user.	/api/access/roles, /permissions, /users/{id}/roles
FR-13	Doctor, administrator	Open one patient in an emergency with the exact UHID and a written reason for a limited time; the administrator reviews each grant.	/api/access/break-glass
FR-14	Administrator	View which integrations are ready, adapter-only or not built.	GET /api/integrations/status

ID	Role	Requirement	Endpoint or component
FR-15	Reception	Register a patient with a generated UHID; a same-name, same-birth-date record blocks the save unless the staff member confirms a different person.	POST /api/patients; /api/patient-registry/duplicates
FR-16	Reception, administrator	Record consent (treatment, data sharing, ABHA, photo, research), the ABHA number only with consent, and insurance details.	/api/patient-registry/{id}/consents, /abha
FR-17	Administrator	Merge two records of one person with a written reason; the old record is archived.	POST /api/patient-registry/merge
FR-18	Reception, patient	Book, reschedule or cancel an appointment; double booking of a doctor is refused.	/api/appointments
FR-19	Reception, doctor	Check in an appointment or walk-in to obtain a daily token; urgent patients are called first; mark no-show.	/api/opd/check-in, /walk-in, /queue
FR-20	Reception, doctor, nurse	Register an emergency visit, including an unidentified person, with triage category 1 to 5.	/api/emergency/visits, /triage
FR-21	Reception, doctor	Upload a report for an authorised patient.	POST /api/reports/upload
FR-22	System	Validate file type by content, size (10 MB) and signature; reject non-medical or unreadable files and save nothing.	/api/reports/upload
FR-23	System	Read the text layer of a PDF; use OCR for scanned pages and images when an OCR engine is installed, otherwise return a clear error.	/api/reports/upload
FR-24	System	Extract test name, value, unit and printed range; normalise names; read decimal commas and Indian digit grouping.	services/report_parser
FR-25	System	Classify each value as within, above, below or unknown using only the printed range; never invent a range.	services/analysis_engine
FR-26	System	Produce key statements, urgent flags, doctor suggestions and patient tips for any report type; an optional AI model may reword after redaction and a safety filter.	GET /api/reports/{id}/analysis; POST /insights/refresh
FR-27	System	Warn when the name printed on a report does not match the selected patient.	/api/reports/upload
FR-28	Doctor	See reports waiting for review, appointments and follow-ups on a dashboard.	GET /api/doctor/dashboard
FR-29	Doctor	Review a report (approved, modified, clarify); a comment is required for the last two; only the assigned doctor may review.	PUT /api/doctor/reports/{id}/review
FR-30	Doctor	Add notes and recommendations; labelled as the doctor's own.	/api/doctor/notes, /recommendations
FR-31	Doctor	Create a prescription with items and advised tests; only doctors may create, only the author may cancel.	/api/prescriptions
FR-32	Doctor	View a clinical summary with sources, timeline, report trends, medication history and follow-ups.	/api/doctor/patients/{id}/*
FR-33	Doctor	Record past medical history (diabetes, hypertension, admissions, surgeries) with source, verification and an append-only change log.	/api/health-history/*
FR-34	Doctor	Run the OPD queue, record a consultation as a draft, sign it (then locked) and add addenda.	/api/opd/*
FR-35	Doctor (junior, consultant)	Interns cannot prescribe; resident prescriptions, orders and discharge summaries wait for countersign by a consultant.	/api/countersign/*
FR-36	Doctor	Refer a patient to a department or doctor, and accept or decline referral requests.	/api/referral-requests, /api/referrals
FR-37	Doctor, nurse	Admit a patient, write progress notes and medication orders, assign staff, write and approve the discharge summary, discharge.	/api/ipd/*

ID	Role	Requirement	Endpoint or component
FR-38	Nurse	Record vitals with threshold alerts, give, refuse or hold medicines (reason required), record intake and output and shift handover.	/api/nursing/*
FR-39	Pharmacy	Search open prescriptions; dispense fully or partly, or mark a medicine unavailable; never edit a prescription.	GET /api/pharmacy/prescriptions; POST /dispense
FR-40	System	Refuse dispensing above the remaining quantity, of cancelled, expired or unreviewed (pending countersign) prescriptions, or of expired or recalled stock.	/api/pharmacy/dispense
FR-41	Pharmacy, store	Manage stock batches, alerts for low stock and expiry, recall, write-off, returns, and a ledger of every movement.	/api/pharmacy/inventory, /api/stock/*
FR-42	Pharmacy	Sell with a barcode and print a GST bill; bill a prescription once.	/api/pharmacy/sales, /prescriptions/{id}/bill
FR-43	Store, nurse, pharmacy	Requisition, purchase order and goods receipt update stock once; ward medicine requests are issued from usable stock only.	/api/procurement/*, /api/stock/*
FR-44	Laboratory	Work a test list: collect a sample, enter structured results against hospital ranges, reject a bad sample, mark urgent; a pathologist approves; critical values raise an alert.	/api/lab/*, /api/diagnostics/*
FR-45	Radiology	Schedule orders, attach image files, write a report and approve it; release to the doctor before the patient.	/api/radiology/*
FR-46	Billing, accountant	Raise invoices from charges, apply discounts with approval, take payments, advances and refunds, print receipts, and view daily cash, outstanding and revenue.	/api/finance/*
FR-47	Insurance desk	Handle claims through pre-authorisation, submission, decision and settlement.	/api/claims/*
FR-48	OT staff	Book theatres without overlap; record pre-operative steps, consent, checklist, anaesthesia, operation note, consumables and recovery.	/api/ot/*
FR-49	Blood bank staff	Register donors and donations with screening, keep unit stock, cross-match, issue and verify transfusion; one unit cannot serve two requests.	/api/bloodbank/*
FR-50	Administrator, nurse	Maintain wards and beds; one active patient per bed and one active admission per patient are enforced by the database.	/api/ipd/wards, /beds
FR-51	Patient	See own dashboard, reviewed reports, timeline, notes, advice and prescriptions with dispensing status.	/api/patient/*
FR-52	Patient, doctor	Compare reports and see parameter trends; units are compared only when a validated conversion exists.	/api/patients/{id}/comparison, /trends
FR-53	Patient	View own final invoices and receipts.	GET /api/finance/my-invoices
FR-54	Patient	View own health history and print the history sheet; view the emergency card.	/api/health-history/me
FR-55	Patient	Ask questions by text or voice and receive answers from own reviewed records, with sources; urgent messages get fixed emergency guidance first.	/api/patient-assistant/*
FR-56	System	Refuse requests to start, stop or change a medicine and requests for a diagnosis in the assistant.	/api/patient-assistant/chat
FR-57	System	Queue reminders and result notices in an outbox; send e-mail only when SMTP is configured; never include a result value in a notice.	/api/integrations/*
FR-58	Doctor, patient	Export a FHIR R4-shaped bundle for an authorised patient, labelled unvalidated.	/api/integrations/fhir/*
FR-59	System	Show the medical disclaimer on every page and in the health check.	GET /api/health

ID	Role	Requirement	Endpoint or component
FR-60	System	Add missing tables and columns at start-up without losing data.	database/connection.py

5.1.2 Non-functional requirements

The non-functional requirements are listed in Table 5.2. Each is verifiable by a test, a measurement or an inspection, and the last column states which. Values that depend on deployment and were not measured are marked as such.

Table 5.2: Non-functional requirements

ID	Category	Requirement	Verification
NFR-P1	Performance	A text report of ordinary length is analysed within seconds on a development laptop; OCR is slower.	Observation in the test suite; no benchmark was run
NFR-P2	Performance	Input limits prevent runaway work: request body limited to 1 MB (12 MB for uploads) and deeply nested JSON refused; limits on analysed text length and image size are only partly in place.	Bust tests D-01, J-01 (fixed); R-02, U-01, U-02 (see Chapter 8 for their status)
NFR-P3	Performance	Regular expressions on report text must not take quadratic time on one long line.	Bust test R-01; the personal-data redaction step was still slow in the final run (Chapter 8)
NFR-S1	Security	Passwords are hashed (SHA-256 then bcrypt); minimum 8 and maximum 128 characters, one letter and one number, not common, not containing the e-mail name.	Unit and API tests
NFR-S2	Security	JWT tokens carry a password fingerprint so that a password change ends older tokens; deactivated users lose access at once.	test_auth_rbac
NFR-S3	Security	Every endpoint is limited to its roles; patient-level access is checked per record; refused calls return 403 or 404 and never data.	Endpoint matrix, IDOR sweeps
NFR-S4	Security	Uploads are checked by content, stored under random names outside the web root and served as downloads with no-sniff headers.	test_uploads_static_config
NFR-S5	Security	Security headers, no-store caching of API answers, restricted CORS, API documentation not public.	test_uploads_static_config
NFR-S6	Security	The default signing secret is refused outside SQLite development.	Start-up check; the secret written in the compose file still passes it (finding O-01, Chapter 8)
NFR-S7	Security	An audit line is written for every route that changes data. Refused attempts at patient data are not recorded yet.	test_phase9_security; bust finding B-04 is still open (Chapter 8)
NFR-PR1	Privacy	Names, phone numbers and addresses are removed before text goes to an optional outside AI service; the assistant needs the patient's consent for AI wording.	Redaction tests; bust finding P-01 (open)
NFR-PR2	Privacy	The assistant stores the question and answer text but the audit log never contains them; chats are deleted after a retention period (90 days by default).	test_patient_assistant
NFR-PR3	Privacy	Clinical values are never returned to employees or administrators.	test_access_control
NFR-R1	Reliability	Invalid input gives a 422 with a readable sentence, never a 500; concurrent saves give a 409 or one winner.	Fuzz and concurrency tests

ID	Category	Requirement	Verification
NFR-R2	Reliability	The application works without an OCR engine and without an AI key.	Tests with the key unset
NFR-SC1	Scalability	SQLite suits a small hospital; PostgreSQL is supported by configuration, with row locks that take effect there.	Design; PostgreSQL not run in this project
NFR-U1	Usability	Screens work at 375 px phone width without horizontal scroll; plain-language labels; big action buttons.	Browser sweep; layout tests
NFR-U2	Usability	The login page, the assistant and the disclaimer banner pass automated accessibility checks (axe-core); colour contrast, a keyboard-only walk and a manual audit were not done.	a11y.axe.test.jsx
NFR-U3	Usability	Voice input and output use the browser; typing and reading remain available when speech is not.	voice tests
NFR-M1	Maintainability	Routers carry HTTP concerns only; logic lives in services; one validation rule set is shared; a route-drift test flags unreviewed new endpoints.	Code structure; test_route_drift
NFR-M2	Maintainability	The schema file is generated from the models and tested against them.	test_schema_and_ops
NFR-PT1	Portability	Runs on Windows, Linux and macOS with Python 3.11 or later and Node 20 or later; Docker files are provided but were not run.	README; bust report section 5
NFR-L1	Legal	Educational use only; disclaimer on every page; compliance with the DPDP Act and clinical-establishment rules must be reviewed by qualified professionals before real use.	Documents; Chapter 10

5.2 Feasibility Study

5.2.1 Technical feasibility

The solution uses mature open-source components that are listed with their versions in Section 7.1: FastAPI and Pydantic for the API, SQLAlchemy for persistence, SQLite for development and PostgreSQL for production, React with Vite and Tailwind CSS for the interface, and Tesseract for optional OCR. All of them are documented [9, 11, 12, 10, 14], run on ordinary hardware and are widely known. The skills needed are Python web development, relational modelling, front-end development with React, regular-expression and rule design for text extraction, and security testing. The project demonstrates technical feasibility directly: the whole stack runs on a single laptop, the automated suites described in Chapter 8 pass, and the interface was exercised in a real browser for Chapter 11. Two technical limits were found: OCR needs Tesseract, which was not installed in the development environment, so the OCR path was tested only through its error handling and a stubbed engine; and PostgreSQL and Docker were not run.

5.2.2 Economic feasibility

Software licences cost nothing: all components named above are open source. The recurring costs are hosting, backups and, optionally, a language-model service. Table

5.3 gives indicative figures. They are planning assumptions, not quotations, and must be re-checked against current provider price lists before being relied upon. No cost figure here has been measured in this project.

Table 5.3: Indicative cost items

Item	Basis	Indicative cost
Software licences (FastAPI, React, SQLAlchemy, PostgreSQL, Tesseract, Vite, Tailwind)	Open-source licences	None
Application server and PostgreSQL for a small hospital	One small virtual server (2 virtual CPUs, 4 GB memory), managed or self-managed	A low monthly amount in the range of a small cloud instance; to be priced with a chosen provider
Domain name and TLS certificate	Annual domain; free certificates from public authorities	A small annual amount for the domain
Backups and storage for uploaded files	Object storage or a second disk	Scales with report volume; small for a text-heavy workload
Optional language-model service	Pay per use; disabled by default; wording only	Zero when not configured
Development effort	See Section 6.2: the basic COCOMO model gives the effort of a conventional team	[Author to provide a labour rate; none is assumed here]

The economic argument is therefore qualitative: because the licence cost is zero and the infrastructure is modest, the recurring cost is dominated by hosting, backup, support and the cost of validating the software clinically, which is outside this project.

5.2.3 Operational feasibility

User adoption depends on how little the screens demand. The interface uses role-specific menus, plain-language labels, large action buttons and honest empty states, and it was reviewed for non-technical users in the second week of development (commit 'Make report pages and buttons easier for non-technical users'). Training needs differ by role: a reception employee must learn registration and upload; a doctor must learn review and prescribing; a pharmacist must learn dispensing; the administrator must learn account creation. The user manual in Chapter 11 is intended as the basis of that training. Mobile use is supported in three ways: the layout is checked at 375 px width; the assistant has a phone-first voice panel; and a Capacitor wrapper for Android is part of the repository. No pilot with real staff took place, so adoption has not been measured.

CHAPTER 6

6. DESIGN AND PLANNING

6.1 System Design and Analysis

6.1.1 Architecture

HealthWise AI follows a three-tier architecture, shown in Figure 6.1. A React single-page application in the browser calls a REST API under the path `/api`, secured by a JSON Web Token [18] in the Authorization header. The FastAPI application [9] contains a thin layer of routers that handle HTTP concerns only (input schema, status codes, the role check), a layer of services that hold all business logic, Pydantic schemas [11] for request and response validation, and SQLAlchemy models [12] for persistence. At the time of writing the application has 38 router modules exposing 385 operations, 58 service modules and 125 database tables. Uploaded files are kept in a private folder and served only through an authorised API call. Optional external services are Tesseract for OCR, a language-model API for wording, and an SMTP server for e-mail; all three are off unless configured.

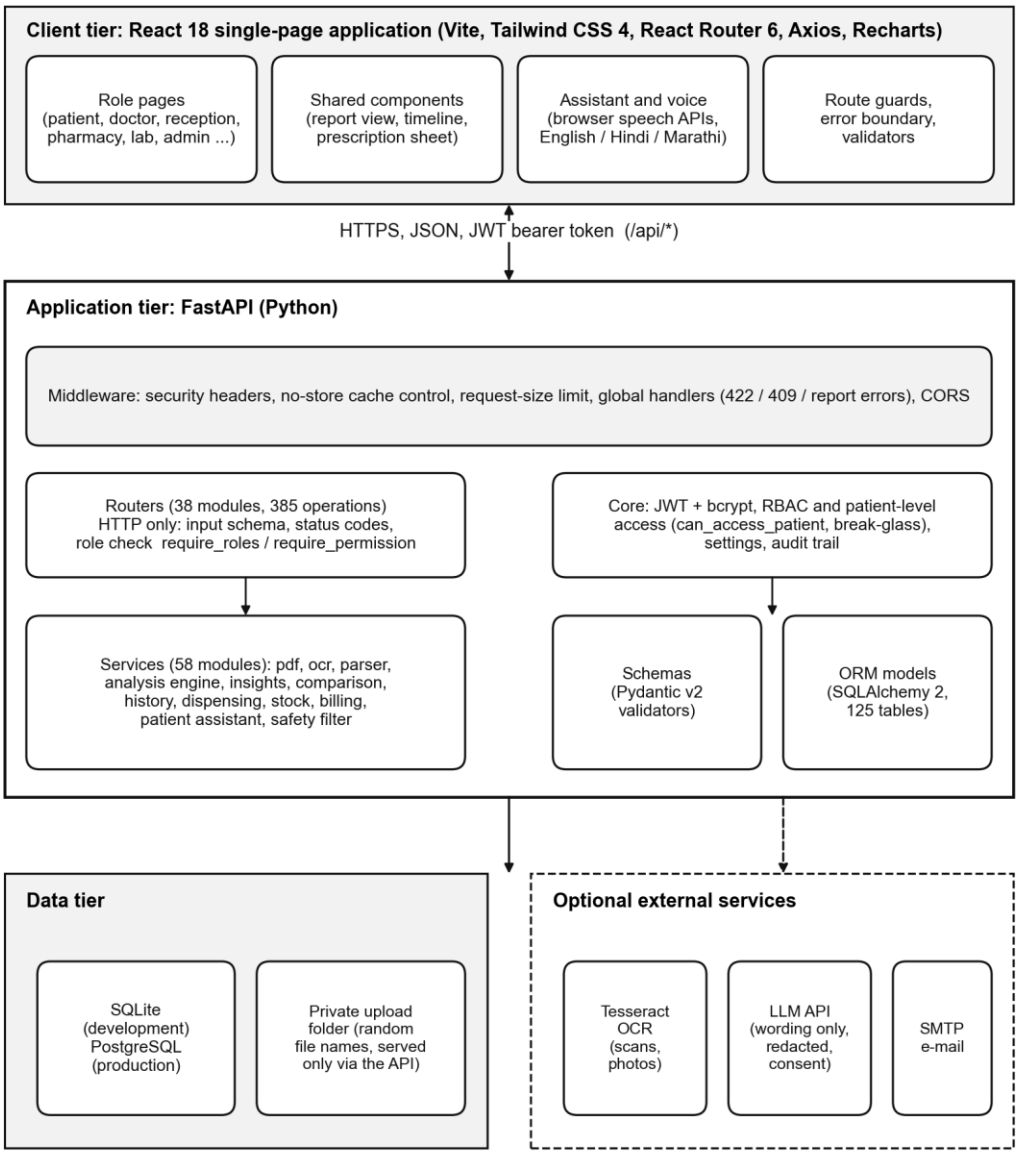


Figure 6.1: High-Level System Architecture

Two design rules run through the layers. First, the hybrid rule: deterministic code extracts values and decides every status, while the optional language model may only rewrite an explanation, and only after a safety filter. Second, the doctor-first rule: clinical output reaches a patient only after a doctor has reviewed it. Role checks are made twice, once for the endpoint (`require_roles` or `require_permission`) and once for the individual record (`get_accessible_patient`), so that a correct role is not enough to read another person's data.

6.1.2 Data flow diagrams

Figure 6.2 is the context diagram (level 0). It shows the system as one process, the six groups of human actors that exchange data with it, and the two optional external

services. Figure 6.3 decomposes the system into seven processes and six data stores for the main report-to-pharmacy workflow. Process 1.0 authenticates every flow; process 7.0 writes the audit log; the patient portal (6.0) reads only the reviewed part of the report store and the prescription store.

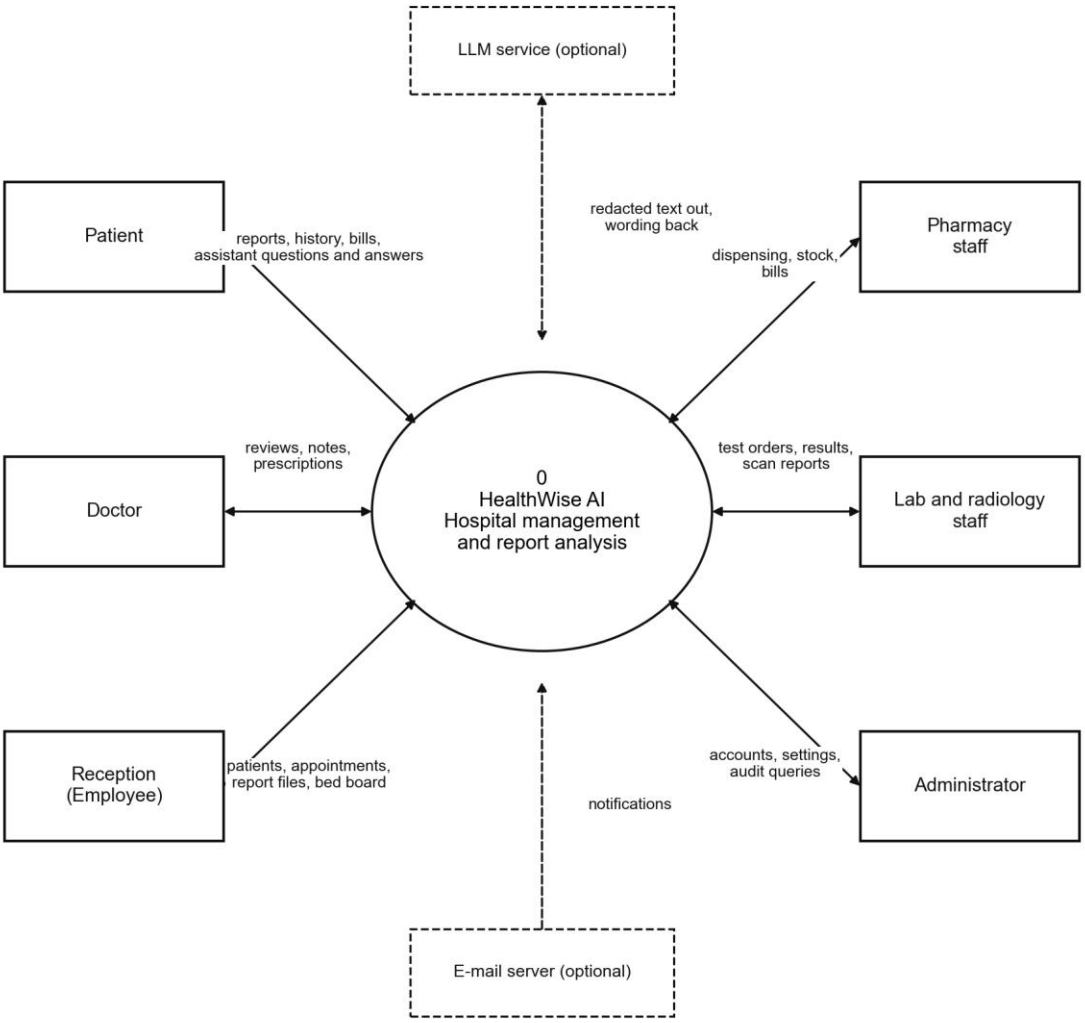


Figure 6.2: Data Flow Diagram, Level 0 (Context Diagram)

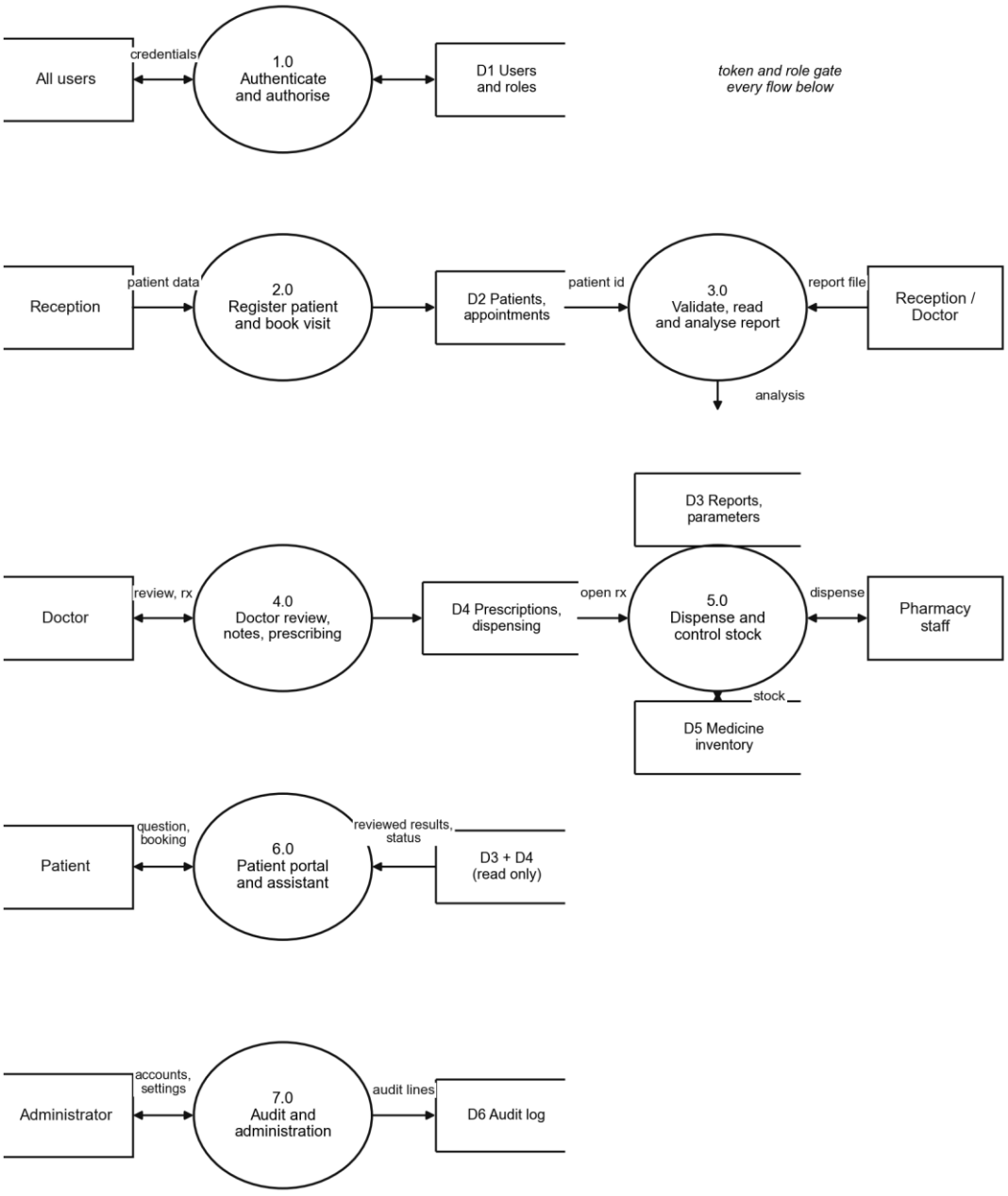


Figure 6.3: Data Flow Diagram, Level 1

6.1.3 Use-case diagrams

The use cases are drawn per role in Figure 6.4 to Figure 6.9. The system boundary is the rectangle; each ellipse is a use case that the actor can perform. Roles that share the reception desk (registration, upload, appointments) are drawn as one actor. Nurses, billing staff, the store, operation-theatre and blood-bank staff are not drawn separately; their use cases are listed in Table 4.1 and in the functional requirements.

Patient. A patient logs in, reads reviewed reports and trends, sees prescriptions and dispensing status, asks the assistant, and books or cancels appointments (Figure 6.4).

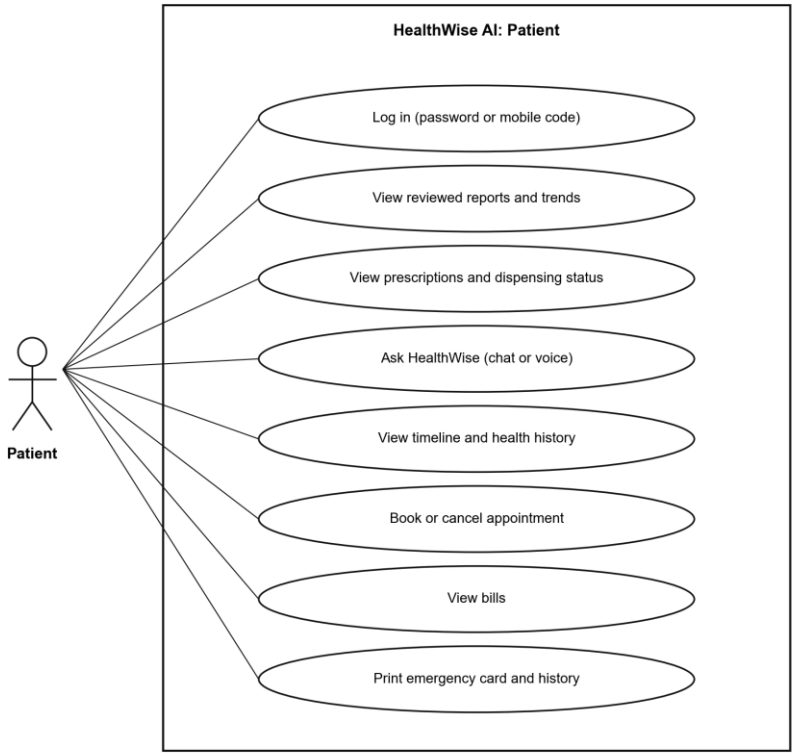


Figure 6.4: Use-Case Diagram for the Patient

Doctor. A doctor reviews reports, records notes and prescriptions, runs the consultation, manages inpatients and, in the senior grade, countersigns the work of junior doctors (Figure 6.5).

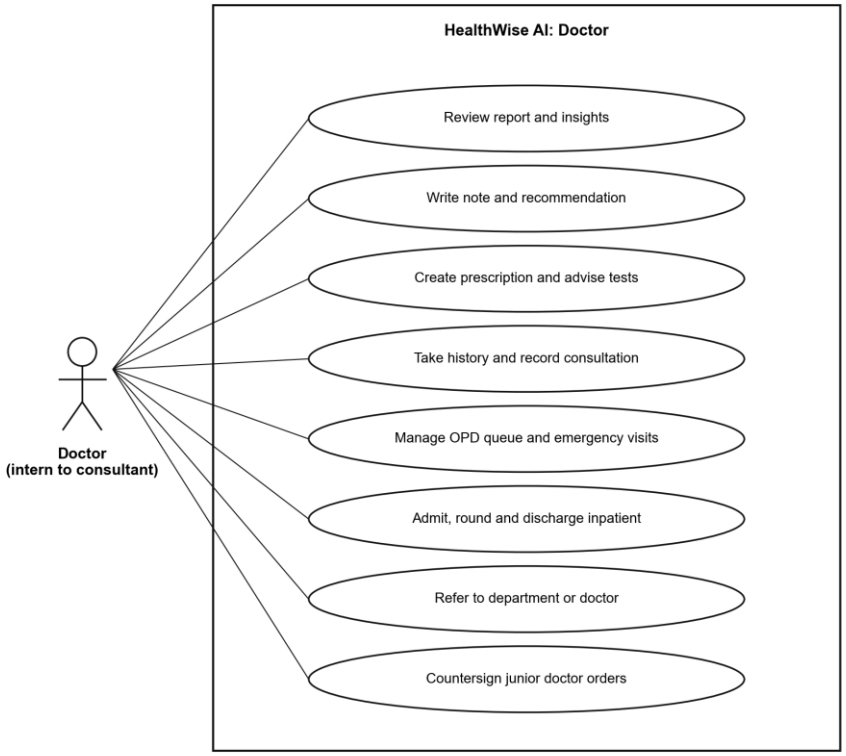


Figure 6.5: Use-Case Diagram for the Doctor

Reception. Reception registers patients, uploads reports, books appointments, runs the OPD desk and registers emergency visits (Figure 6.6).

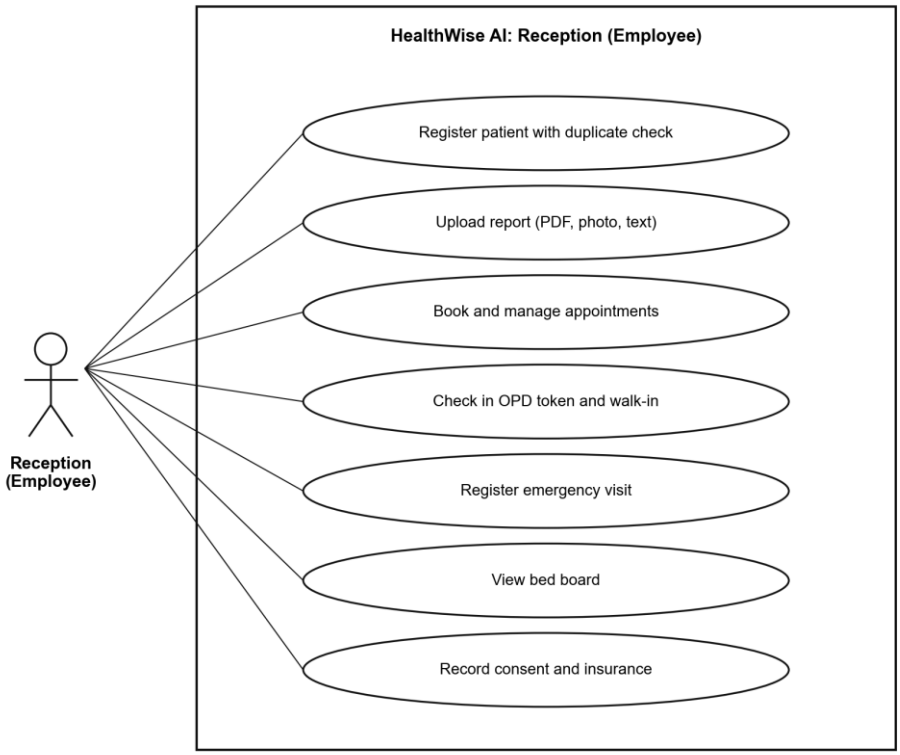


Figure 6.6: Use-Case Diagram for Reception (Employee)

Pharmacy. Pharmacy staff search open prescriptions, dispense, sell and bill, and keep stock (Figure 6.7).

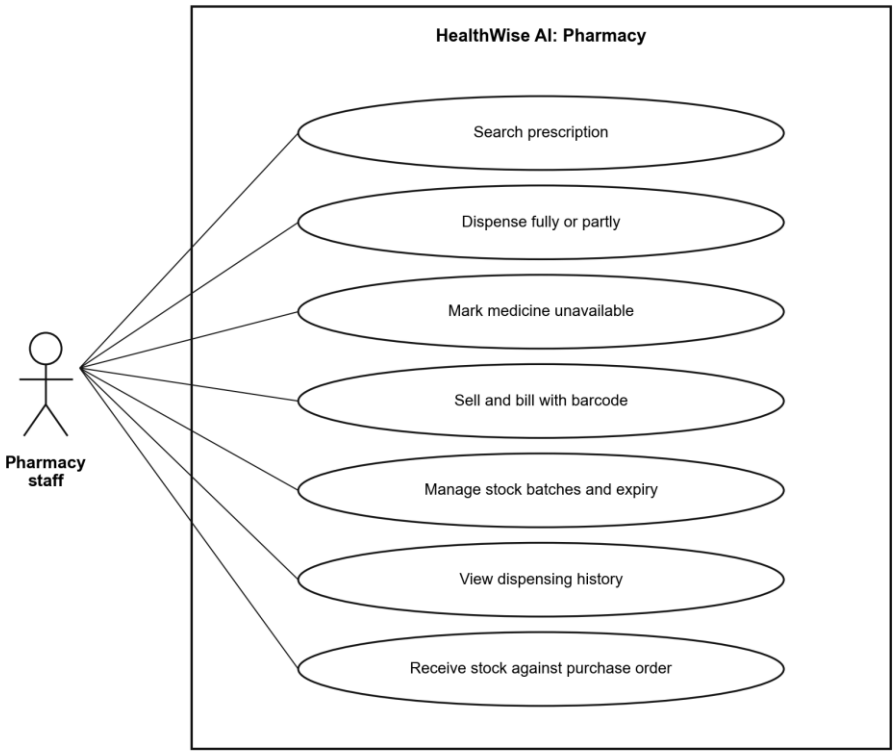


Figure 6.7: Use-Case Diagram for Pharmacy Staff

Laboratory and radiology. Laboratory technicians collect samples and enter results, a pathologist approves them, and radiology staff schedule scans and report on them (Figure 6.8).

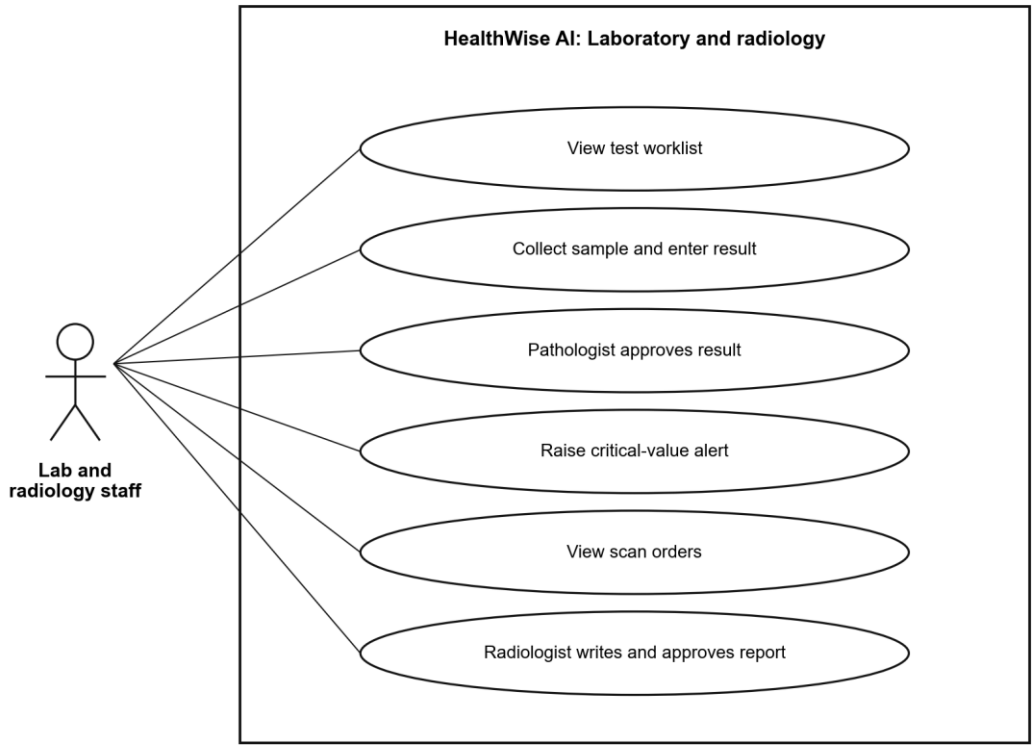


Figure 6.8: Use-Case Diagram for Laboratory and Radiology Staff

Administrator. The administrator creates and manages accounts, reads the audit log and configures the hospital (Figure 6.9). The administrator has no access to clinical content.

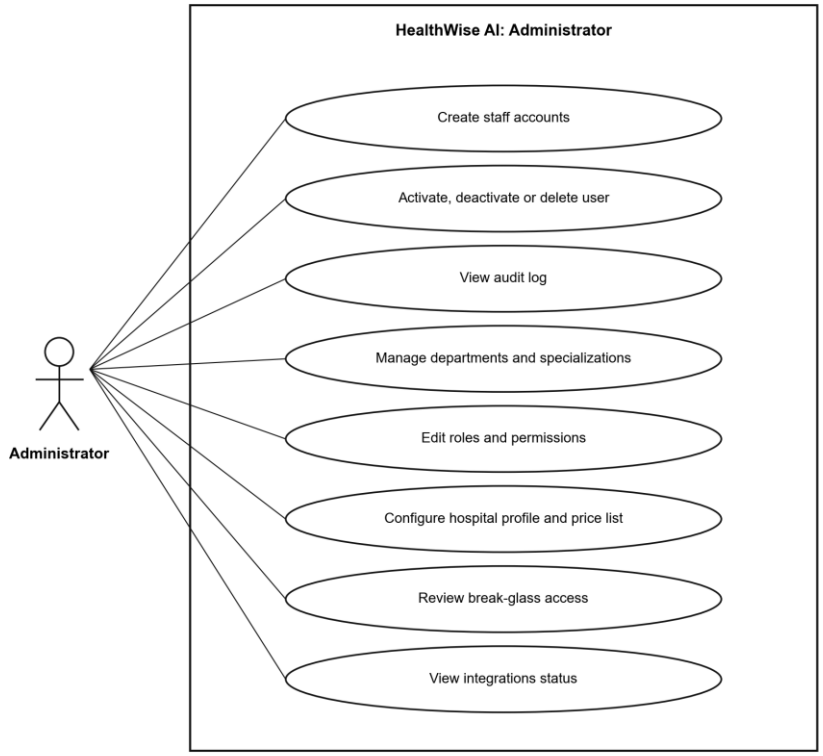


Figure 6.9: Use-Case Diagram for the Administrator

6.1.4 Class diagram

Figure 6.10 shows the principal classes of the report-to-pharmacy workflow. The role model is a single `users` table with a `role` column; one-to-one profile tables hold the extra fields of each role. A report contains parameters; a prescription lists items; each item may be dispensed in several steps, and each dispensing deducts stock. The audit log and the assistant conversations refer to users and patients by identifier.

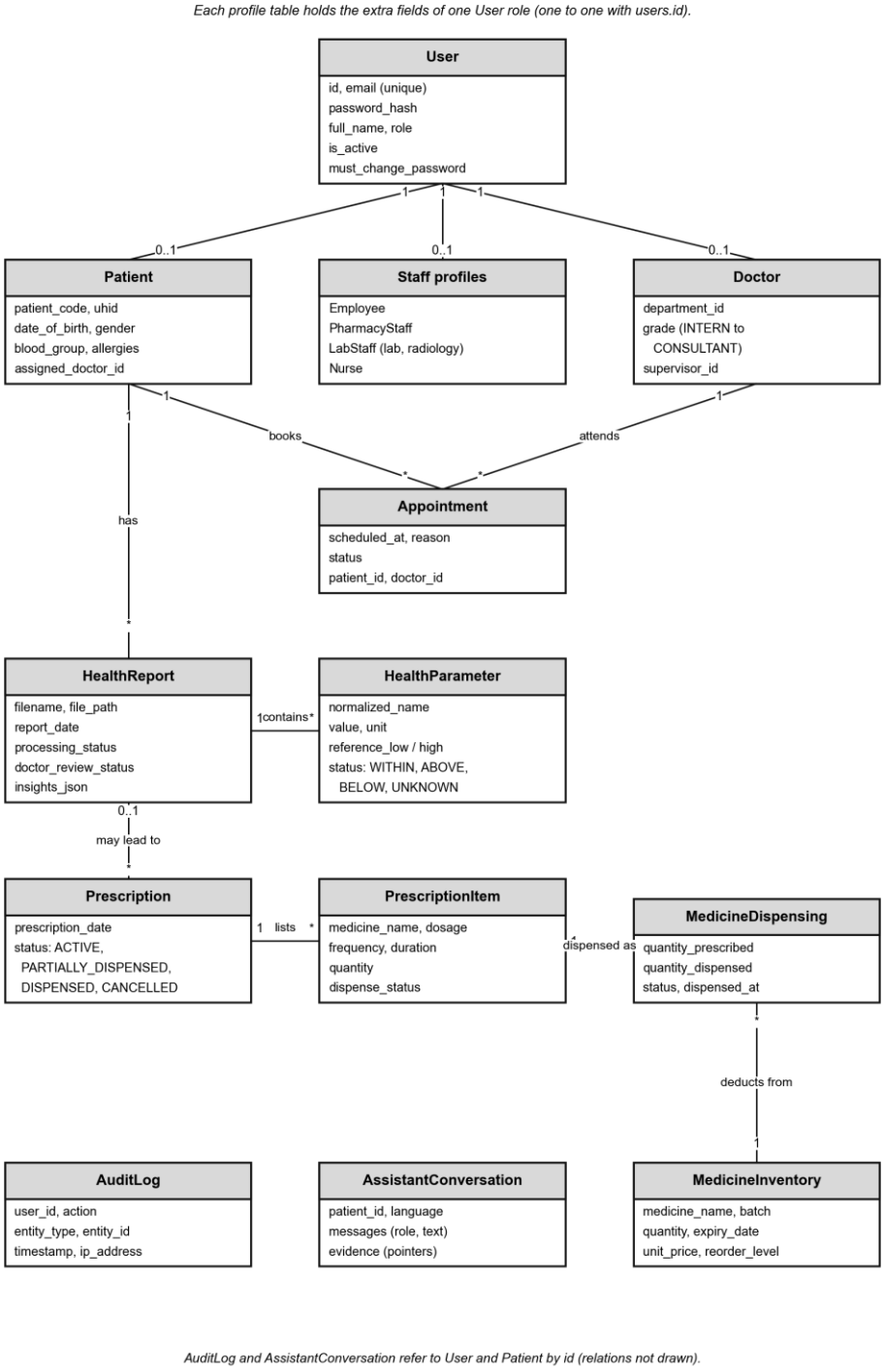


Figure 6.10: Class Diagram of the Main Entities

6.1.5 Entity-relationship diagram

The schema contains 125 tables (`backend/schema.sql`, generated from the SQLAlchemy models). Figure 6.11 draws 16 of them, those behind the report, prescription, dispensing, stock and bed flows. Two constraints deserve mention because they are enforced by the database and not only by the application: a partial unique index allows one open bed assignment per bed and one per admission, and another allows one active admission per patient (`uq_bed_one_active`, `uq_admission_one_bed`, `uq_one_active_admission`).

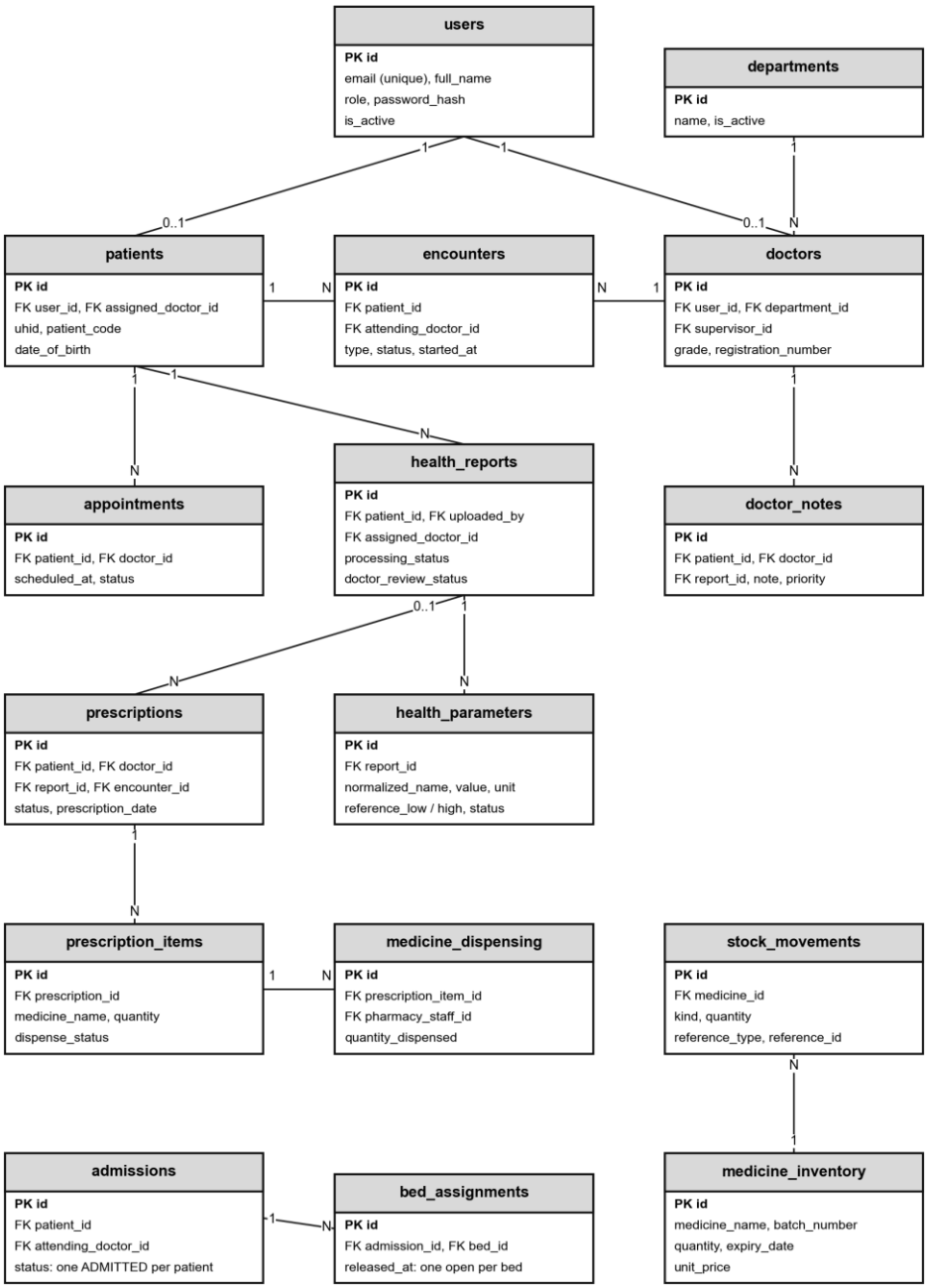


Figure 6.11: Entity-Relationship Diagram (Selected Tables)

6.1.6 Sequence diagrams

Four sequences are shown. Figure 6.12 is the login. The server reads the user, checks recent failures, verifies the password hash, creates a token and writes an audit line; every later request decodes the token and reloads the user, so that deactivation and password changes take effect at once.

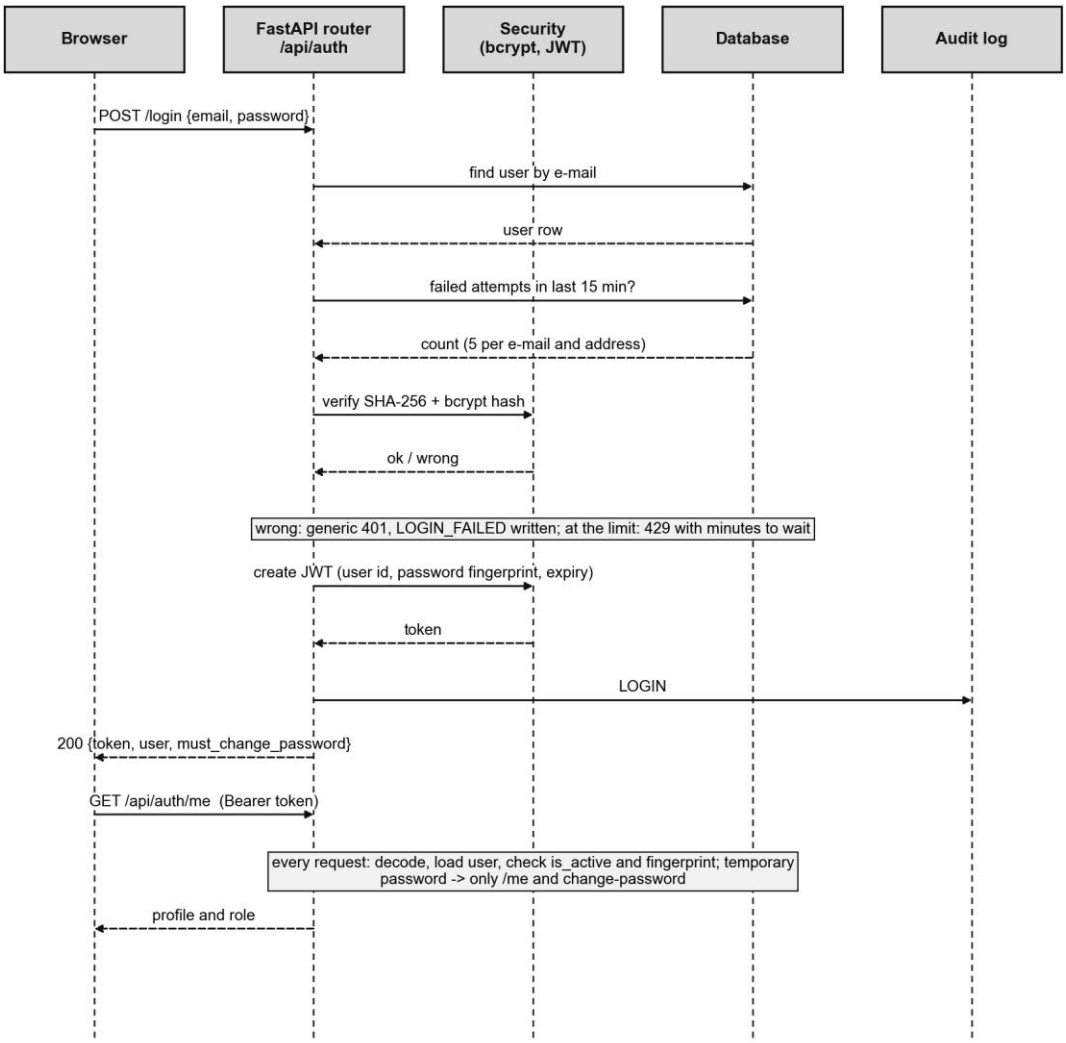


Figure 6.12: Sequence Diagram: Login and Token Use

Figure 6.13 is the upload and analysis of a report, followed by the doctor's review and the patient's view. A rejected file is never saved, and the employee who uploads receives only the status and no values.

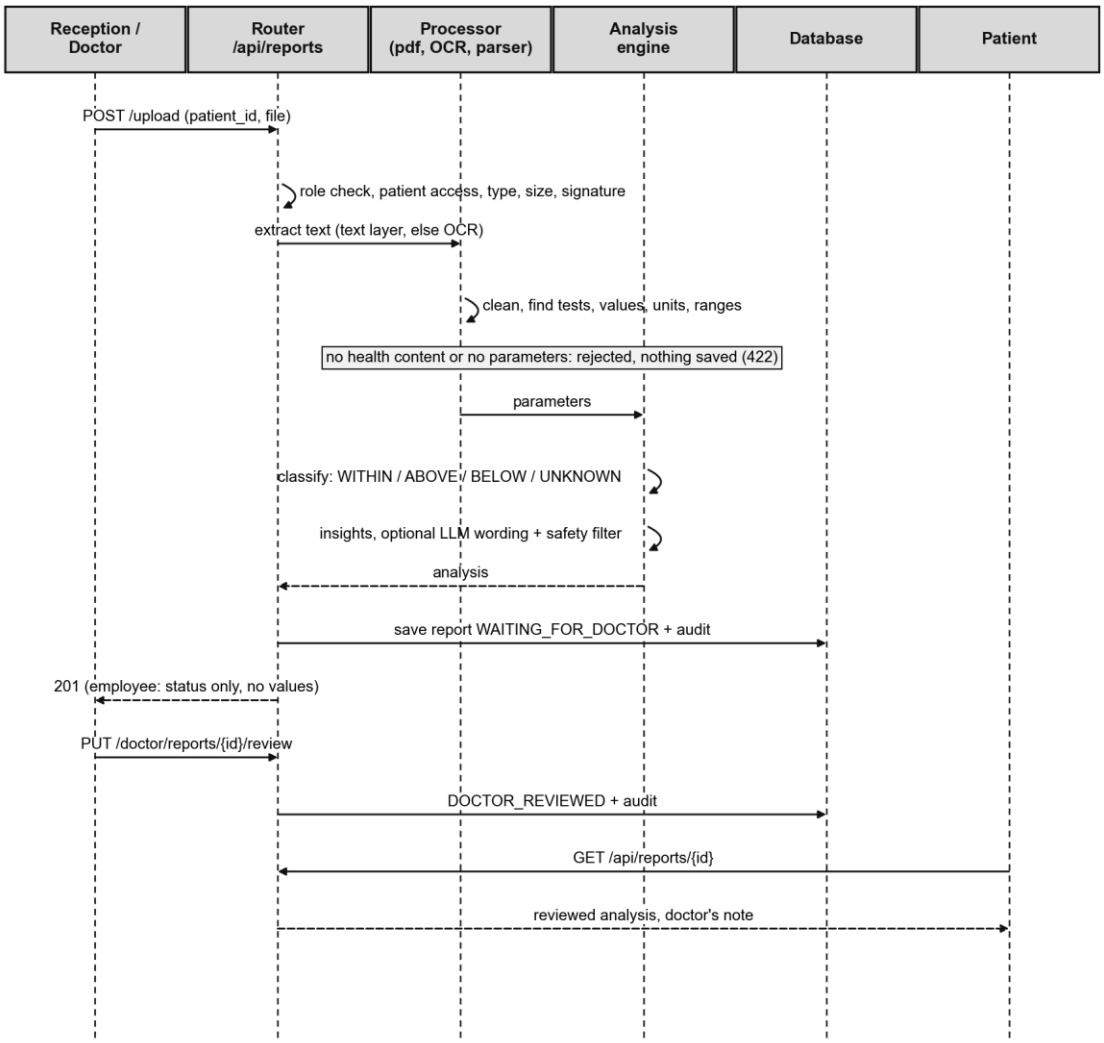


Figure 6.13: Sequence Diagram: Report Upload, Analysis and Review

Figure 6.14 follows a prescription from the doctor to the pharmacy and back to the patient. The dispensing service locks the item before it reads the remaining quantity.

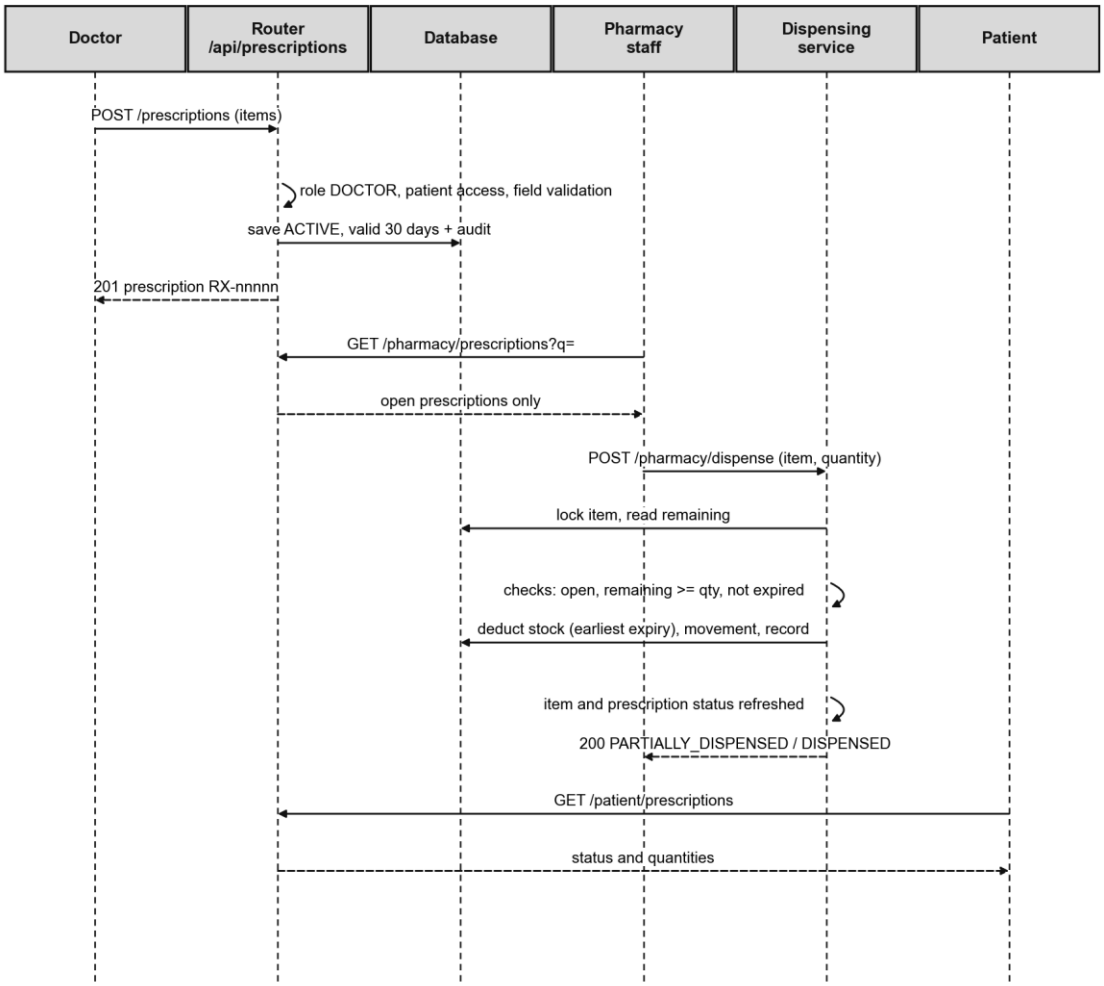


Figure 6.14: Sequence Diagram: Prescription to Pharmacy

Figure 6.15 is an assistant question. The patient is identified by the token and never by a field of the request; safety rules run before any record is read.

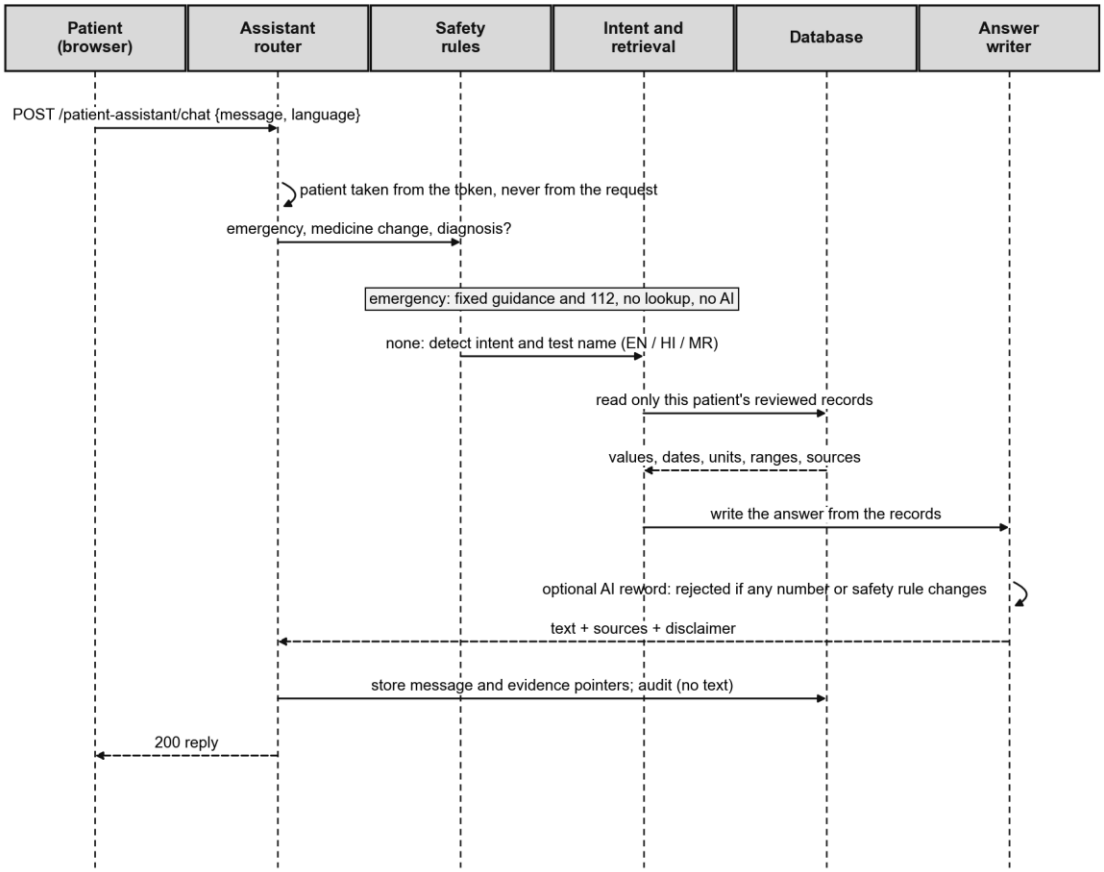


Figure 6.15: Sequence Diagram: Patient Assistant Question

6.2 Effort Estimation

Two sources are used for the effort chapter: the git history, which gives the real timeline, and the basic COCOMO model [15], which gives the effort that a conventional team would need for software of this size. The git history records commit dates only. It does not record hours worked or the size of the team, and the report does not invent them.

6.2.1 Timeline and phases from the git history

The repository holds 43 commits between 2026-10-08 and 2026-11-04, on 10 separate days, within a calendar span of 28 days (counted before the commit that adds this report). Figure 6.16 shows the phases and the number of commits per day; Table 6.1 gives the same phases with their commits.

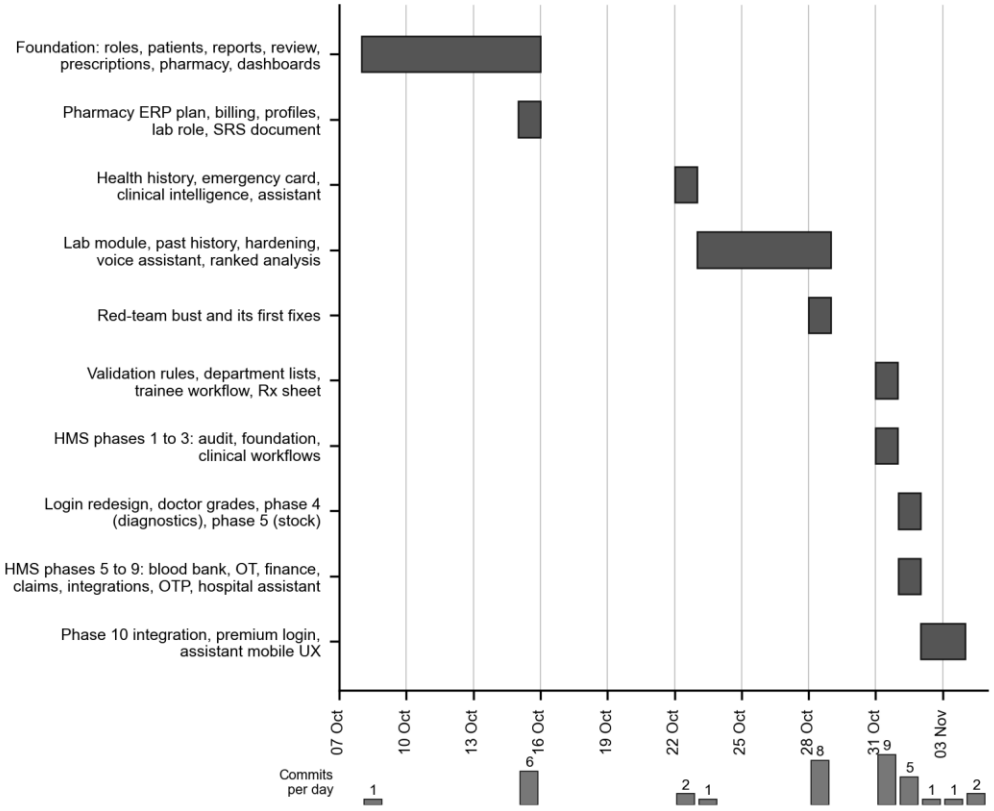


Figure 6.16: Gantt Chart of the Development Phases from the Git History

Table 6.1: Development phases and commits

Phase	Dates (2026)	Commits	Main content (from commit messages)
1	8 to 15 Oct	3	Initial commit with the full base system (roles, patients, reports, review, prescriptions, pharmacy, dashboards); extractor fix for tests without a range; simpler report pages.
2	15 Oct	4	Doctor pick-lists, lab role, medical-shop billing with barcodes, profiles with photos, pharmacy inspection plan, SRS document.
3	22 Oct	2	Patient health history, emergency card, doctor history-taking, clinical intelligence, patient chat and voice assistant.
4	23 to 28 Oct	5	Lab module, past history, security hardening; ranked analysis, report PDF, any report type, hands-free voice assistant.
5	28 Oct	4	Red-team bust with reproducible tests and the bust report; fixes for the clinical-reading findings; error boundary and phone-width fixes.
6	31 Oct	6	Shared input-validation rules, date-of-birth limit, Rx-pad prescription sheet, department and specialization lists, trainee workflow.
7	31 Oct	3	Hospital phases 1 to 3: audit, roles and permissions, UHID, consent, break-glass, encounters, OPD, nursing, emergency, wards, IPD, discharge.
8	1 Nov	4	Login redesign, doctor grades with countersign, hospital phase 4 (diagnostics) and phase 5 (stock and procurement).
9	1 Nov	1	Hospital phases 5 to 9: blood bank, operation theatre, finance, claims, integrations, mobile-code login, hospital assistant, security hardening.

Phase	Dates (2026)	Commits	Main content (from commit messages)
10	2 to 3 Nov	2	Hospital phase 10 (end-to-end workflow tests, schema and endpoint matrix, integrations page), premium login page, assistant mobile layout.
11	4 Nov	2	Native text-to-speech stand-in and more reliable native speech recognition for the Android wrapper; longer timeout for frontend tests under parallel load.

The history is compressed: 9 commits fall on the busiest day (2026-11-04), and the last five calendar days (31 October to 4 November) hold 25 of the 43 commits. Across the history, 77,754 lines were added and 4,192 were removed in total (`git log --shortstat`), including generated and documentation files.

6.2.2 Size and COCOMO estimate

The size was counted as non-blank, non-comment-only source lines of the application code, excluding tests, generated files (`schema.sql`), the Android project, dependencies and documents. The backend application contains 18,944 lines of Python and the frontend source contains 13,820 lines of JavaScript, JSX and CSS, a total of 32,764 lines, or 32.76 KLOC. The tests add a further 8,400 lines (backend) and 2,518 lines (frontend) that are not part of the estimate.

Basic COCOMO estimates the effort E in person-months, the development time D in months and the average staffing N from the size in thousands of delivered lines of code:

$$E = a \times (KLOC)^{b} \quad \text{(Eq. 6.1)}$$

$$D = c \times E^{d} \quad \text{(Eq. 6.2)}$$

$$N = E / D \quad \text{(Eq. 6.3)}$$

For the organic mode (a small, experienced team with a familiar problem) the coefficients are a = 2.4, b = 1.05, c = 2.5 and d = 0.38; for the semi-detached mode they are a = 3.0, b = 1.12, c = 2.5 and d = 0.35 [15]. With KLOC = 32.76, Eq. 6.1 gives E = 2.4 × 32.76^1.05 = 93.6 person-months for the organic mode. Eq. 6.2 gives D = 2.5 × 93.6^0.38 = 14.0 months, and Eq. 6.3 gives N = 93.6 / 14.0 = 6.7 persons. Table 6.2 shows both modes.

Table 6.2: Basic COCOMO results for 32.76 KLOC

Mode	a / b	c / d	E (person-months)	D (months)	N (persons)
Organic	2.4 / 1.05	2.5 / 0.38	93.6	14.0	6.7
Semi-detached	3.0 / 1.12	2.5 / 0.35	149.4	14.4	10.4

6.2.3 Interpretation and points to confirm

The COCOMO figures are an estimate for a conventional team, not a record of what happened. The git history shows that the system was built in a calendar span of 28 days, far shorter than the 14.0 months of the organic model. The difference cannot be explained from the repository alone. One relevant fact is visible in it: the commit trailers of 42 of the 43 commits name an AI coding assistant (Claude) as co-author, and the phase documents record decisions taken in dialogue with that assistant. Development was therefore assisted, and the compressed timeline should not be read as the output of one person working unaided. The report does not state person-hours, team size or any cost derived from them.

[TO BE CONFIRMED BY THE AUTHOR: the number of people who worked on the project, the approximate hours per week, the share of the work done with the AI coding assistant, and how the university expects assisted development to be declared. Replace this note with those facts; no figure has been assumed.]

CHAPTER 7

7. IMPLEMENTATION AND CODING

7.1 Implementation

7.1.1 Backend modules

The backend is organised by responsibility. Routers (`backend/app/routers`, 38 modules) accept and validate requests and apply the role check; services (`backend/app/services`, 58 modules) hold the logic; schemas (16 modules) define the Pydantic request and response models; `core` holds configuration, security and permissions; `database` holds the SQLAlchemy models (125 tables) and the start-up migration; `utils` holds validators, file checks and row locking. The application code counts 18,944 non-blank, non-comment-only lines of Python. Table 7.1 groups the modules by function.

Table 7.1: Backend modules by function

Group	Modules	Responsibility	Operations
Core and data	`core/security`, `core/permissions`, `core/config`, `database/models`, `database/connection`, `utils/validators`, `utils/locking`	JWT and bcrypt, role and patient-level checks, settings, 125 tables, start-up migration, shared validation, row locks	-
Accounts and administration	`auth`, `admin`, `profile`, `access`, `catalog`, `hospital`	Login, OTP, staff accounts, roles and permissions, break-glass, departments, hospital profile and price list	57
Report analysis	`reports`; `pdf_service`, `ocr_service`, `report_parser`, `parameter_catalog`, `normalization_service`, `analysis_engine`, `report_processor`, `insight_service`, `report_measures`, `safety_filter`, `llm_service`	Upload, text and OCR, parsing, status rules, insights, safety filter, optional AI wording	7
Records and history	`patients`, `patient_portal`, `patient_registry`, `health_history`, `clinical_intel`; `comparison_engine`, `report_trends`, `history_service`, `clinical_summary`, `clinical_timeline`	Registry, consent, merge, timeline, summary, comparison, trends, past history	56
Doctor workflow	`doctors`, `prescriptions`, `countersign`, `referrals`, `referral_requests`, `trainee`; `prescription_service`, `grade_service`	Review, notes, prescriptions, grades and countersign, referrals	44
Clinical operations	`appointments`, `opd`, `nursing`, `emergency`, `ipd`, `ot`, `bloodbank`; matching services	Visits, queue, consultation, nursing chart, emergency, wards and beds, discharge, theatre, blood bank	102
Diagnostics	`lab`, `diagnostics`, `radiology`; `lab_service`, `diagnostic_service`	Sample tracking, structured results, approval, radiology reports	29
Pharmacy and stock	`pharmacy`, `billing`, `stock`, `procurement`; `dispensing_service`, `stock_service`, `billing_service`, `procurement_service`	Dispensing, sales, stock ledger, procurement	42
Finance	`finance`, `claims`; `finance_service`	Invoices, payments, refunds, claims	34

Group	Modules	Responsibility	Operations
Patient assistant	`patient_assistant`; `patient_assistant` service, `assistant_safety`, `assistant_text`, `assistant_talk`, `assistant_hospital`	Intent, retrieval, safety rules, wording in three languages, voice commands	6
Integration	`integrations`; `integration_service`, `otp_service`	Outbox, reminders, FHIR bundle, status list	6

The operation counts were obtained by loading every router module and counting its route methods; the groups add up to 383; the remaining two are the dashboard and employee-dashboard routes (`dashboard`, `employees`), giving 385 in the 38 router modules. The doctor directory route and the health check registered in `main.py` are counted separately.

7.1.2 Frontend modules

The frontend (`frontend/src`) is a React 18 application with 73 page files, 78 component files, 21 API service modules, 4 hooks and one authentication context. It contains 13,820 non-blank, non-comment-only lines. Routes are protected by three wrappers: `ProtectedRoute` (a token is needed), `RoleRoute` (the role must be listed) and `PermissionRoute` (the user must hold a permission). The backend enforces the same rules; the wrappers only avoid showing pages that would fail.

Table 7.2 lists the frontend folders and their contents.

Table 7.2: Frontend modules

Folder	Contents
`pages/patient`	Dashboard, assistant, reports, prescriptions, history, bills, notes, advice, profile, emergency card
`pages/doctor`	Dashboard, report review, patient details, consultation, OPD queue, inpatients, referrals, countersign, prescriptions, emergency access
`pages/employee`, `pages/auth`	Registration, report upload, OPD desk; login (password and mobile code), change password
`pages/pharmacy`, `pages/stock`	Dispensing, billing, inventory, history; stock control, procurement, ward requests
`pages/lab`, `pages/radiology`, `pages/nurse`, `pages/ot`, `pages/bloodbank`, `pages/finance`	Work lists and result entry, scan worklist, nursing chart, theatre schedule, blood bank, billing and claims
`pages/admin`	Dashboard, users and staff, hospital set-up, integrations, audit log, settings
`components`	Report and result tables, insights panel, timeline, prescription sheet and printing, error boundary, disclaimer banner, assistant chat and voice panel
`services`, `hooks`, `utils`	Axios clients per area; fetch, save and validation hooks; shared validators, voice and native speech, print helpers

7.1.3 Environment setup and technology stack

The project runs on Windows, Linux and macOS. The backend needs Python 3.11 or later and the frontend needs Node.js 20 or later; the development environment used

Python 3.13.1, Node.js 24.13.0 and npm 11.6.2. Tesseract OCR is optional. `backend/requirements.txt` lists the Python packages with lower bounds only (for example `fastapi>=0.110`), so the versions below are those installed in the development environment when the tests of Chapter 8 were run. The frontend versions are taken from `package-lock.json`.

Table 7.3 gives the technology stack with declared and installed versions.

Table 7.3: Technology stack and versions

Component	Declared	Installed	Purpose
Python (runtime)	3.11+	3.13.1	Backend language
FastAPI	>=0.110	0.142.2	Web framework and routing
Uvicorn	>=0.29	0.54.0	ASGI server
Pydantic / pydantic-settings	>=2.6 / >=2.2	2.13.5 / 2.15.0	Validation and settings
SQLAlchemy	>=2.0	2.1.3	ORM
PyJWT / bcrypt	>=2.8 / >=4.1	2.15.1 / 5.0.0	Tokens and password hashing
pypdf / pypdfium2	>=4.0 / >=4.30	6.19.0 / 5.14.0	PDF text and page rendering
Pillow / pytesseract	>=10.2 / >=0.3.10	12.3.0 / 0.3.13	Images and OCR interface
psycopg2-binary / reportlab	>=2.9 / >=4.1	not installed in this environment / 5.0.1	PostgreSQL driver (PostgreSQL was not run) and PDF output
pytest / httpx	>=8.0 / >=0.27	9.1.1 / 0.28.1	Backend tests and test client
React / React DOM	^18.3.1	18.3.1	User interface
React Router	^6.26.0	6.30.6	Routing
Axios	^1.7.0	1.20.0	HTTP client
Recharts / lucide-react	^2.12.7 / ^0.460.0	2.15.4 / 0.460.0	Charts and icons
Vite / Tailwind CSS	^6.0.0 / ^4.0.0	6.4.5 / 4.3.3	Build tool and styling
Vitest / jsdom / axe-core	^2.1.9 / ^25.0.1 / ^4.10.2	2.1.9 / 25.0.1 / 4.10.2	Frontend tests and accessibility audit
Capacitor	^8.6.0	8.6.0	Android wrapper

To run the system locally, the steps are the following; Chapter 11 repeats them with screenshots.

```
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python -m app.seed_demo
uvicorn app.main:app --reload
```

```
cd frontend
npm install
npm run dev
```

Settings are read from `backend/.env` (template `.env.example`): `DATABASE_URL` (empty selects SQLite), `JWT_SECRET`, `LLM_API_KEY` (optional), `UPLOAD_DIR`, `MAX_FILE_SIZE`, `TESSERACT_CMD`, `CORS_ORIGINS` and `OTP_DEMO_MODE`.

7.2 Coding

This section shows the code that carries the main design decisions. The snippets are copied from the repository; docstrings were removed and long lines were wrapped for printing, so the layout differs slightly from the source files. Each snippet is explained briefly.

7.2.1 Report extraction and abnormal-value flagging

Figure 7.1 shows the pipeline. A file is validated by type and signature, text is taken from the PDF text layer or by OCR, the text is cleaned and parsed, and each value is classified against the range printed next to it. A file with no recognisable health content, or with no extracted parameter, is rejected and nothing is saved.

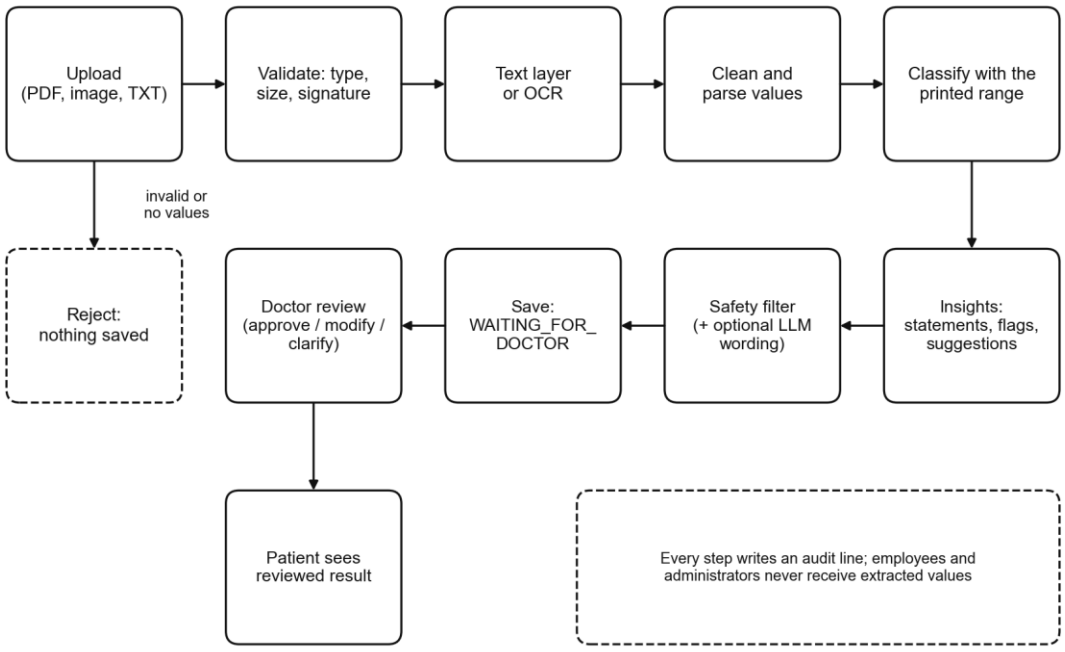


Figure 7.1: Report Processing Pipeline

The status of a value is decided by a pure function of the value and the parsed reference. The rule is given in Eq. 7.1, where L and H are the printed lower and upper bounds; if no bound is printed the status is UNKNOWN. The code follows.

$$S(v) = BELOW \text{ if } v < L;\ \ ABOVE \text{ if } v > H;\ \ WITHIN \text{ otherwise} \quad \text{(Eq. 7.1)}$$

```
def classify_status(value: float, reference: ParsedReference |
    None) -> str:
    if reference is None or (reference.low is None and
        reference.high is None):
        return UNKNOWN
    if reference.low is not None:
        too_low = value < reference.low or (value ==
            reference.low and not reference.low_inclusive)
        if too_low:
            return BELOW
    if reference.high is not None:
        too_high = value > reference.high or (value ==
            reference.high and not reference.high_inclusive)
        if too_high:
            return ABOVE
    return WITHIN
```

A value equal to a bound is inside an inclusive bound and outside an exclusive one: with a printed range of '< 150', a value of 150 is above. When no range is printed, the result is UNKNOWN: the system never supplies a range of its own. Reading the range correctly is the harder part. When a report prints several labelled tiers, only a tier labelled as normal (normal, desirable, optimal, sufficient and similar words) is taken as the reference range; if none is labelled normal, the printed tiers are kept as text and no range is used. The same applies to separate ranges for men and women, because the patient's sex is not known to the parser.

```
if len(labelled) >= 2:
    for label, comparator, first, second in labelled:
        if NORMAL_LABEL.match(label.strip()):
            if second:
                low, high = _to_float(first), _to_float(second)
                if low <= high:
                    return ParsedReference(f"{first} -
                        {second}", low, high)
            elif comparator in ("<", "<=", "≤"):
                return ParsedReference(f"< {first}", None,
                    _to_float(first), True, comparator != "<")
            elif comparator in (">", ">=", "≥"):
                return ParsedReference(f"> {first}",
                    _to_float(first), None, comparator != ">",
                    True)
    # several labelled ranges and none of them says "normal":
        no range is guessed, the printed text is kept for the
        doctor
    shown = "; ".join(f"{lab.strip()}: {cmp or ''}{a}{('-' +
        b) if b else ''}" for lab, cmp, a, b in labelled)
    return ParsedReference(shown[:120], None, None)
```

For free-text reports, a sentence is classified as normal, worth a look, or neutral, and as urgent or not. A fix recorded in the bust report (T-02) rewrites phrases such as 'cannot rule out' to 'possible' before the negation check, because a naive negation list would otherwise read 'cannot rule out pulmonary embolism' as a normal sentence.

```
UNCERTAIN_RE = re.compile(r"\b(?:cannot|can not|can't|unable
    to|not able to|not possible to|impossible to|difficult to|hard
    to)\s+(?:be\s+)?(?:rule[sd]?\s+out|exclude[sd]?)\b"

r"|\b(?:not|never)\s+(?:been\s+)?(?:excluded|ruled
                                out)\b")
...
lowered = sentence.lower().replace("body mass index", "bmi")
    # "mass" in BMI is not a finding
lowered = UNCERTAIN_RE.sub("possible", lowered)
    # "cannot rule out X" = X is possible, not absent
```

7.2.2 Role-based access checks

Access control has two layers. The first answers whether a role may call an endpoint at all. The dependency `get_current_user` decodes the token, loads the user from the database on every request, and refuses a missing user, an inactive user, a token issued before the last password change, and a user who still has a temporary password; `require_roles` then compares the role.

```
decoded = decode_token(credentials.credentials)
user = db.get(User, decoded[0]) if decoded else None
if user is None or not user.is_active or decoded[1] !=
    password_fingerprint(user.password_hash):
    raise unauthorized
if user.must_change_password and request.url.path not in
    TEMP_PASSWORD_PATHS:
    raise HTTPException(status.HTTP_403_FORBIDDEN,
        MUST_CHANGE_MESSAGE)
return user
```

```
def require_roles(*roles: str):

    def checker(user: User = Depends(get_current_user)) -> User:
        if user.role not in roles:
            raise forbidden()
        return user

    return checker
```

The second layer answers whether this user may see this patient. A patient sees only their own record; a doctor needs a relationship with the patient (assignment, appointment, report, visit, admission, referral or a recorded emergency grant); a nurse needs the patient to be in the hospital now; administrator and employee are refused clinical data altogether.

```
def get_accessible_patient(db: Session, user: User,
    patient_id: int, clinical: bool = True) -> Patient:
    patient = db.get(Patient, patient_id)
    if patient is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND,
            "Patient not found.")

    if user.role == PATIENT:
        if patient.user_id != user.id:
            raise forbidden()
    elif user.role == DOCTOR:
        if not doctor_can_access_patient(db,
            get_doctor_profile(db, user), patient):
            raise forbidden()
    elif user.role == NURSE:
        if not nurse_can_access_patient(db, patient):
            raise forbidden()
    elif user.role in (ADMIN, EMPLOYEE):
        if clinical:
            raise forbidden()
    else:
        raise forbidden()
    return patient
```

The frontend repeats the role rule only to avoid showing pages that would fail:

```
export default function RoleRoute({ roles }) {
  const { user } = useAuth();
  if (!roles.includes(user.role)) return <Navigate
      to={ROLE_HOME[user.role]} replace />;
  return <Outlet />;
}
```

7.2.3 Password and validation rules

One set of validators is written once and used by every request schema. A new password has to be 8 to 128 characters with a letter and a number and must not be a common password; a stored password is hashed with SHA-256 first so that the whole password counts, and then with bcrypt [19, 7]. A date of birth must be a real date, not in the future, not before 1 January 1930 and not implying an age above 120 years.

```
def check_password(value: str) -> str:
    if not isinstance(value, str) or len(value) < 8:
        raise ValueError("The password must have at least 8
            characters.")
    if len(value) > 128:
        raise ValueError("The password is too long (128
            characters at most).")
    if not re.search(r"[A-Za-z]", value) or not
        re.search(r"\d", value):
        raise ValueError("The password must contain at least
            one letter and one number.")
    if value.lower() in COMMON_PASSWORDS or
        value.lower().rstrip("!@#$%^&*") in COMMON_PASSWORDS:
        raise ValueError("This password is too common. Please
            choose another one.")
    return value
```

```
def _date_of_birth(value):
    if value is None:
        return value
    today = date.today()
    if value > today:
        raise ValueError("The date of birth cannot be in the
            future.")
    if value < MIN_DATE:
        raise ValueError("The date of birth cannot be before
            1930.")
    age = today.year - value.year - ((today.month, today.day)
        < (value.month, value.day))
    if age > 120:
        raise ValueError("The age cannot be more than 120 years.")
    return value
```

Invalid input to any endpoint gives a 422 with one readable sentence and one message per field, never a 500; the frontend repeats the same rules for instant feedback (`frontend/src/utils/validators.js`, 62 unit tests).

7.2.4 Prescription countersign and dispensing

The status of a new prescription depends on the grade of the author. An intern cannot prescribe; a resident's prescription waits for a consultant's countersign and is hidden from the patient, the pharmacy and the laboratory until then; a consultant's prescription is active at once.

```
def prescription_status(doctor: Doctor) -> str:
    grade = doctor.current_grade
    if grade == INTERN:
        raise HTTPException(403, "An intern cannot prescribe.
            Please write a draft note for your consultant.")
    if grade == CONSULTANT or doctor.can_prescribe_independently:
        return "ACTIVE"
    return "PENDING_COUNTERSIGN"
```

The pharmacy can only dispense what a doctor prescribed. The service refuses a prescription that is not open, a quantity above the remaining quantity, and a zero or negative quantity; it then deducts stock and sets the status of the item and of the prescription.

```
already = dispensed_quantity(item)
remaining = item.quantity - already
if data.mark_unavailable:
    item.dispense_status = "UNAVAILABLE"
    quantity = 0
else:
    quantity = data.quantity
    if quantity <= 0:
        raise HTTPException(422, "Quantity must be greater
            than zero.")
    if remaining <= 0:
        raise HTTPException(status.HTTP_409_CONFLICT, "This
            medicine has already been fully dispensed.")
    if quantity > remaining:
        raise HTTPException(status.HTTP_409_CONFLICT,
                            f"Only {remaining} remaining to
                                dispense for
                                {item.medicine_name}.")
    if deduct_stock:
        _deduct_stock(db, item.medicine_name, quantity, rx.id,
            staff.user_id)
    item.dispense_status = "DISPENSED" if already + quantity
        >= item.quantity else "PARTIALLY_DISPENSED"
```

7.2.5 Assistant retrieval and the 'never invent values' guardrails

The patient assistant is a retrieval system with a fixed answer writer, not a free-text generator. For each question the server (1) takes the patient from the token, (2) runs the safety rules, (3) detects the intent and the test name from English, Hindi and Marathi keywords, (4) reads only the records that intent needs and that the patient may already see, and (5) writes the answer with ordinary code from those records. The numbers, dates, units and names in an answer are therefore the record's own. This follows the idea of retrieval-augmented generation [13] but keeps generation deterministic.

The safety rules run before any record is read. An emergency pattern wins over everything; a request to stop, start or change a medicine, a request for a diagnosis, and a possible medicine reaction are each refused or redirected.

```
def check(message: str) -> str | None:
    text = _clean(message)
    if EMERGENCY_RE.search(text):
        about_records = RECORD_QUESTION.search(text) or
            EDUCATIONAL.search(text)
        if not (about_records and not HAPPENING_NOW.search(text)):
            return "EMERGENCY"
    if MED_CHANGE_RE.search(text):
        return "MED_CHANGE"
    if REACTION_RE.search(text):
        return "REACTION"
    if DIAGNOSIS_RE.search(text):
        return "DIAGNOSIS"
    return None
```

An optional language model may reword an answer only if the patient consented and a key is configured. The reworded text is accepted only if the set of numbers is identical to that of the original, it contains no unsafe pattern, it is not much longer, and, for Hindi and Marathi, it is written in Devanagari script; otherwise the plain answer is shown.

```
old_numbers, new_numbers = set(re.findall(r"\d+(?:\.\d+)?",
    _digits(answer))), set(re.findall(r"\d+(?:\.\d+)?",
    _digits(new)))
if not new or len(new) > len(answer) * 1.6 + 200 or
    UNSAFE_AI.search(new):
    return None
if new_numbers != old_numbers:
    # a changed, added or dropped number: reject
    return None
if lang != "en" and not has_devanagari(new):
    return None
return new
```

7.2.6 Bed and stock transaction handling

Two kinds of conflict are handled differently. For beds, the database itself is the guard: two partial unique indexes (`uq_bed_one_active`, `uq_admission_one_bed`) make it impossible for a bed to hold two active patients or a patient to lie in two beds, and the service turns a violation into a readable 409 message inside a savepoint, so that nothing is half changed.

```
try:
    with db.begin_nested():
        if current:
            current.released_at = now()
            current.bed.status = "CLEANING"
            db.flush()
        row = BedAssignment(admission_id=admission.id,
            bed_id=bed.id, assigned_by=user.id, reason=reason,
            assigned_at=now())
        db.add(row)
        bed.status, bed.reserved_for_patient_id = "AVAILABLE",
            None
        db.flush()
except IntegrityError:
    raise HTTPException(409, "This bed was taken a moment ago.
        Please choose another bed.")
```

For stock and money, a row lock is taken before the deciding numbers are read. PostgreSQL receives a real `SELECT ... FOR UPDATE`; SQLite has no row locks, so a no-op update takes its database write lock, and the next request waits and then sees the first one's result. The lock is used by payments, refunds, claims, requisitions, stock counts and blood-unit matching.

```
def lock_row(db: Session, model, row_id: int) -> None:
    if db.get_bind().dialect.name == "sqlite":
        db.execute(update(model).where(model.id ==
            row_id).values(id=model.id))
    else:
        db.execute(select(model.id).where(model.id ==
            row_id).with_for_update())
```

Stock is taken from batches in order of earliest expiry, expired batches are skipped, a shortage is refused before anything changes, and every deduction leaves a movement in the stock ledger:

```
remaining = quantity
for batch in batches:
    if batch.expiry_date is not None and batch.expiry_date <
        date.today():
        continue  # never dispense expired stock
    take = min(batch.quantity, remaining)
    batch.quantity -= take
    stock_service.record(db, medicine=batch, kind="DISPENSE",
        quantity=-take, reference_type="prescription",
        reference_id=rx_id, user_id=user_id)
    remaining -= take
    if remaining == 0:
        break
```

CHAPTER 8

8. TESTING

8.1 Testing Strategy

The test strategy combines automated suites at several levels with a red-team exercise whose findings were turned into reproducible tests. Table 8.1 lists the levels, what each one covers and where it lives.

Table 8.1: Testing levels

Level	What is covered	Where
Unit	Parser, range rules, validators, safety filter, voice helpers, formatting helpers	`test_extraction_analysis`, `test_validation*`, `test_result_priority`, frontend `utils/*.test.js`
Integration	Routers with a real database per module: roles, patients, reports, prescriptions, pharmacy, clinical workflows, billing, lab, blood bank, OT	`test_auth_rbac`, `test_access_control`, `test_workflow`, `test_clinical_workflows`, `test_pharmacy_stock`, `test_finance` and others
System	Seven end-to-end acceptance workflows across modules (registration to paid consultation, lab order to report, inpatient stay to final bill, emergency to admission, prescription to pharmacy and others)	`test_acceptance_workflows`, `test_complete_hospital_workflow`
Security and chaos	Endpoint-by-role matrix, IDOR sweeps, token and upload attacks, AI-integrity attacks, business-logic abuse, deployment checks, route drift	`tests/security_and_chaos/*`, `test_idor_sweep`, `test_idor_hms`, `test_phase9_security`
Fuzz	Every write endpoint by every role with malformed, extreme and hostile bodies; read endpoints with odd query values; hostile uploads	`test_fuzz_no_500`, `test_validation_audit_api`
Concurrency	Parallel requests for booking, registration, consultation save and form re-submission; row-lock tests for money and stock	`test_concurrency_flows`, `test_phase9_security`, race tests in the bust suite
Accessibility	axe-core audit of the login page, the assistant and the disclaimer banner in a simulated browser	`frontend/src/__tests__/a11y.axe.test.jsx`
Mobile and UI chaos	Phone-width layout, voice panel on a phone, error boundary, route guards, session and cross-tab logout, voice commands	`AssistantChat.mobile.test`, `Login.mobile.test`, `__tests__/chaos/*`; browser sweep script `ui_chaos_playwright.py` (run during the red-team, not part of the final run)

Static checks (`ruff`, `bandit`) and the 25-question assistant sweep are recorded in `docs/TESTING_AND_HARDENING.md`; they were not re-run for this report. No code-coverage tool was run, so no coverage percentage is claimed.

8.2 Test Environment and Method

The tests were run on 4 November 2026 on a Windows 11 laptop with Python 3.13.1, pytest 9.1.1, Node.js 24.13.0 and Vitest 2.1.9. The repository head was commit `0316d2a`, and the working tree held 21 modified tracked files and several untracked test files at that moment (the report describes the working tree, not only the last

commit). Several test files cited in this chapter (`test_idor_hms`, `test_concurrency_flows`, `test_validation_audit*`, `test_sample_report_folder`, `test_route_drift`, `a11y.axe.test.jsx`) were untracked in git at that moment. Every backend test uses a fresh temporary SQLite database and upload folder, the language-model key is unset, and no OCR engine is installed. A single serial run of the backend suite would have taken several hours because of the two exhaustive hostile-input sweeps, so the 52 backend test files were divided into eight groups and run as parallel pytest processes, each with its own temporary database. Two of the groups were stopped part-way because they were progressing too slowly and were restarted as one process per test file, so the results below come from 25 pytest processes in total, whose junit reports were combined; each test was run once. No test was skipped or deselected: the `slow` marker was not excluded.

8.3 Results

Table 8.2 gives the totals. In the backend, 1,145 of 1,170 tests passed, 1 failed, 24 are expected failures and 0 were skipped. An expected failure (xfail) is a test written for a confirmed finding of the red-team exercise: it asserts the secure behaviour and is marked as a strict expected failure while the bug exists, so that the day the bug is fixed the test fails loudly and the mark is removed. An xfail is therefore an open finding, not a pass. In the frontend, 276 of 276 tests passed and 0 failed, in 33 test files.

Table 8.2: Automated test results of 4 November 2026

Suite	Tests	Passed	Failed	Expected failures	Skipped
Backend (pytest)	1,170	1,145	1	24 (open findings)	0
Frontend (Vitest)	276	276	0	0	0
Total	1,446	1,421	1	24	0

Table 8.3 splits the backend tests by area. The grouping is by file name and is approximate at the edges.

Table 8.3: Backend tests by area

Area	Backend tests
Security and chaos	171
Fuzz and validation audit	356
Concurrency	6
Assistant and AI	108
Clinical, pharmacy, finance and other integration tests	529
Total	1,170

The longest single process (the group with the fuzz and validation-audit suites) took 63.2 minutes. The frontend suite ran in one Vitest process.

8.4 Test Cases

Table 8.4 lists representative test cases across the modules named in the brief: login, validation, date of birth, role-based access and IDOR, report extraction, prescriptions, pharmacy stock, assistant refusal and file upload. Each row is a real test in the repository; the identifier TC-nn is local to this report and the second line of the first column names the test file. The Actual Output column describes what the test checked and observed in the run of 4 November 2026, and the last column is the outcome of that run. Parametrised tests are one row, and the number of inputs is given where useful.

Table 8.4: Summary test cases

Test Case ID	Input	Expected Output	Actual Output	Pass/Fail
TC-01 `auth_rbac`	Login with a wrong password; login with an unknown e-mail.	Both answers are 401 with the same message.	Both returned 401 with identical detail text.	Pass
TC-02 `auth_rbac`	Five wrong passwords for one account, then the correct password.	429 'Too many failed attempts', even for the right password; other accounts unaffected.	429 returned; a different account logged in with 200.	Pass
TC-03 `auth_rbac`	Log in with a generated temporary password and call GET /api/patients.	403 'change your temporary password'; only /me and change-password allowed.	Blocked with 403 and the message; change of password accepted.	Pass
TC-04 `auth_rbac`	Change password to 'abcdefgh', to the same password, then to 'Another456'.	422 for the weak and the unchanged password; 200 for the valid one; the old token then fails.	422, 422, 200; old token returned 401; new token 200.	Pass
TC-05 `auth_rbac`	Read the stored hash of a seeded user.	Not the plain password; prefix of the bcrypt-of-SHA-256 scheme.	Hash differed from the password and started with the scheme prefix.	Pass
TC-06 `auth_rbac`	Administrator deactivates an employee; the employee logs in and uses an old token.	Login 403; old token 401.	403 on login and 401 on the old token.	Pass
TC-07 `otp_login`	Request a mobile code, send five wrong 6-digit codes, then the right code.	Each wrong code 401; the right code is refused afterwards.	Five 401 answers; the right code also returned 401.	Pass

Test Case ID	Input	Expected Output	Actual Output	Pass/Fail
TC-08 `otp_login`	Inspect the stored login code row.	A 64-character hash; the code itself is not stored.	Hash length 64 and the code not contained in it.	Pass
TC-09 `validation`	13 passwords, including 'abcdefg1', '12345678', 'password1', 'Welcome@123' and a 129-character one.	Accepted only with 8 to 128 characters, a letter and a number, not a common password.	All inputs accepted or refused as specified.	Pass
TC-10 `validation`	15 inputs, including ' Rahul.Patil@Example.COM ', 'a b@c.com', 'a@@b.com', a 250-character name, '<script>@b.com'.	Valid forms accepted and lower-cased; the others refused.	All inputs behaved as specified.	Pass
TC-11 `validation`	16 inputs, including '+91 98765 43210', '1234567890', '9876543210; DROP' and foreign '+' numbers.	Indian numbers normalised to 10 digits; invalid or hostile text refused.	All inputs behaved as specified.	Pass
TC-12 `validation`	Today, 1930-01-01, tomorrow, 2999-01-01, 1929-12-31, 0001-01-01 and 121 years ago.	Today and 1930-01-01 accepted; the others refused.	Each boundary was accepted or refused as specified.	Pass
TC-13 `validation`	'2026-02-30', 'not-a-date', '31/12/1990', '9999-99-99', 12345.5.	Refused.	All five were refused.	Pass
TC-14 `validation`	Eight hostile login bodies: broken JSON, 1,000 nested arrays, invalid bytes, null, and others.	400 or 422, never 500.	No 500 was returned for any body.	Pass
TC-15 `access_control`	A second patient requests the first patient's report, file, list, record and comparison.	403 or an empty list.	All requests refused or empty.	Pass
TC-16 `access_control`	An unrelated doctor reads and writes (history, patient list, notes, prescription) for a patient who is not theirs.	403, empty lists, no write.	All refused.	Pass
TC-17 `access_control`	Administrator opens a report, its file and the comparison.	No parameters; file 403; comparison 403.	Parameters empty; both other calls 403.	Pass
TC-18 `access_control`	Reception updates a phone number together with allergies and medications.	Phone changes; clinical fields stay as they were.	Phone updated; allergies and medications unchanged.	Pass
TC-19 `access_control`	Patient opens a report before and after the doctor approves it.	No values, no AI summary and no AI step in the timeline before; values after.	Hidden before review and visible after.	Pass
TC-20 `idor_sweep`	Another patient and an unrelated doctor call about 35 id-based endpoints, read and write.	Every answer 403 or 404; nothing changed.	All answers were 403 or 404 and the data was unchanged.	Pass
TC-21 `idor_hms`	Another patient tries the hospital-module endpoints of patient 1 (bills, results, scans, export).	No call succeeds.	No successful call.	Pass
TC-22 `bac_matrix`	Each of 258 operations is called with no token and with each login role.	A role that is not intended never receives a success; no 500.	No unintended success and no crash.	Pass
TC-23 `extraction`	Range '10 - 20' with values 15, 25, 5, 10 and 20.	WITHIN, ABOVE, BELOW, WITHIN, WITHIN.	As expected for all five values.	Pass

Test Case ID	Input	Expected Output	Actual Output	Pass/Fail
TC-24 `extraction`	Range '< 100' with values 80, 120 and 100.	WITHIN, ABOVE, ABOVE (the bound is strict).	As expected.	Pass
TC-25 `extraction`	Value 27 with no printed range.	UNKNOWN; no range is invented.	UNKNOWN returned.	Pass
TC-26 `extraction`	Line 'Total Cholesterol / HDL Cholesterol Ratio 4.1'.	Not read as Total Cholesterol.	No Total Cholesterol parameter produced.	Pass
TC-27 `extraction`	'Glucose Fasting 5,6 mmol/L 3,9 - 5,6', 'Creatinine 1,1', 'Total Cholesterol 1,200'.	5.6 mmol/L with range 3.9 - 5.6; 1.1; 1200.	Values, unit and ranges as expected.	Pass
TC-28 `extraction`	'Hemoglobin 12.5 g/dL Male: 13.0 - 17.0 Female: 12.0 - 15.5' plus an ordinary Vitamin D row.	Hemoglobin UNKNOWN with the printed text kept; Vitamin D BELOW.	As expected.	Pass
TC-29 `ai_integrity`	'Vitamin D 15 ng/mL Deficient <20 Insufficient 20-30 Sufficient 30-100'; 'HDL Cholesterol 35 mg/dL Low <40 High >60'.	Vitamin D BELOW; HDL UNKNOWN (finding T-06).	As expected.	Pass
TC-30 `ai_integrity`	'Cannot rule out pulmonary embolism.', 'ST-segment elevation in the anterior leads.', 'Features of STEMI.'	Worth a look and urgent (findings T-02, T-03).	As expected.	Pass
TC-31 `sample_reports`	Each laboratory PDF of `sample_reports/` uploaded and read.	Real numeric values; no range where none is printed.	Values were real numbers and no range was invented.	Pass
TC-32 `workflow`	Upload an invoice text file and a file named x.exe.	422 'recognizable laboratory' and 415; nothing saved.	422 and 415; the report list stayed empty.	Pass
TC-33 `workflow`	Upload a scanned PDF while OCR is unavailable.	503 'OCR is not configured'; nothing saved.	503 with the message; nothing saved.	Pass
TC-34 `validation`	11 file names: a.pdf, a.PNG, a.jpg, a.txt, a.webp, a.exe, a.svg, a.html, 'a', 'a.pdf.exe'.	PDF, PNG, JPG, JPEG and TXT accepted; the others refused.	All 11 behaved as specified.	Pass
TC-35 `validation`	A JPEG named .png, a PNG named .jpg, HTML named .pdf, a GIF named .png.	Refused.	All refused.	Pass
TC-36 `uploads_static`	Upload with a traversal file name such as '../..'.	Stored under a random name inside the upload folder.	Stored under a random name; no path escape.	Pass
TC-37 `uploads_static`	Upload a file above the 10 MB limit.	Refused.	Refused.	Pass
TC-38 `workflow`	Employee uploads a PDF; doctor reviews, adds note, advice and prescription; patient views; pharmacy dispenses 4, then 7 (refused), 6 and the second item.	Each step allowed for its role only; over-dispensing refused; statuses and audit log follow.	The test failed in this run.	Fail
TC-39 `access_control`	Prescription dated 45 days ago is dispensed.	Refused (validity is 30 days).	Refused.	Pass
TC-40 `doctor_grades`	An intern creates a prescription.	403.	403.	Pass
TC-41 `doctor_grades`	A resident prescribes; the pharmacy searches and tries to dispense.	Status PENDING_COUNTERSIGN; hidden from the pharmacy.	Pending and not available to the pharmacy.	Pass
TC-42 `doctor_grades`	The consultant countersigns the resident's prescription; the pharmacy dispenses.	Active after countersign; dispensing allowed.	Dispensing succeeded after countersign.	Pass

Test Case ID	Input	Expected Output	Actual Output	Pass/Fail
TC-43 `business_logic`	Dispense above the prescribed quantity, and after cancellation or expiry.	Refused each time.	Refused each time.	Pass
TC-44 `business_logic`	Eight parallel dispenses of 5 units against a 5-unit prescription (finding B-01).	At most 5 units in total.	The finding still reproduces; the test is an expected failure.	Open finding (xfail)
TC-45 `pharmacy_stock`	Receive, sell and dispense stock.	One ledger movement per change with the balance after it.	Movements recorded with balances.	Pass
TC-46 `pharmacy_stock`	Recall a batch, then sell, dispense and write it off.	Sale and dispensing refuse the batch; write-off allowed.	Behaved as specified.	Pass
TC-47 `pharmacy_stock`	Requisition, approval, purchase order, goods receipt.	Stock increases once, not twice.	Stock updated once.	Pass
TC-48 `business_logic`	Bill the same prescription twice.	Second bill refused.	Second bill refused.	Pass
TC-49 `clinical_flows`	Admit two patients to one bed through the API and directly in the database.	409 from the API; the unique index rejects the direct write.	Both guards held.	Pass
TC-50 `patient_assistant`	Patient asks whether to stop a prescribed medicine.	Refusal with the prescription as written.	Refusal and the prescription shown.	Pass
TC-51 `patient_assistant`	Messages describing an emergency that is happening now.	Emergency guidance with the emergency number; no record lookup.	Guidance returned first.	Pass
TC-52 `patient_assistant`	A question about chest pain that appears in the records, with nothing happening now.	Answered from the records, not as an emergency.	Not treated as an emergency.	Pass
TC-53 `patient_assistant`	Patient asks about a report the doctor has not reviewed.	Told it is waiting for review; no values shown.	Waiting message; no values.	Pass
TC-54 `patient_assistant`	Patient asks about a test that has no result.	'Could not find a result'; nothing invented.	Missing result acknowledged.	Pass
TC-55 `patient_assistant`	Model reply that alters a number; no consent given.	No model call without consent; altered text rejected.	Plain answer shown in both cases.	Pass
TC-56 `patient_assistant`	A doctor's note containing an instruction to the assistant.	The answer is unchanged; the note is treated as data.	Answer unchanged; instruction not passed on.	Pass
TC-57 `patient_assistant`	Ask about a test that only another patient has.	No value of the other patient.	No leak.	Pass
TC-58 `patient_assistant`	Questions in Hindi and Marathi.	Answers in the same language.	Answers in Devanagari script.	Pass
TC-59 `voice.test.js`	Spoken sentences that are commands, and real questions containing command words.	Commands recognised; questions left alone.	11 voice tests passed.	Pass
TC-60 `a11y.axe.test.jsx`	axe-core on the login page (e-mail and mobile-code tabs).	No violations.	No violations found.	Pass
TC-61 `Login.mobile.test.jsx`	Mobile numbers in several styles; wrong code; demo code only when the server sends it.	Valid numbers accepted; wrong code shown as an error.	As expected.	Pass
TC-62 `errorboundary.chaos.test.jsx`	A page that throws while drawing.	A message and a way out instead of a blank screen.	Message and way out shown.	Pass

Of the 62 cases, 60 passed in the run of 4 November 2026.

The remaining cases were: TC-38 (test_complete_hospital_workflow): Fail; TC-44 (test_two_pharmacists_dispensing_the_same_item_at_the_same_moment_cannot_give_out_more_than_was_prescribed): Open finding (xfail). They are discussed in Section 8.6 and Chapter 9.

The one failing test. `test_complete_hospital_workflow` stops at its recommendation step: it posts a follow-up date of 15 January 2030 and receives a 422 instead of a 201. The cause is a stricter date rule in the working tree (the `PlannedDate` type in `utils/validators.py`, used by the recommendation schema), which refuses a planned date that far ahead, while the committed test still contains the fixed 2030 date. The behaviour of the application is therefore consistent with its new validation rules, and the test data is out of date. Because a test stops at its first failed assertion, the later steps of this end-to-end test (patient view, dispensing, audit log) were not executed in this run; they are covered separately by other cases of Table 8.4 (for example the dispensing, countersign and assistant cases) and by the acceptance workflows. The test was not changed before the run. Afterwards it was corrected to use a follow-up date 90 days after the day of the run (commit e3b5fac), and the test file `test_workflow.py` was run again on its own: all 12 of its tests passed, including this one. The full suite was not repeated, so the totals in Table 8.2 and the table above remain those of the original run.

8.5 Bust and Chaos Findings and Their Fixes

On 28 October 2026 the project was attacked on a local copy by a red-team exercise (`docs/BUST_REPORT.md`). Only synthetic data and a copy of the development database were used. All 128 operations then known were called by every role and with no token, and no role gained access it should not have had; forged, expired and tampered tokens, mass assignment, upload tricks, static-file exposure and cross-origin attacks failed. The exercise nevertheless produced 38 findings: 8 High, 18 Medium and 12 Low or informational. Each is reproduced by a test marked with the finding identifier. Table 8.5 lists the main findings by theme, with their status as shown by the final run of this report.

Table 8.5: Main findings of the bust report and their status

IDs	Finding	Severity	Status
T-02, T-03, T-04, T-06	Clinical reading: 'cannot rule out' read as normal; serious findings (STEMI, aortic dissection, free air) not flagged; tiered ranges read from the wrong tier.	High	Fixed: uncertainty rewrite before negation; urgent words imply attention; inline tiers use the tier that means normal.
A-01, A-02, A-03	Lockout per e-mail could lock out the owner; passwords equal after 72 bytes; no maximum length.	Medium, Low	Fixed: lock per e-mail and address; SHA-256 before bcrypt; 128-character cap.
D-01, J-01, X-01, X-02	30 MB body read; nested JSON gave 500; birth dates of year 1 to 9999; NUL in names.	Medium, Low	Fixed: 1 MB body limit (12 MB uploads), 400 or 422 for deep JSON, birth date from 1930, control characters refused.
F-02 to F-06, V-01	Drafts kept after logout; no error boundary (68 blank-screen cases); other tabs stay logged in; wake phrase fires on normal speech; spoken 'start over' deletes the chat; sideways scroll at 375 px.	Medium, Low	Fixed: drafts cleared; error boundary; storage event; greeting required; confirmation; layout wrap.
S-03	`schema.sql` lacked 19 of 37 tables.	Medium	Fixed: file generated from the models and tested.
B-01	Dispensing race: parallel calls gave out up to five times the prescribed quantity.	High	See TC-44 for the status in the final run.
R-01, R-02, U-01, U-02	Quadratic regular expression on a long line; no cap on analysed text; decompression bomb image; huge PDF page.	High, Medium	See the list of open findings below.
P-01, S-01, S-02, L-01	Personal details sent to an outside AI service; AI wording passes only the doctor filter; AI can turn a flagged finding normal; a non-text AI reply crashes the upload.	High, Medium	See the list of open findings below.
B-02 to B-05	Report re-review without history; stock changes not audited; denied access attempts not logged; audit trail anonymised after user deletion.	Medium	See the list of open findings below.
D-02, O-01, O-03, A-04	Vulnerable Pillow and pypdf versions; default secret in the compose file; unpinned packages; two first administrators at the same moment.	High, Medium, Low	See the list of open findings below.

In the final run, 18 findings were still reproduced by strict expected-failure tests: A-04, B-01, B-02, B-03, B-04, B-05, C-01, D-02, L-01, O-01, O-03, P-01, R-01, R-02, S-01, S-02, U-01, U-02. The red-team report orders them for fixing and records that its first fixes were limited to the clinical-reading, validation and browser findings; the later hospital phases added audit and row-locking fixes for money, refunds and blood units (`docs/HMS_PHASE7_to_10.md`).

8.6 Observations and Limits of the Testing

- The endpoint matrix and the route-drift test cover the 258 operations known when they were written. The later routers (blood bank, operation theatre, finance, claims, procurement, stock, radiology, diagnostics and integrations) are covered by their own module tests, `test_idor_hms`, `test_phase9_security` and the fuzz suite, but not by the matrix.

- OCR was not exercised with a real engine because Tesseract is not installed in the test environment; the OCR path was tested through its error handling and a stubbed engine.
- PostgreSQL and Docker were not run, so the row locks were exercised only in their SQLite form.
- Colour contrast, a keyboard-only walk and a screen-reader check were not performed; the accessibility result covers the automated axe rules on three screens.
- One backend test (the complete hospital workflow) failed in the run because of an out-of-date fixed date in the test, as explained in Section 8.4; the test was corrected afterwards and passes when run alone.
- No test used real patient data, and no performance or load test was run.
- The tests describe the behaviour of the code on 4 November 2026; a pass is not a clinical validation.

CHAPTER 9

9. RISK ANALYSIS

The risks below were identified from the design, the red-team exercise and the test results. Likelihood and impact are qualitative ratings (Low, Medium, High) assigned for this report on the basis of the evidence in the repository; they are judgements, not measurements. The mitigation column lists only measures that exist in the code or the process, and the last column states what remains open. The open findings are those whose strict expected-failure tests still failed in the final run (Section 8.5).

Table 9.1: Risk register

ID	Risk	Likelihood	Impact	Mitigation implemented	Open or remaining
R1	Security: unauthorised access to patient data (privilege escalation, IDOR)	Medium	High	Two-layer access control (role, then patient relationship); endpoint-by-role matrix, IDOR sweeps and hospital-module IDOR tests; break-glass with a written reason and review; audit log.	The matrix covers 258 of 385 operations; later routers rely on module tests. Still open in the final run: B-04, B-05.
R2	Security: password guessing, token theft, account takeover	Medium	High	Password rules, SHA-256 plus bcrypt, lockout per e-mail and address, token fingerprint ending old tokens, one-time hashed mobile codes with 5 tries, forced change of temporary passwords.	Tokens live in the browser's local storage for 24 hours with no server-side logout (design note I-01); no second factor. Still open in the final run: A-04.
R3	Security and performance: hostile uploads (fake types, huge images, long lines)	Medium	Medium	Type decided by content, 10 MB limit, 1 MB body limit for JSON, random stored names, downloads with no-sniff headers, nothing saved when parsing fails.	Still open in the final run: R-01, R-02, U-01, U-02.
R4	Data loss: failed disk, accidental deletion, migration error	Medium	High	Start-up migration only adds tables and columns; histories are hidden, never deleted, with an append-only change log; user deletion has a prior delete check; stock and money changes leave ledger rows.	No backup or restore feature exists; SQLite in a single file is suitable for development only. Backups must be arranged by the deployer.
R5	Privacy and legal: personal data sent to an outside AI service; compliance with data-protection law	Medium	High	The language model is optional and off by default; assistant wording needs the patient's consent; redaction of names, contacts and addresses; consent records; chat retention of 90 days; the audit log never stores assistant text.	Redaction is partial. Still open in the final run: P-01. No legal review against the DPDP Act has been done; this is stated in the disclaimer.
R6	Wrong AI or analysis output (misread value, missed urgent wording, unsafe model text)	Medium	High	Status by deterministic rules from the printed range only; UNKNOWN instead of a guess; sex-specific and tiered ranges not guessed; safety filter on every model sentence; model cannot add or change numbers; doctor review before the patient sees anything.	Free-text reading is wording-based and can miss or misread statements; English only; OCR quality varies. Still open in the final run: S-01, S-02, L-01.

ID	Risk	Likelihood	Impact	Mitigation implemented	Open or remaining
R7	Assistant gives unsafe advice or misses an emergency	Low	High	Safety rules run before retrieval; fixed emergency guidance; refusal of medicine changes and diagnosis; answers built from records by code; 59 golden questions and 23 emergency phrases checked in earlier test passes.	Keyword understanding can fall back to a help message; Hindi and Marathi wording and emergency keyword lists have not been reviewed by clinicians.
R8	Concurrency errors: double dispensing, double payment, bed or blood unit conflicts	Medium	High	Row locks for money, stock, claims and blood units; unique indexes for beds and admissions; database conflicts turned into a 409 message; parallel-request tests.	Locks were tested on SQLite only. Still open in the final run: B-01.
R9	Performance and availability: slow requests, denial of service	Medium	Medium	Request and file limits; bounded regular expressions in the parser; assistant rate limit of 20 messages a minute.	Rate limits are per process and in memory; no load test was run; a single SQLite writer limits throughput.
R10	Supply chain: vulnerable or unpinned dependencies	Medium	Medium	Dependencies are listed with lower bounds; a test checks the installed versions.	Still open in the final run: D-02, O-03.
R11	Deployment misconfiguration (default secret, public API documentation)	Medium	High	Start-up refuses a short secret and the placeholder secret outside SQLite development; security headers and restricted CORS.	Still open in the final run: O-01, C-01. The interactive API documentation is public (C-01). Docker files were not run.
R12	Clinical thresholds and escalation rules are generic	Medium	High	Alert thresholds are editable by the hospital and documented as defaults that must be reviewed; no machine learning is used for alerts.	A clinical team must validate all thresholds before any real use.
R13	Timeline: very compressed schedule and little real-world validation	High	Medium	Work in ten short phases, each with its own document and tests; a red-team exercise in the middle of the project; honest status screens for features that are not built.	The system was built in 27 calendar days; no pilot with real users was carried out; effort was not tracked in hours.

Table 9.1 shows that the largest remaining risks are not in access control, which was the best-tested part of the system, but in three other places: the optional AI path (privacy of text sent out and the strictness of the safety filter for patient-visible wording), operational hardening (secrets, dependency versions, backups) and the absence of clinical and legal review. Each of them is outside what an educational project can close alone and is carried into Chapter 10.

CHAPTER 10

10. LIMITATION AND FUTURE SCOPE

10.1 Limitations

The following limits are stated plainly so that the system is not mistaken for something more complete than it is.

10.1.1 Clinical and legal status

- The software is not clinically certified, is not a medical device under any regulation, and has had no clinical validation. Every page carries the disclaimer that it provides educational, AI-assisted analysis that a qualified professional must review.
- Compliance with the Digital Personal Data Protection Act, 2023 [4], clinical-establishment rules and medical-software standards such as IEC 62304 [20] has not been assessed.
- Alert thresholds, blood-bank rules, reference ranges and the emergency keyword lists are defaults chosen by the developer and need review by clinicians.
- Hindi and Marathi wording of the assistant was written without a native-speaking clinician's review.

10.1.2 Technical limits

- **Database.** Development and all tests use SQLite. PostgreSQL is supported by configuration and `schema.sql`, but it was not run in this project, and the row locks that matter there were exercised only in their SQLite form.
- **OCR.** Scanned reports and photos need Tesseract. It was not installed in the development environment, so the OCR path was tested through its error handling only; accuracy on poor scans is not known, and OCR depends heavily on scan quality.
- **Report reading.** Only numeric laboratory results are analysed as parameters (the parser knows a catalogue of common tests); free-text reports are read by wording rules that can miss or misread statements; reports are read in English only.
- **Assistant.** Question understanding is keyword-based, not a language model; a question that mixes topics gets one topic's answer. Speech recognition and

speech output depend on the browser and on the voices installed on the device; Firefox has no speech recognition.

- **Demo mode.** Mobile-code login shows the code on the screen when `OTP_DEMO_MODE` is on, because no SMS provider is connected. SMS and WhatsApp messages wait in an outbox and are never sent.
- **No native app.** The interface is a responsive web application. A Capacitor Android project was added to the repository on 4 November and a debug APK was built locally (the APK file itself is not tracked); the native speech support is new, was not tested on devices for this report, and no store release was prepared.
- **Schema evolution.** The database is upgraded at start-up by adding missing columns and tables; there is no migration tool with versioned scripts, so renames and removals would need manual work.
- **Rate limits and sessions.** Rate limits are kept in memory per process; tokens are valid for 24 hours with no server-side logout; the token is held in local storage.
- **Integrations.** There is no payment gateway, ABDM exchange, DICOM or PACS, analyser link or insurance portal link; the FHIR export is not validated.

10.1.3 Testing and process limits

- 18 red-team findings remained open in the final run (A-04, B-01, B-02, B-03, B-04, B-05, C-01, D-02, L-01, O-01, O-03, P-01, R-01, R-02, S-01, S-02, U-01, U-02); they are listed in Chapter 8 and Chapter 9.
- The endpoint matrix covers 258 of 385 operations. No coverage percentage, load test, colour-contrast check, keyboard-only walk or screen-reader test was run.
- No survey, interview or pilot with clinicians, pharmacists or patients is recorded; usability and adoption are therefore unmeasured (Section 3.2).
- Effort in hours and team size were not recorded; the COCOMO figures are an estimate for a conventional team (Section 6.2).
- Docker and Docker Compose files exist but were not run.

10.2 Future Scope

Table 10.1 lists the planned work by area, with the reason for each item.

Table 10.1: Future scope

Area	Planned work	Reason
Deployment	Run on PostgreSQL behind a reverse proxy with TLS; containerised deployment; automated backups and restore tests; pinned dependency versions with a lock file; versioned database migrations.	Closes the SQLite, Docker and dependency risks (R4, R10, R11).
Mobile	Finish the Capacitor Android and iOS builds, publish a Progressive Web App with offline reading of already released results, add push notifications for appointments and released results.	Extends the phone-first assistant and removes the need for browser speech support.
Messaging	Connect an SMS and WhatsApp provider and make mobile-code login production-grade with rate limits shared across processes.	Turns the outbox from adapter-only to working.
Interoperability	Validate the FHIR export against a FHIR server and ABDM profiles [5, 3]; connect the ABDM gateway, health-information exchange and consent manager.	Needed for exchange of records between facilities.
Imaging	A DICOM store and viewer (PACS) [21] behind the radiology module.	Replaces JPG, PNG and PDF scan files.
AI features	Retrieval over curated clinical guidance for the doctor's suggestions; translation of report explanations into more Indian languages; a clinician-reviewed glossary; evaluation sets with clinician labels; keep the rule-first and doctor-review design [13].	More help for patients and doctors, with measured accuracy.
Laboratory	Analyser connectivity; reference ranges by age and sex when the patient's sex is known; delta checks.	Reduces manual entry and the number of UNKNOWN ranges.
Security	HTTP-only cookies with a content-security policy, two-factor login, server-side session revocation, audit of denied attempts, and closing the remaining red-team findings.	Addresses R2 and the open findings.
Quality	A coverage report, load and endurance tests, a full accessibility audit with keyboard and screen-reader checks [6], and a pilot with real staff under supervision.	Turns assertions about usability and performance into measurements.
Compliance	Legal and clinical review for the DPDP Act [4] and medical-software lifecycle requirements [20] before any real-world use.	Prerequisite for deployment in a hospital.

CHAPTER 11

11. USER MANUAL

This manual explains how to install and start HealthWise AI, how to sign in, and how each role uses its main screens. All screenshots were taken by the author of the report from the running application on 4 November 2026, in a desktop browser at a width of 1366 pixels. They show only the synthetic demonstration data created by `python -m app.seed_demo` and by uploading the files of `sample_reports/`; they contain no real patient data.

11.1 Installing and Running the Software

11.1.1 Requirements

- Python 3.11 or later and Node.js 20 or later (the report environment used Python 3.13.1 and Node.js 24.13.0).
- Optional: Tesseract OCR, only for scanned PDFs and photographs. On Windows the installer of the UB Mannheim project is commonly used; on Ubuntu, `sudo apt install tesseract-ocr`. Without it, text PDFs and text files still work, and a scan returns the message 'OCR is not configured'.
- A modern browser. Chrome and Edge provide speech recognition for the voice assistant; other browsers allow typing and reading aloud only.

11.1.2 Starting the backend

Open a terminal in the project folder and run the following commands. The seed command creates demonstration accounts and sample data; the server then listens on port 8000, and the interactive API documentation is available in a development setup.

```
cd backend
python -m venv .venv
.venv\Scripts\activate          (Linux or macOS: source
    .venv/bin/activate)
pip install -r requirements.txt
python -m app.seed_demo
uvicorn app.main:app --reload
```

11.1.3 Starting the frontend

In a second terminal:

```
cd frontend
npm install
npm run dev
```

The interface opens at `http://localhost:5173`; requests to `/api` are forwarded to the backend on port 8000 by the development server. If port 5173 is taken, the tool chooses another port and prints it.

11.1.4 Environment variables

Settings are read from `backend/.env`. Copy `backend/.env.example` and edit it; Table 11.1 lists the variables. The default database is a local SQLite file, which is correct for demonstration. Do not reuse the demonstration secret or password for real use.

Table 11.1: Main environment variables

Variable	Meaning	Default or note
`DATABASE_URL`	Database connection string	Empty uses the local SQLite file; PostgreSQL example `postgresql+psycopg2://user:pass@host:5432/db`
`JWT_SECRET`	Key that signs login tokens	Must be long and random outside SQLite development
`LLM_API_KEY`	Key for the optional AI wording service	Empty: the application works fully without it
`UPLOAD_DIR`, `MAX_FILE_SIZE`	Folder for uploaded files; size limit	Private folder inside the backend; 10 MB
`TESSERACT_CMD`	Path of the OCR program if it is not on the path	Empty
`CORS_ORIGINS`	Browser origins allowed to call the API	The local development addresses
`OTP_DEMO_MODE`	Shows the mobile login code on screen	Off; for local demonstration only

11.1.5 Seed data and demonstration accounts

`python -m app.seed_demo` creates the accounts of Table 11.2. The password of every demonstration account is `Demo@12345`. On a new database with no users, the login page offers to create the first administrator instead. In the run used for this manual, the sample files `Report_A_July_Rahul.pdf`, `Report_B_October_Rahul.pdf`, `Chest_Xray_Rahul.pdf` and `Prescription_Rahul.txt` were then uploaded as reception, the two laboratory reports were approved by the doctor, a note, advice and a prescription were added, and the pharmacy dispensed four units of the first medicine. The sixth sample, `Not_a_health_report.txt`, was rejected as intended.

Table 11.2: Demonstration accounts (password `Demo@12345`)

Role	E-mail
Administrator	admin@healthwise.demo.com
Doctor (consultant)	anjali.sharma@healthwise.demo.com

Role	E-mail
Reception (employee)	priya.joshi@healthwise.demo.com
Patient	rahul.patil@healthwise.demo.com
Pharmacy	pharmacy@healthwise.demo.com
Laboratory technician	lab@healthwise.demo.com
Pathologist	pathologist@healthwise.demo.com
Radiology technician / radiologist	radiology.tech@healthwise.demo.com / radiologist@healthwise.demo.com
Nurse	nurse@healthwise.demo.com
Intern / PG resident	intern@healthwise.demo.com / pg.student@healthwise.demo.com
Store, OT, blood bank, billing, accountant, TPA desk	inventory@, ot@, bloodbank@, billing@, accounts@, tpa@ (all at healthwise.demo.com)

To run the automated tests, use `python -m pytest -q` in the backend folder and `npm test` in the frontend folder.

11.2 Logging In

Open the application in the browser. Figure 11.1 shows the login page. Type the e-mail address and password and press Log in. A second tab, Mobile code, lets a user whose phone number is in their profile ask for a six-digit code; in demonstration mode the code is shown on the screen. The Demo accounts list under the form fills in an e-mail address when an account is clicked. A user who still has a temporary password is asked to choose a new one before anything else. After five wrong attempts the page reports that too many attempts were made and says how many minutes to wait. After login, each role is taken to its own home page.

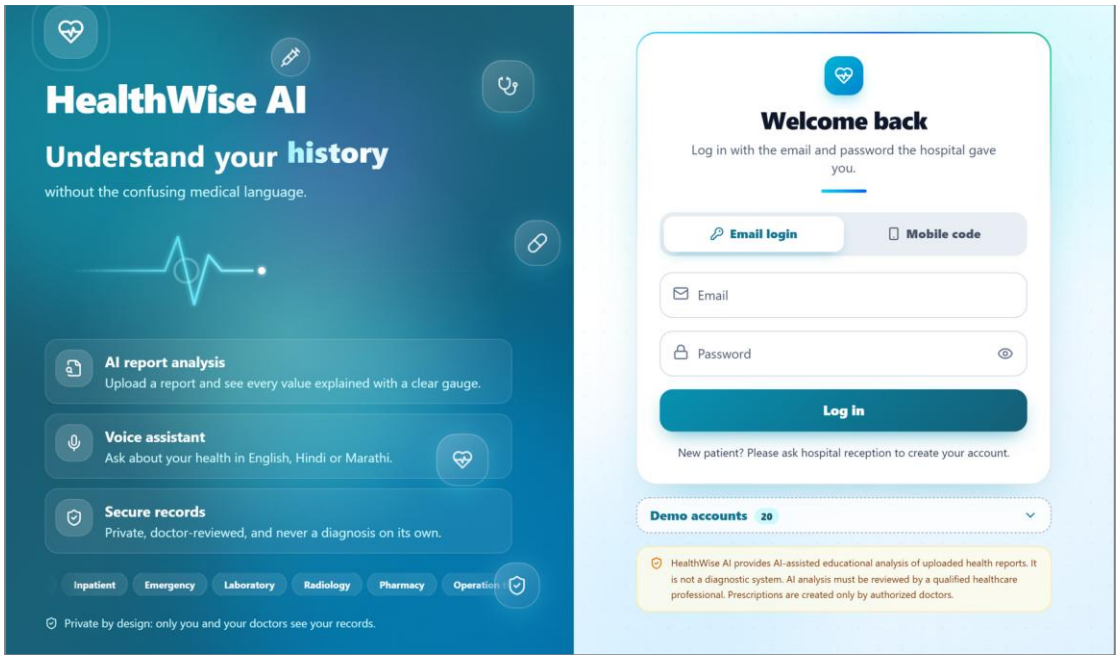


Figure 11.1: Login page

11.3 Patient

The patient menu on the left (Dashboard, Ask HealthWise, My Health History, My Reports, My Bills, My Timeline, Doctor's Notes, Doctor's Advice, My Medicines, Appointments, My Profile) leads to the screens below. The disclaimer is shown at the top of every page.

11.3.1 Dashboard

The dashboard (Figure 11.2) greets the patient, shows whether the latest report is still waiting for the doctor, and offers shortcuts to the next appointment, reports, medicines and timeline.

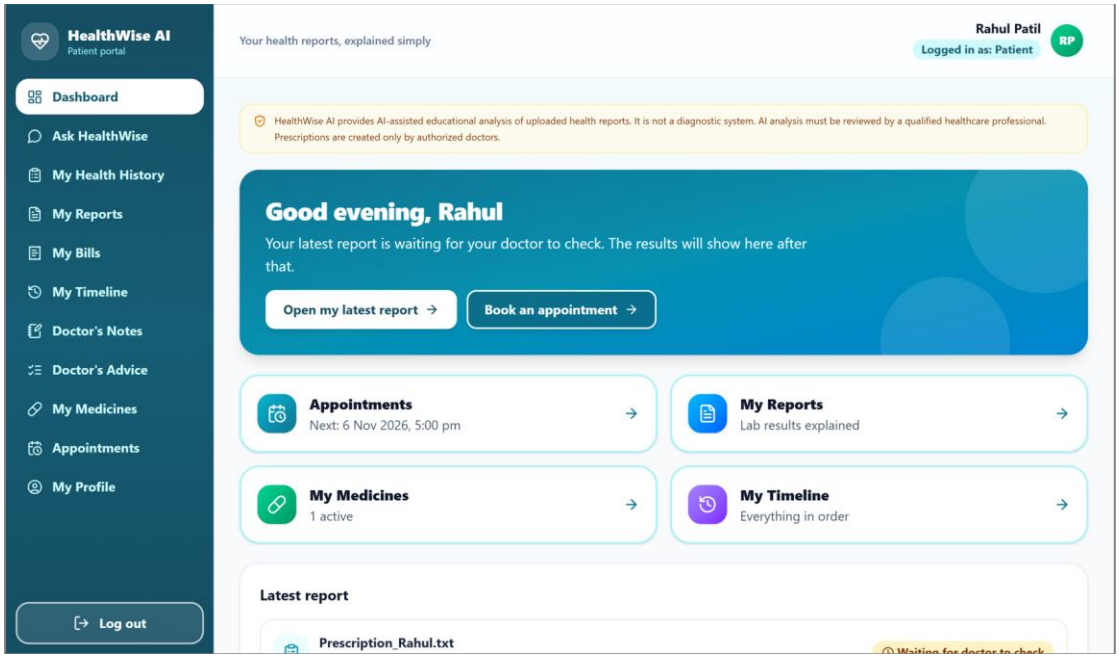


Figure 11.2: Patient dashboard

11.3.2 My Reports and report detail

My Reports (Figure 11.3) lists laboratory tests, scans and uploaded reports. A report that the doctor has not reviewed shows 'Waiting for doctor to check' and no values. After review, the patient opens the report (Figure 11.4) to read the summary, the explanation of each value against the range printed in the report, and the doctor's note.

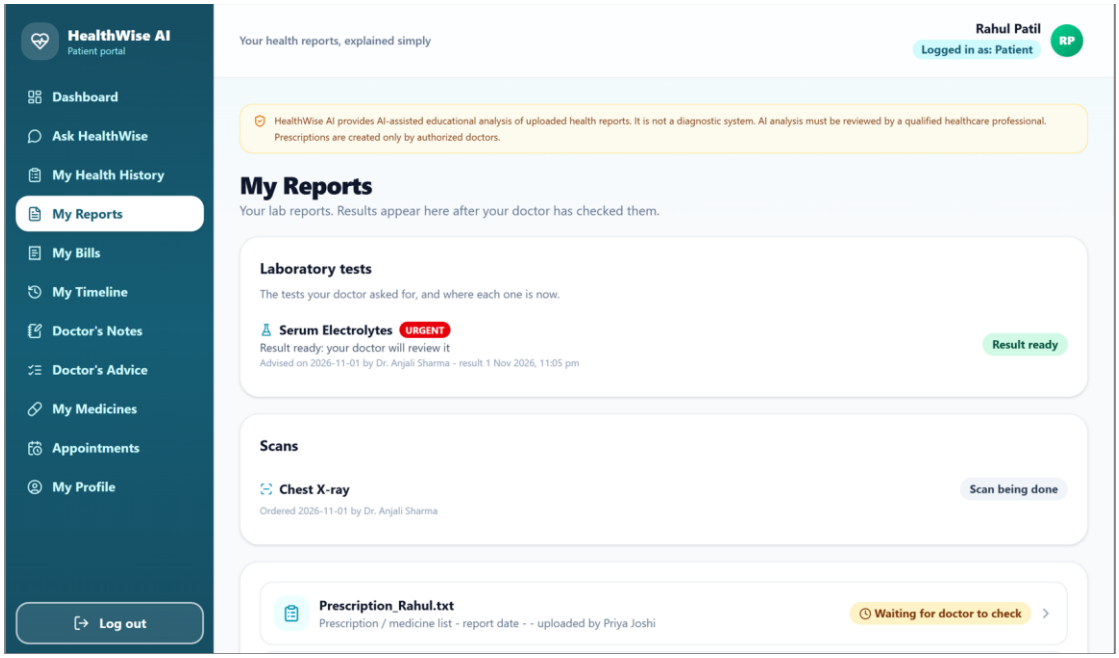


Figure 11.3: Patient report list

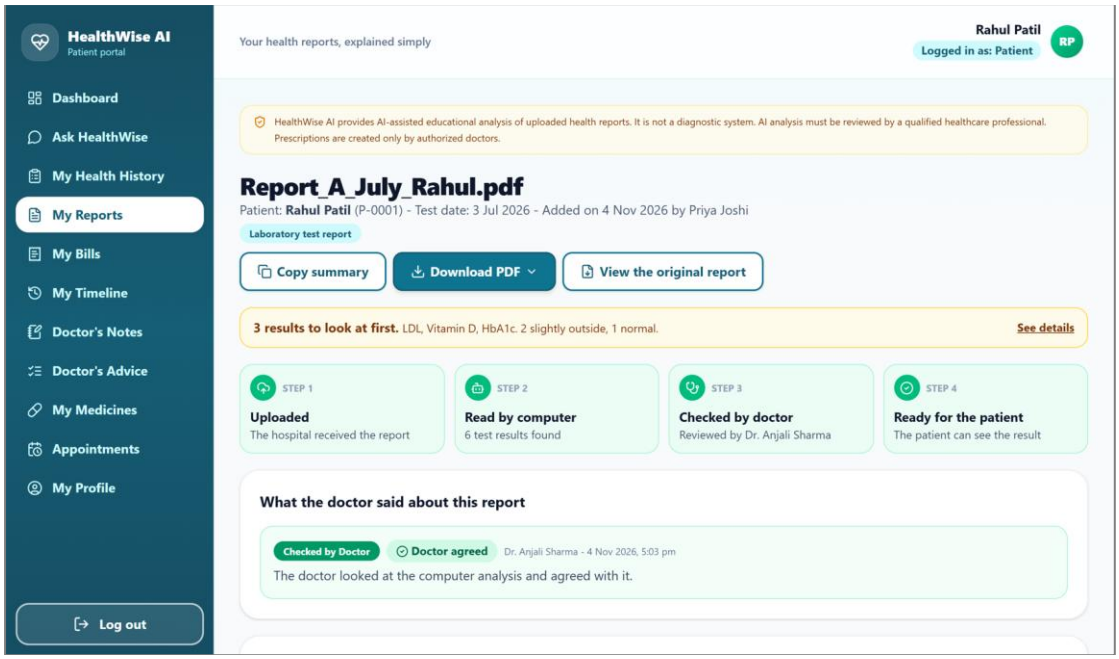


Figure 11.4: A reviewed report as the patient sees it

11.3.3 My Medicines

My Medicines (Figure 11.5) shows each prescription with its items, how many units have been given by the pharmacy, and a button to open or print the prescription sheet.

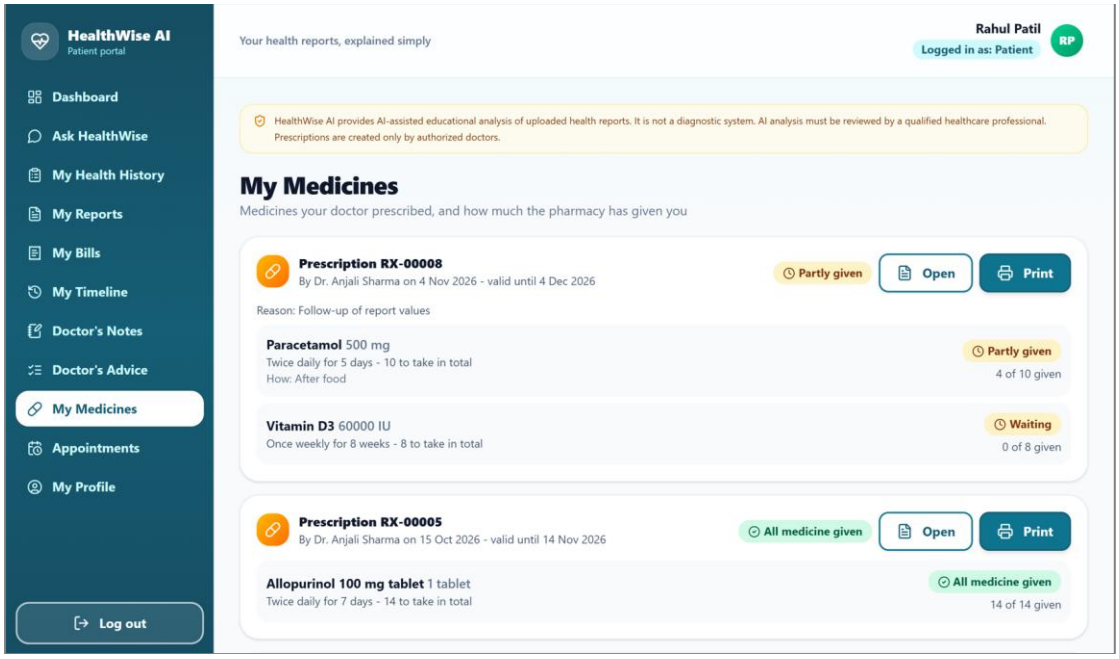


Figure 11.5: Patient prescriptions with dispensing status

11.3.4 Ask HealthWise

Ask HealthWise (Figure 11.6) answers questions about the patient's own records. Type a question or tap the large microphone for a hands-free conversation; choose English, Hindi or Marathi in the language box; use the suggested questions under the chat. Each answer shows the value, date and printed range from the record, a short general explanation and a reminder that the assistant is not a doctor. The bin icon deletes the conversation after confirmation. Requests to change a medicine or to name a disease are refused, and an emergency message receives fixed guidance first.

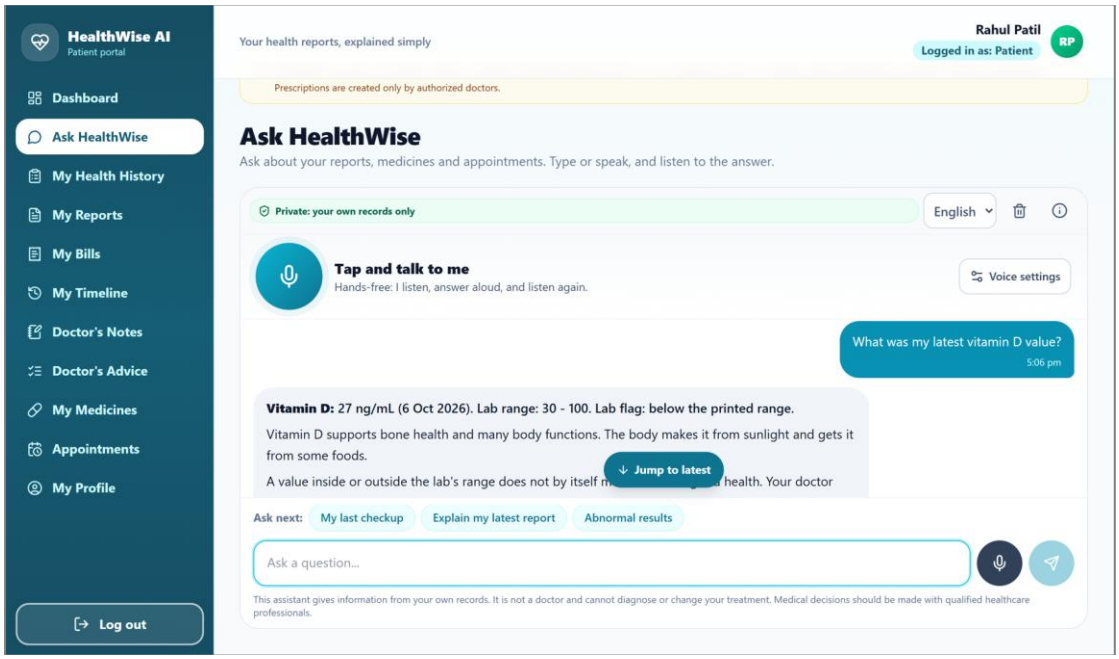


Figure 11.6: Patient assistant answering from a reviewed report

11.3.5 My Timeline

My Timeline (Figure 11.7) is the chronological history of appointments, reports, doctor's notes, advice, prescriptions and dispensing. Tabs add a comparison of reports, trends and the medical background.

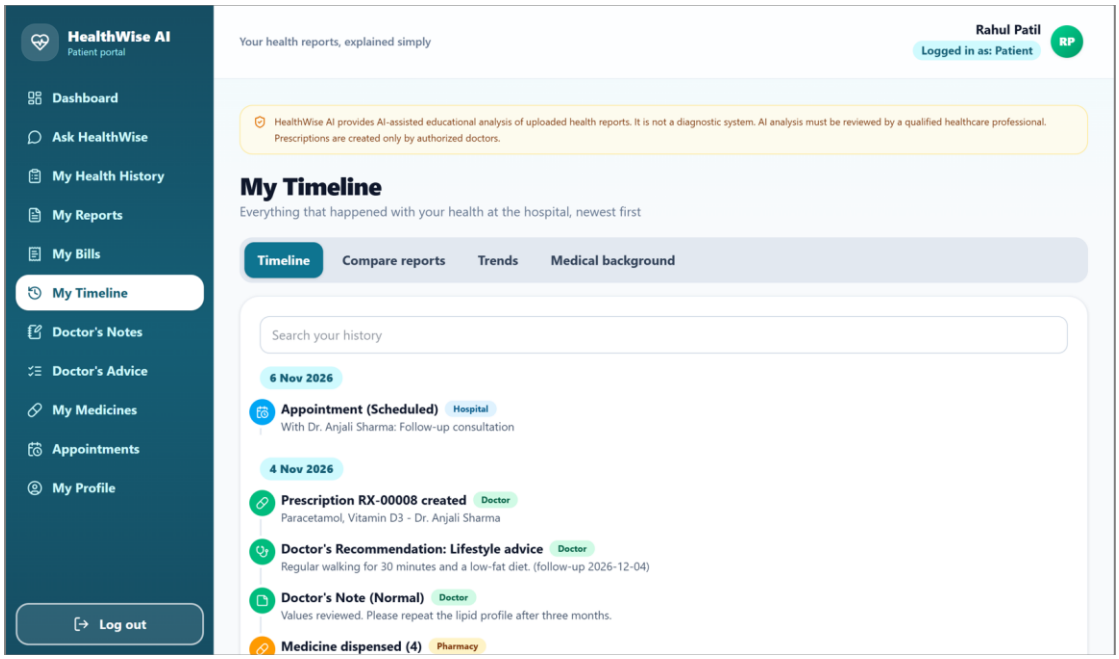


Figure 11.7: Patient timeline

11.4 Doctor

11.4.1 Dashboard

The doctor's dashboard (Figure 11.8) counts the reports waiting for review, urgent wording found in reports, today's appointments and follow-up tasks, and links to the patient list. The menu also holds Today's queue, Inpatients, Emergency, Referrals, Countersign, Beds, notes, advice and prescriptions.

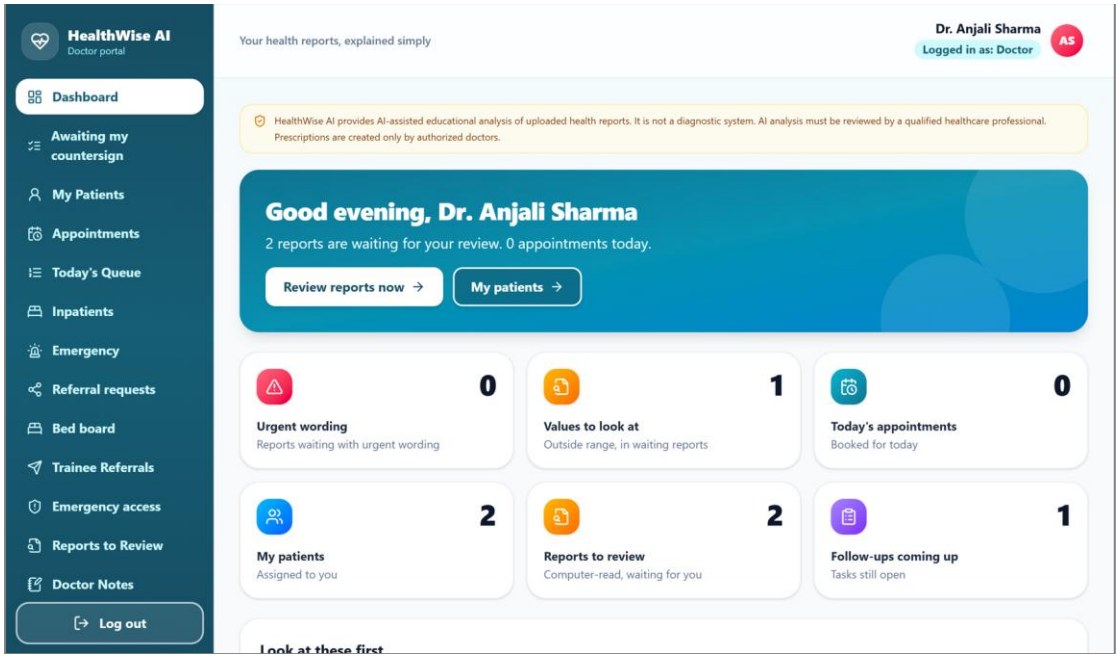


Figure 11.8: Doctor dashboard

11.4.2 Reviewing a report

Open a report from Reports to Review (Figure 11.9). The page shows a progress strip (uploaded, read by computer, checked by doctor, ready for the patient), the summary, the key statements marked as looking normal or worth a look, urgent flags, suggestions for the doctor and draft tips for the patient. The doctor can copy the summary, download a PDF, open the original file and mark the report as reviewed with a decision (approved, modified or clarify; a comment is required for the last two).

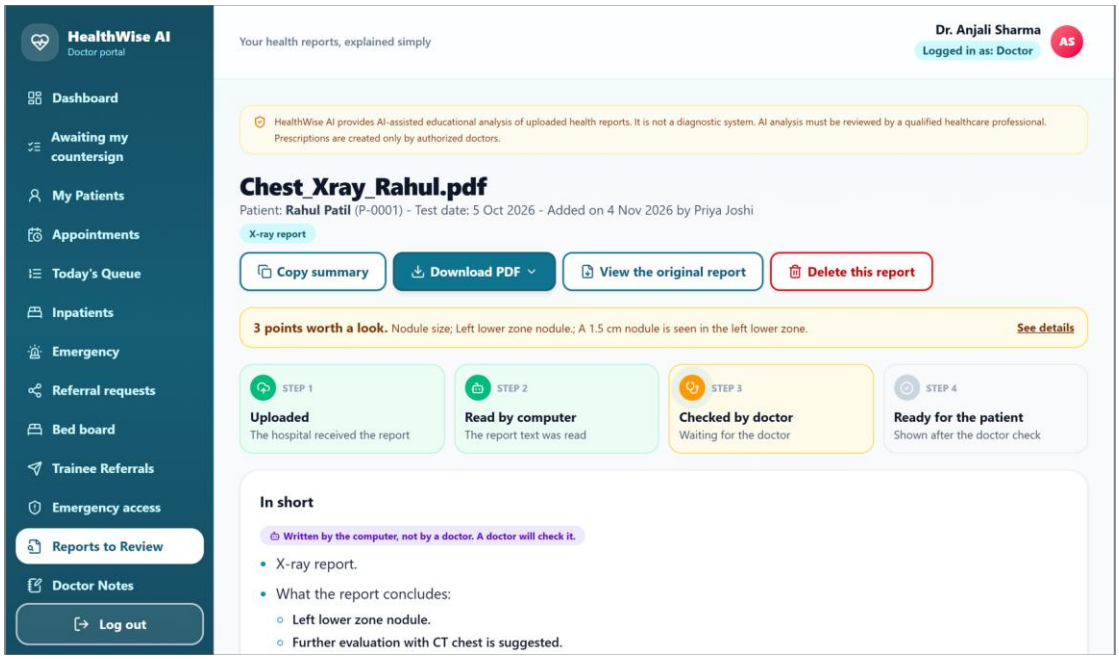


Figure 11.9: Report review page for the doctor

11.4.3 Patient page and prescriptions

A patient page (Figure 11.10) holds the clinical tabs: medical history, clinical summary, timeline, reports, notes and recommendations, and prescriptions. Buttons add a note, a recommendation, a prescription or a history entry. Prescriptions written by the doctor are listed with their dispensing status (Figure 11.11).

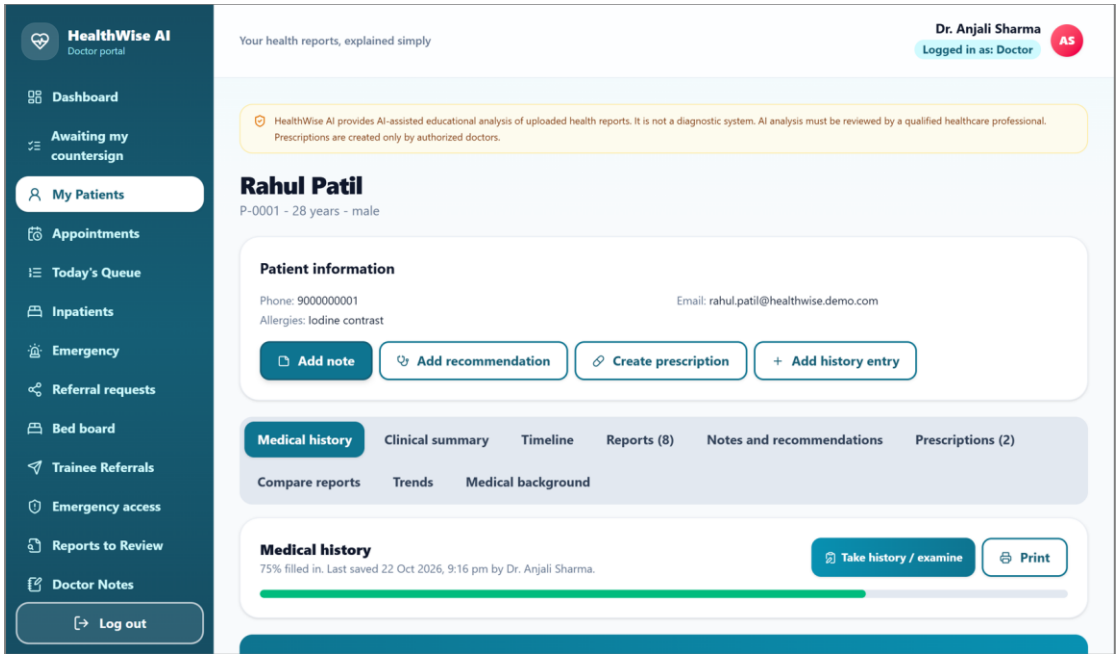


Figure 11.10: Doctor's patient page

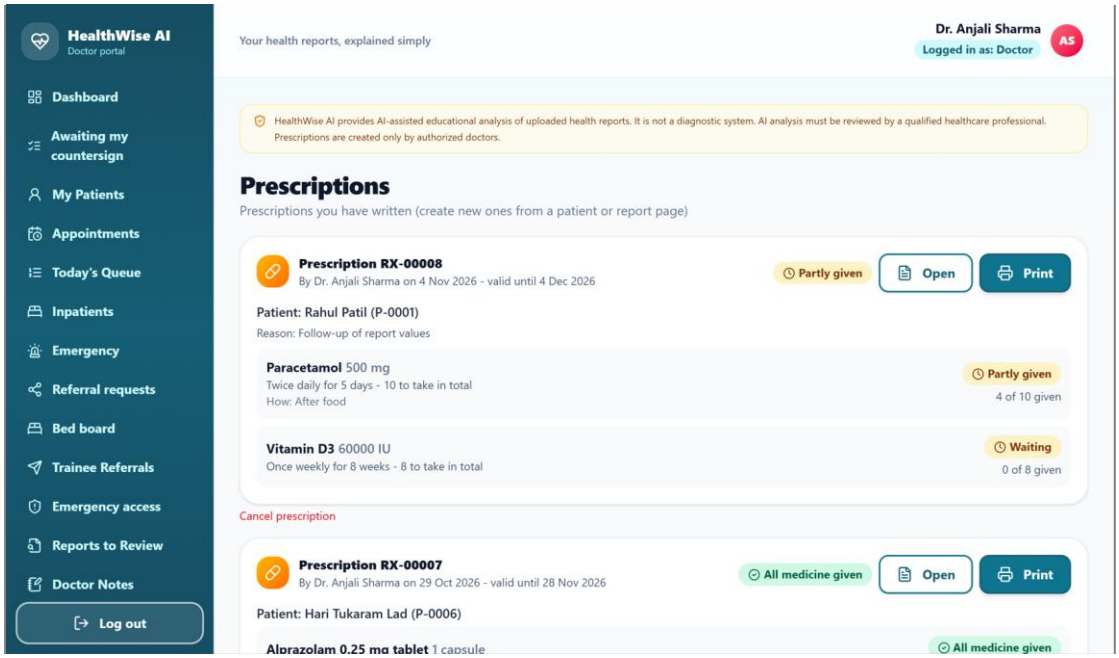


Figure 11.11: Prescriptions written by the doctor

11.5 Reception

The reception dashboard (Figure 11.12) offers Register patient, Upload report and Find a patient and shows appointments and reports waiting. Reception never sees clinical values.

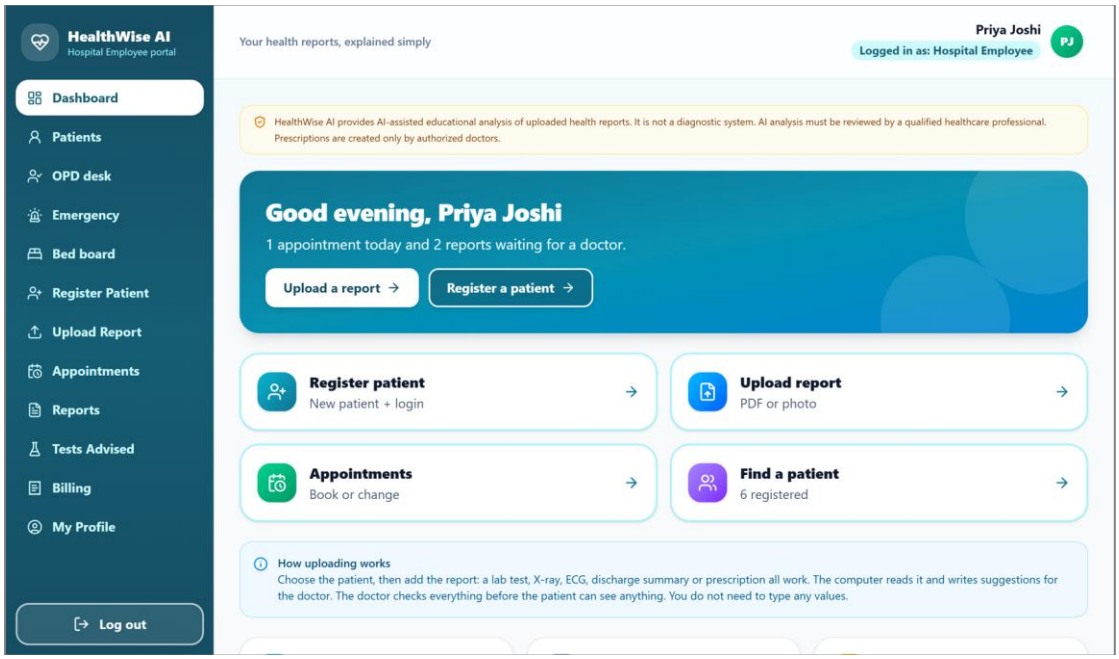


Figure 11.12: Reception dashboard

To register a patient, open Register patient (Figure 11.13), fill in the required fields (marked), and save. If a record with the same name and date of birth exists, the form lists it and asks for confirmation. The save creates the patient's login and a unique health identifier (UHID).

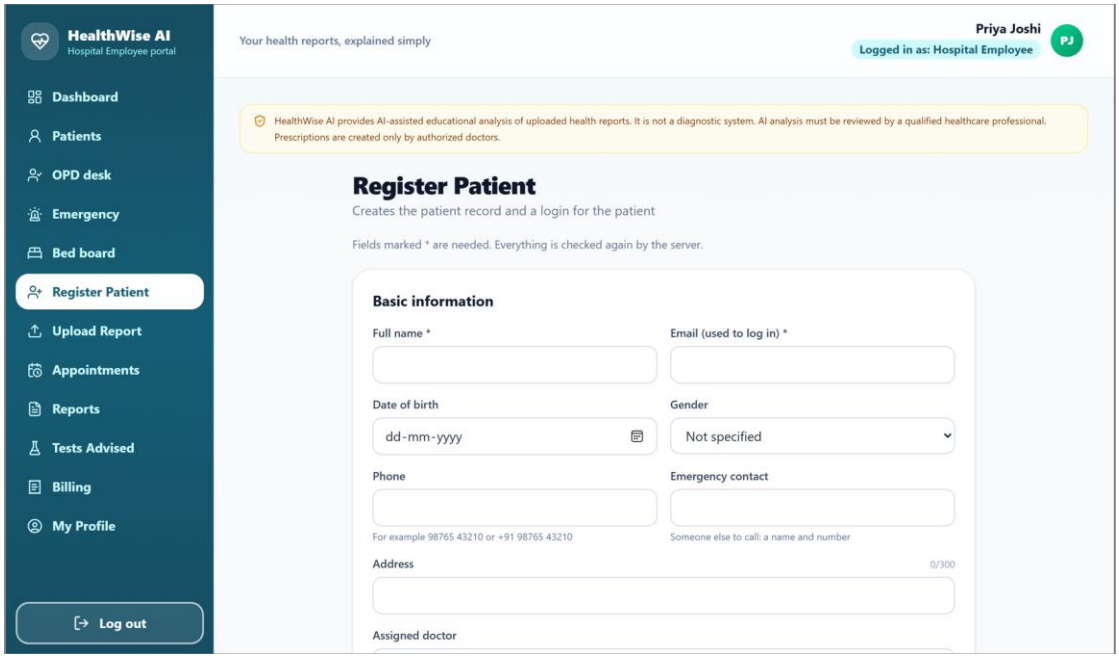


Figure 11.13: Patient registration form

To upload a report, open Upload Report (Figure 11.14): choose the patient, the doctor who should review it and the file. A file with no health content is rejected with a clear message and nothing is saved.

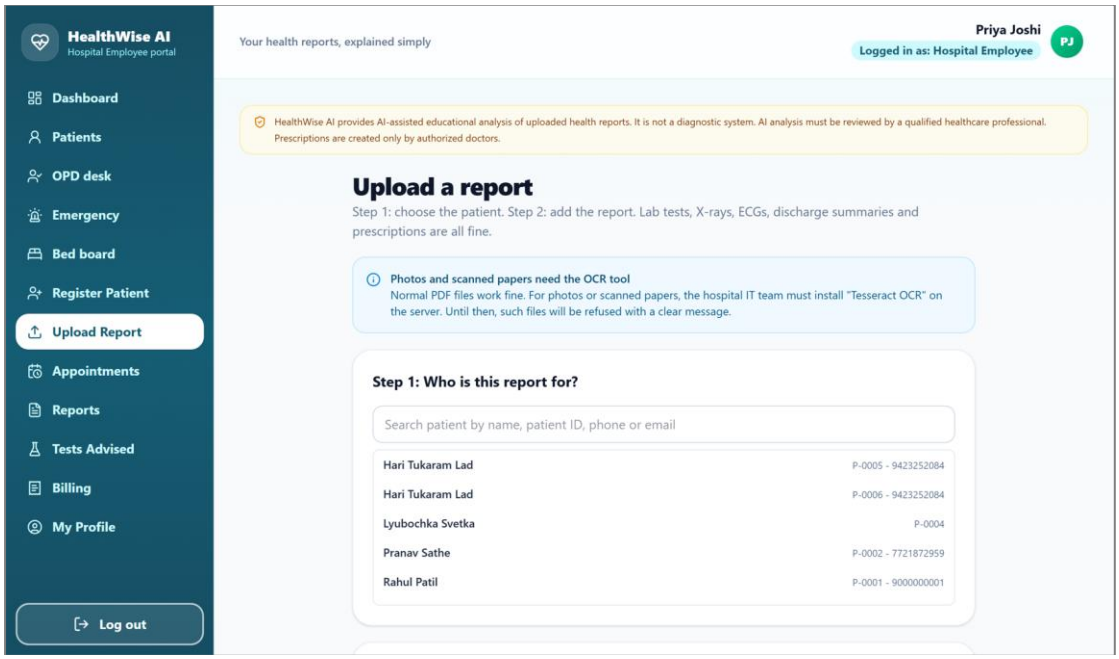


Figure 11.14: Report upload page

The OPD desk (Figure 11.15) checks in booked patients and walk-ins and gives each one a token for the chosen doctor; urgent patients are called first.

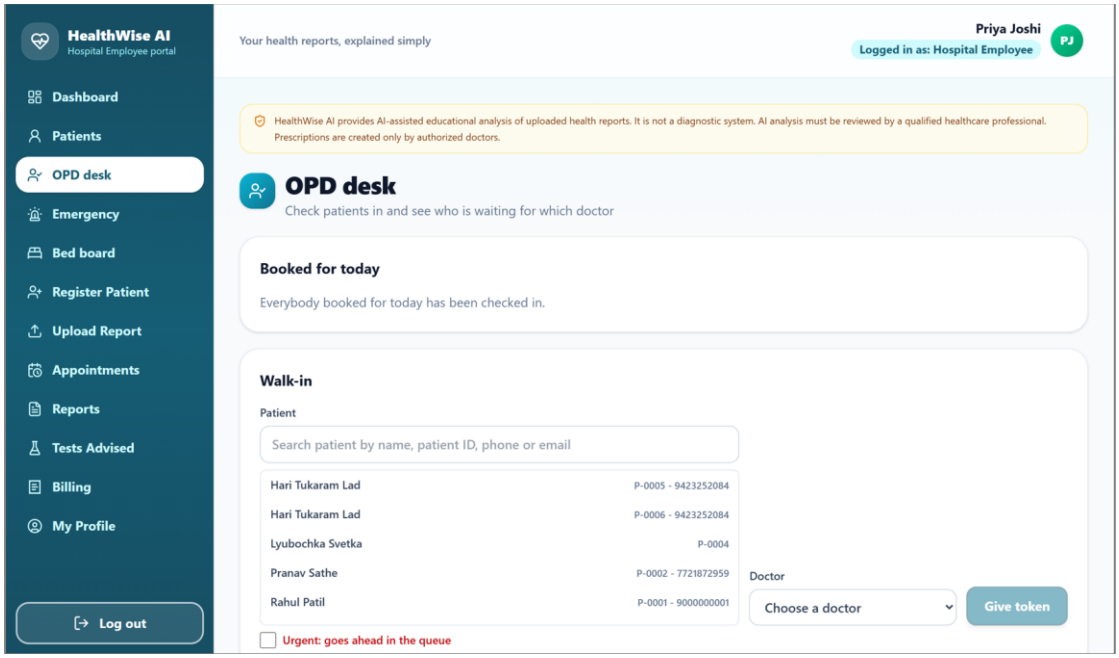


Figure 11.15: OPD desk

11.6 Pharmacy

The pharmacy dashboard (Figure 11.16) shows prescriptions waiting, low stock and expiring items, and offers Sell and make a bill, Give prescribed medicine and Medicine stock.

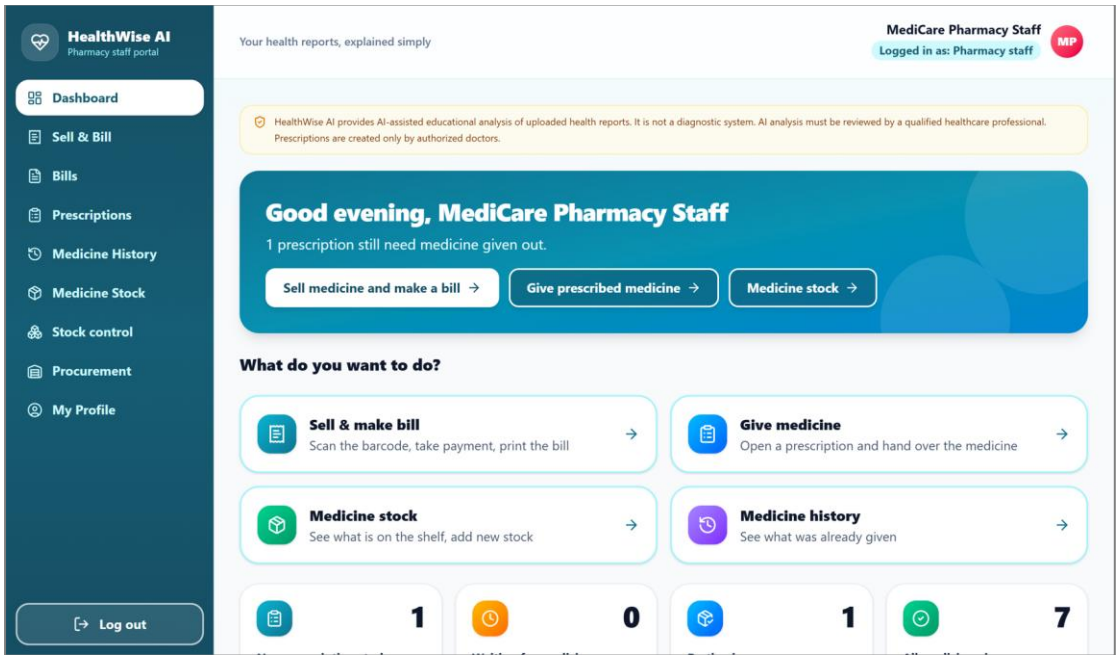


Figure 11.16: Pharmacy dashboard

Prescriptions (Figure 11.17) lists open prescriptions only; a search by patient name, patient identifier or prescription code narrows the list. The button at the foot of a card opens the prescription to give medicine.

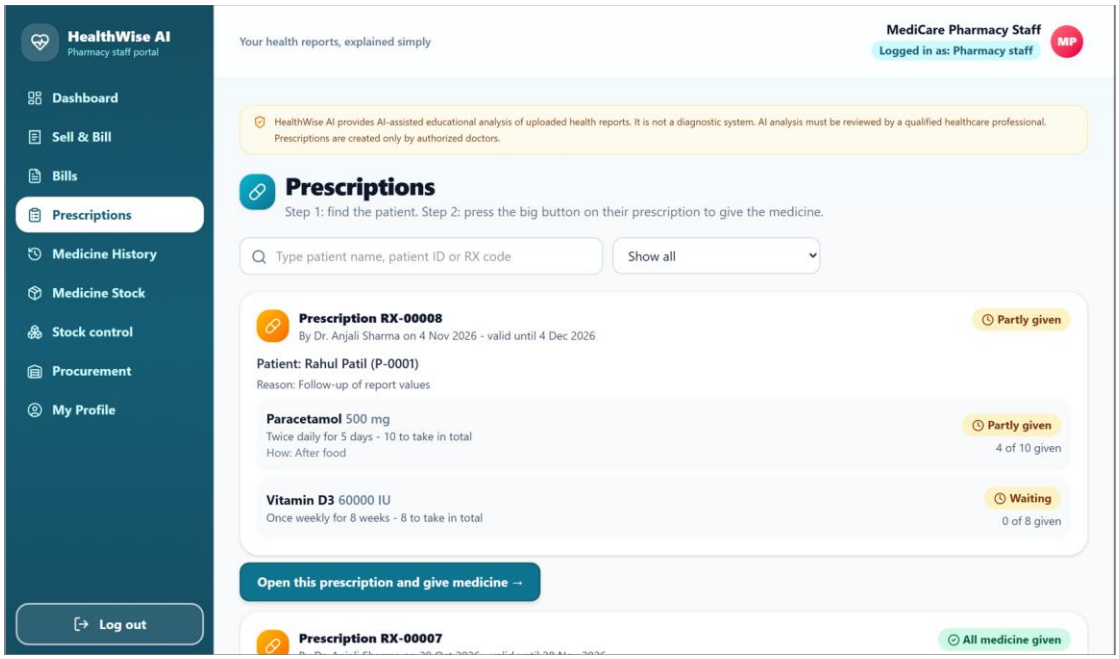


Figure 11.17: Pharmacy prescription list

On the dispensing page (Figure 11.18) the prescription is shown in the Rx-pad layout. Enter the quantity given for each item; the system refuses more than the remaining quantity and takes the stock from the batch that expires first.

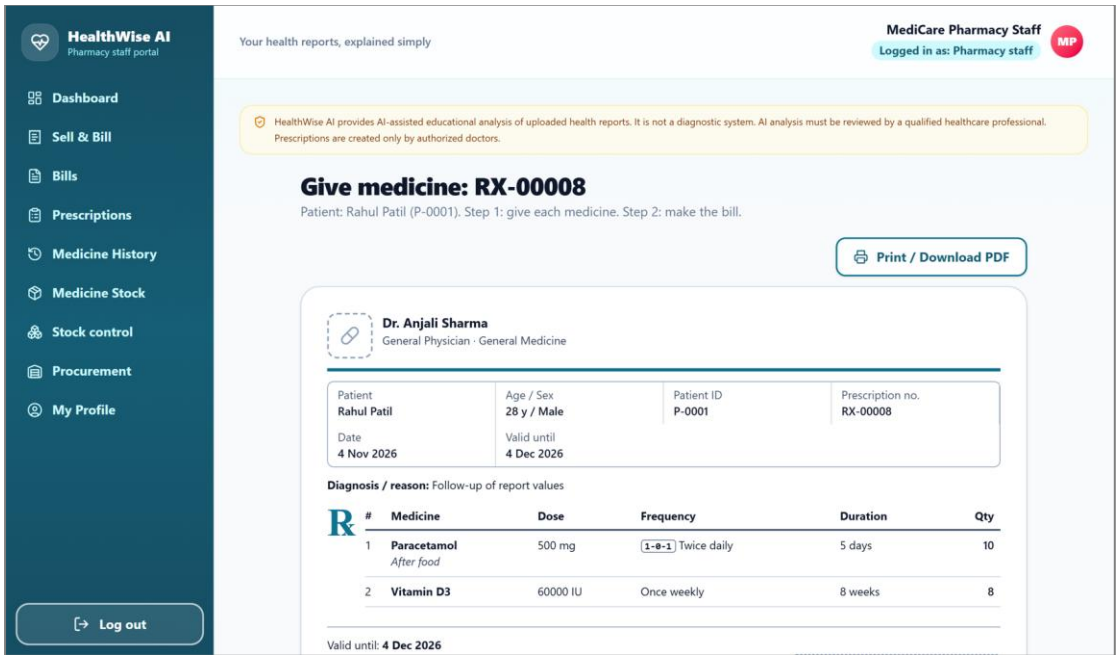


Figure 11.18: Dispensing a prescription

Medicine stock (Figure 11.19) lists batches with quantity, expiry and price; a search accepts a name or a scanned barcode, and buttons change a count or print a barcode label.

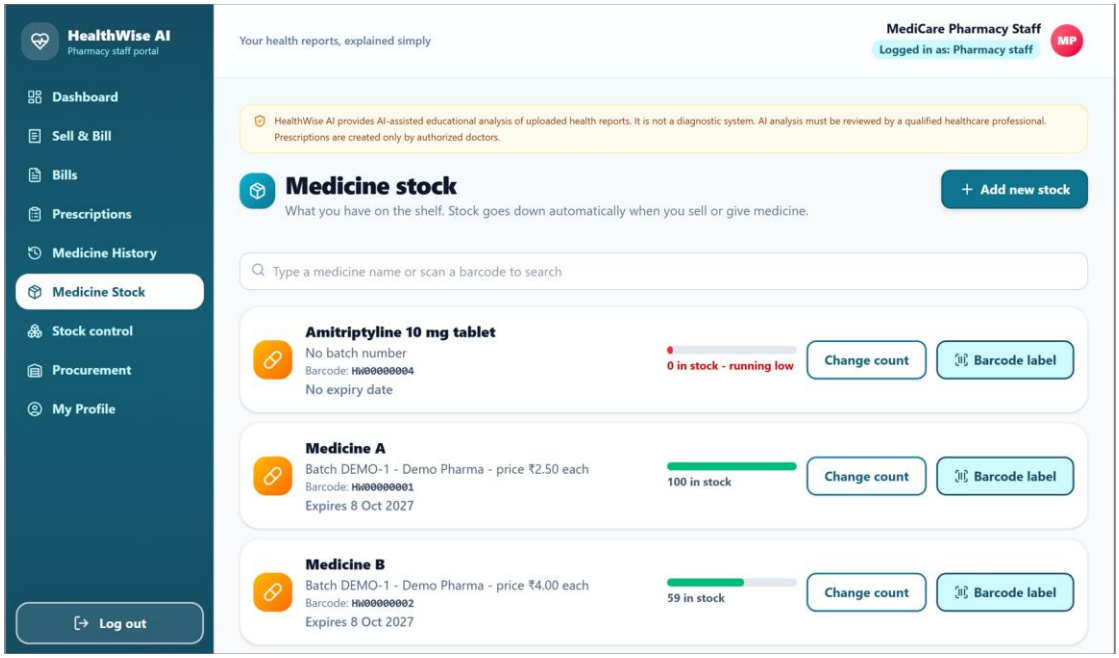


Figure 11.19: Medicine stock

11.7 Laboratory

The laboratory work list (Figure 11.20) shows the tests advised by doctors with counts (to do, urgent, done today, longest wait) and tabs for each stage: waiting for the sample, sample taken, being tested, done and rejected. The screenshot shows the empty state of the demonstration database, in which no test had been advised. When a test is present, the technician collects the sample, enters the result against the hospital's range, and a pathologist approves it in Approvals; a critical value raises an alert.

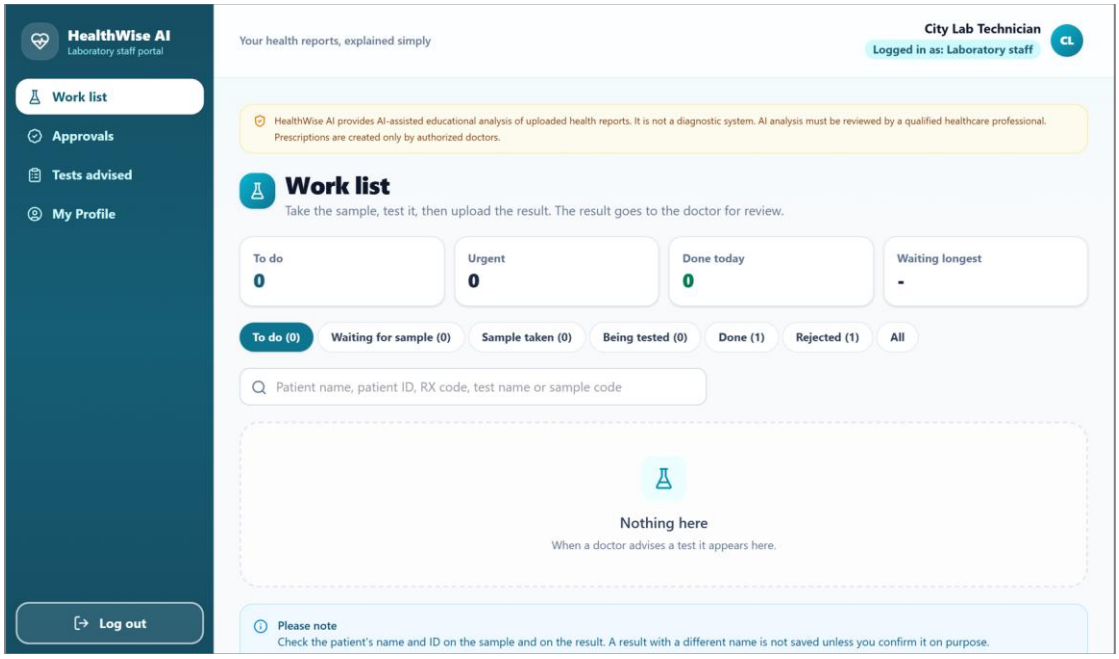


Figure 11.20: Laboratory work list (empty state)

11.8 Administrator

The administrator dashboard (Figure 11.21) shows the number of patients, doctors and reports and offers shortcuts to create staff accounts. The menu leads to People, Doctors, Employees, Pharmacy staff, Laboratory staff, Radiology staff, Nurses, Integrations, Beds, Appointments, Reports (status only), Activity log and Hospital.

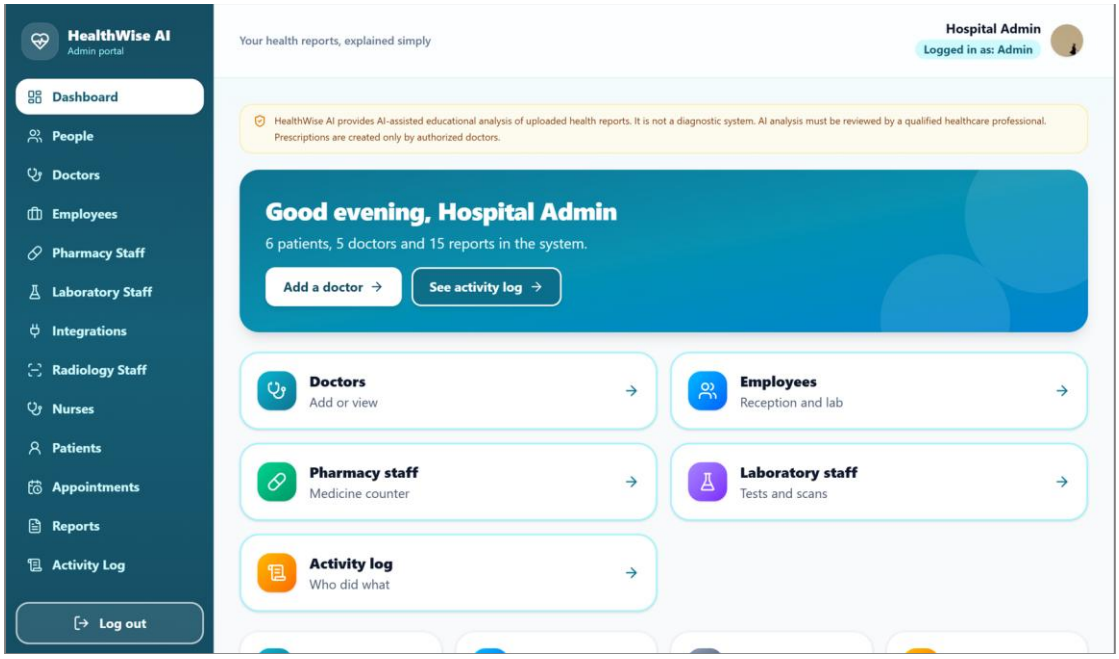


Figure 11.21: Administrator dashboard

People (Figure 11.22) lists everyone with their role and gives buttons to edit, deactivate or delete. Deleting asks for a check first, because a person with records cannot simply be removed. A new account is created from the role pages and receives a temporary password that is shown once.

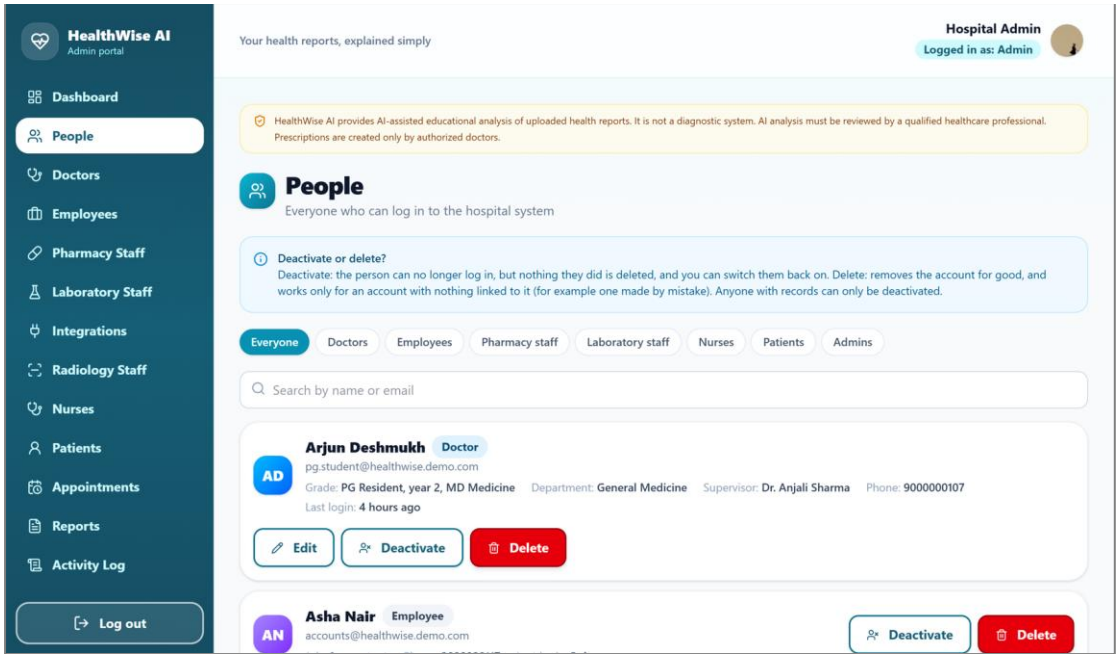


Figure 11.22: People list

The activity log (Figure 11.23) records who did what and when; it can be searched and filtered and cannot be edited.

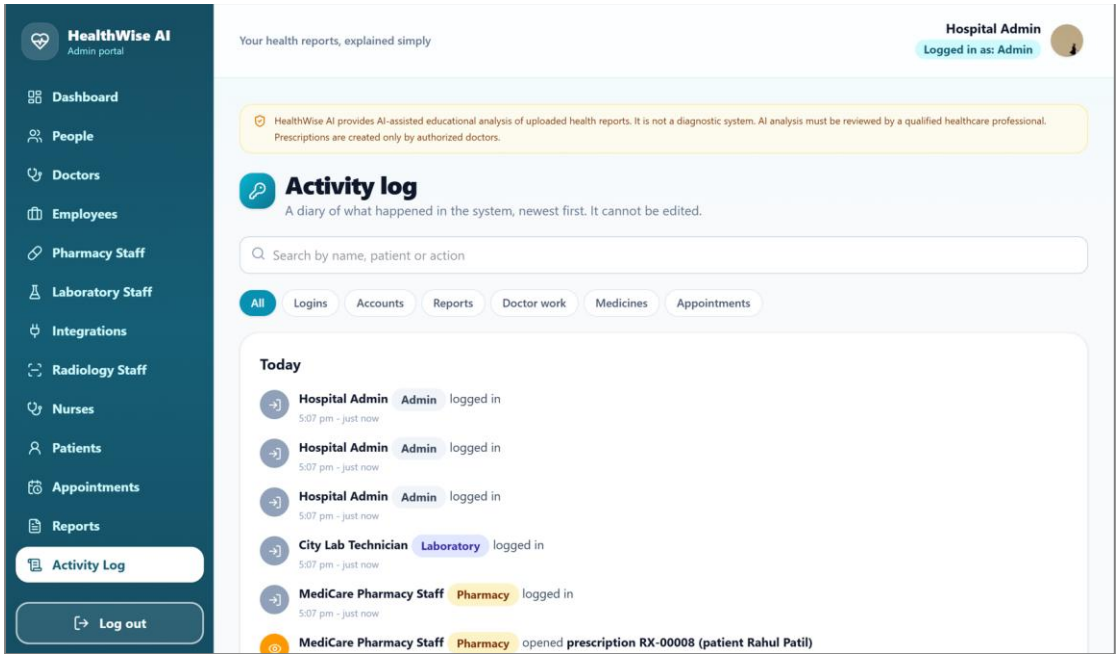


Figure 11.23: Activity (audit) log

Hospital (Figure 11.24) holds the profile used on printed prescriptions, branches, settings, the price list, roles, ward and bed set-up, consultation forms, alert thresholds and emergency access.

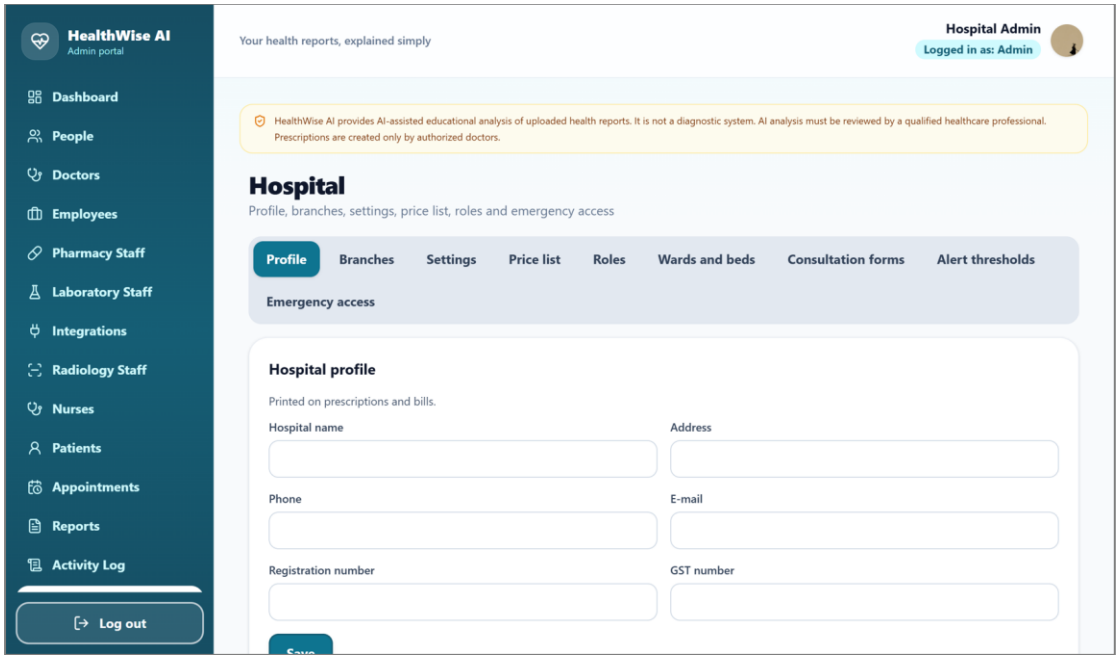


Figure 11.24: Hospital set-up

CHAPTER 12

12. BIBLIOGRAPHY

References are numbered in the order of first citation. Entries marked [verify] could not be checked against the source in this project and must be confirmed by the author before submission. Access dates of online documents are not recorded here and should be added in the format required by the university.

[1] R. S. Evans, "Electronic health records: Then, now, and in the future," Yearbook of Medical Informatics, vol. 25, suppl. 1, pp. S48-S61, 2016. [verify]

[2] World Health Organization, Global Strategy on Digital Health 2020-2025. Geneva, Switzerland: WHO, 2021.

[3] National Health Authority, Government of India, Ayushman Bharat Digital Mission. [Online]. Available: https://abdm.gov.in [verify]

[4] Government of India, The Digital Personal Data Protection Act, 2023 (Act No. 22 of 2023). New Delhi, India, 2023.

[5] HL7 International, HL7 FHIR Release 4. [Online]. Available: https://hl7.org/fhir/R4/

[6] W3C, Web Content Accessibility Guidelines (WCAG) 2.1, W3C Recommendation, Jun. 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[7] OWASP Foundation, OWASP Top 10:2021. [Online]. Available: https://owasp.org/Top10/

[8] OWASP Foundation, OWASP Application Security Verification Standard (ASVS), version 4.0.3. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

[9] FastAPI documentation. [Online]. Available: https://fastapi.tiangolo.com/

[10] React documentation, Meta Open Source. [Online]. Available: https://react.dev/

[11] Pydantic documentation. [Online]. Available: https://docs.pydantic.dev/

[12] SQLAlchemy documentation. [Online]. Available: https://docs.sqlalchemy.org/

[13] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459-9474.

[14] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), Curitiba, Brazil, 2007, pp. 629-633.

[15] B. W. Boehm, Software Engineering Economics. Englewood Cliffs, NJ, USA: Prentice-Hall, 1981.

[16] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2016.

[17] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 8th ed. New York, NY, USA: McGraw-Hill, 2014.

[18] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force, RFC 7519, May 2015.

[19] N. Provos and D. Mazieres, "A future-adaptable password scheme," in Proc. USENIX Annual Technical Conf., Monterey, CA, USA, 1999, pp. 81-91.

[20] Medical device software: Software life cycle processes, IEC 62304:2006, International Electrotechnical Commission, Geneva, Switzerland, 2006.

[21] National Electrical Manufacturers Association, DICOM Standard. [Online]. Available: https://www.dicomstandard.org/